

前言

大家好，我是小林，是这本《大厂后端面试真题》pdf的作者，同时也是小林coding 公众号作者，[图解网站](#)站长，全网粉丝超 40W+，著有《[图解网络](#)》、《[图解系统](#)》、《[图解MySQL](#)》、《[图解 Redis](#)》开源书籍，[Gihutb](#) 开源不到 1 年，斩获 9k+ star，文章阅读量累计超上千万，帮助数万程序员击破八股文的恐惧。

有很多同学一直都没去面试，不知道真实的面试过程中，考察的范围到底是哪些，也总感觉自己有很多技术没有学，一直感觉没有准备到一个满意的状态，越想就越掉头发。

虽然没办法去面试，但是也可以通过其他人的面经来观察一些面试的重点知识。所以这次整理了2024年6月份之前公众号目前发布过的大厂后端面经，大家可以根据面经，去抓住需要重点准备的技术点，希望对同学有帮助！

注意：本文提供的回答都是只是参考回答！不代表一定准确！

后续新的后端面试题真题，可以关注公众号【小林coding】，会持续带来新的后端面经。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

字节面试，有点招架不住啊！

大家好，我是小林。

3 月份很多大厂都开始实习招聘了，近期也有很多同学去面试了，今天分享一位读者的字节实习的面经，读者的技术栈是 C++ 后端。

对于部分问题，我也补充了一些回答，基本上网络、操作系统、mysql、redis 的问题，图解网站的内容都有。

计算机网络

UDP和TCP区别

读者回答：先说了概念一个是面向连接的基于字节流的可靠连接，一个是不需要连接的基于数据报的不可靠传输 然后说了几个小点，比如首部长度、应用场景、服务对象什么的。

小林补充：

TCP 和 UDP 区别：

1. 连接

- TCP 是面向连接的传输层协议，传输数据前先要建立连接。
- UDP 是不需要连接，即刻传输数据。

2. 服务对象

- TCP 是一对一的两点服务，即一条连接只有两个端点。
- UDP 支持一对一、一对多、多对多的交互通信

3. 可靠性

- TCP 是可靠交付数据的，数据可以无差错、不丢失、不重复、按序到达。
- UDP 是尽最大努力交付，不保证可靠交付数据。但是我们可以基于 UDP 传输协议实现一个可靠的传输协议，比如 QUIC 协议，具体可以参见这篇文章：[如何基于 UDP 协议实现可靠传输？](#)

4. 拥塞控制、流量控制

- TCP 有拥塞控制和流量控制机制，保证数据传输的安全性。
- UDP 则没有，即使网络非常拥堵了，也不会影响 UDP 的发送速率。

5. 首部开销

- TCP 首部长度较长，会有一定的开销，首部在没有使用「选项」字段时是 20 个字节，如果使用了「选项」字段则会变长的。
- UDP 首部只有 8 个字节，并且是固定不变的，开销较小。

6. 传输方式

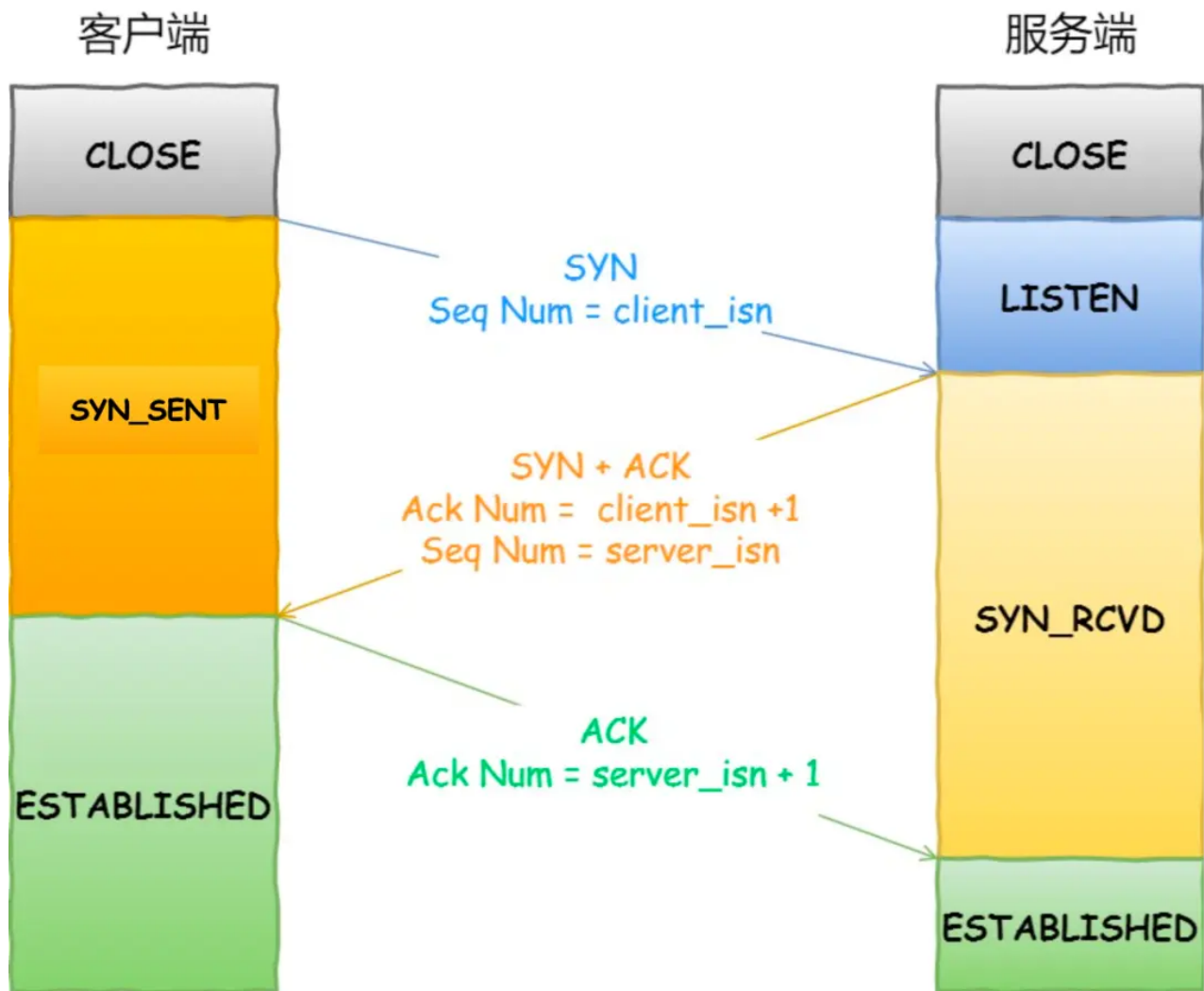
- TCP 是流式传输，没有边界，但保证顺序和可靠。
- UDP 是一个包一个包的发送，是有边界的，但可能会丢包和乱序。

from xiaolincoding.com

还有一个很重要的点：**UDP 的实时性比 TCP 好。**

TCP 工作流程 三次握手 四次挥手

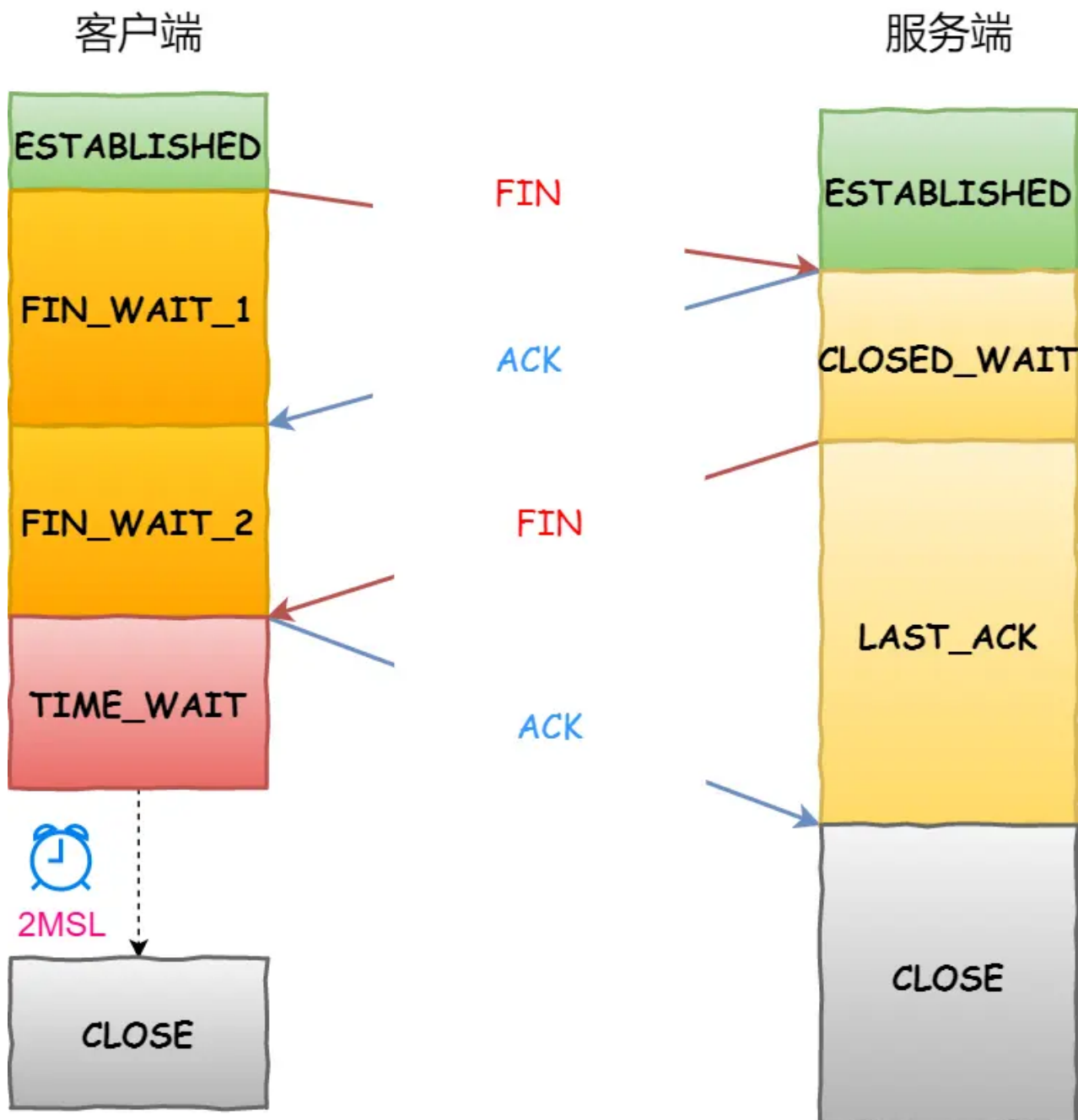
TCP 是面向连接的协议，所以使用 TCP 前必须先建立连接，而**建立连接是通过三次握手来进行的**。三次握手的过程如下图：



- 一开始，客户端和服务端都处于 **CLOSE** 状态。先是服务端主动监听某个端口，处于 **LISTEN** 状态
- 客户端会随机初始化序号 (**client_isn**)，将此序号置于 TCP 首部的「序号」字段中，同时把 **SYN** 标志位置为 **1**，表示 **SYN** 报文。接着把第一个 SYN 报文发送给服务端，表示向服务端发起连接，该报文不包含应用层数据，之后客户端处于 **SYN-SENT** 状态。
- 服务端收到客户端的 **SYN** 报文后，首先服务端也随机初始化自己的序号 (**server_isn**)，将此序号填入 TCP 首部的「序号」字段中，其次把 TCP 首部的「确认应答号」字段填入 **client_isn + 1**，接着把 **SYN** 和 **ACK** 标志位置为 **1**。最后把该报文发给客户端，该报文也不包含应用层数据，之后服务端处于 **SYN-RCVD** 状态。
- 客户端收到服务端报文后，还要向服务端回应最后一个应答报文，首先该应答报文 TCP 首部 **ACK** 标志位置为 **1**，其次「确认应答号」字段填入 **server_isn + 1**，最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 **ESTABLISHED** 状态。
- 服务端收到客户端的应答报文后，也进入 **ESTABLISHED** 状态。

天下没有不散的宴席，对于 TCP 连接也是这样，TCP 断开连接是通过**四次挥手**方式。

双方都可以主动断开连接，断开连接后主机中的「资源」将被释放，四次挥手的过程如下图：



客户端主动关闭连接 —— TCP 四次挥手

- 客户端打算关闭连接，此时会发送一个 TCP 首部 **FIN** 标志位被置为 1 的报文，也即 **FIN** 报文，之后客户端进入 **FIN_WAIT_1** 状态。
- 服务端收到该报文后，就向客户端发送 **ACK** 应答报文，接着服务端进入 **CLOSE_WAIT** 状态。
- 客户端收到服务端的 **ACK** 应答报文后，之后进入 **FIN_WAIT_2** 状态。
- 等待服务端处理完数据后，也向客户端发送 **FIN** 报文，之后服务端进入 **LAST_ACK** 状态。
- 客户端收到服务端的 **FIN** 报文后，回一个 **ACK** 应答报文，之后进入 **TIME_WAIT** 状态。
- 服务端收到了 **ACK** 应答报文后，就进入了 **CLOSE** 状态，至此服务端已经完成连接的关闭。
- 客户端在经过 **2MSL** 一段时间后，自动进入 **CLOSE** 状态，至此客户端也完成连接的关闭。

你可以看到，每个方向都需要一个 **FIN** 和一个 **ACK**，因此通常被称为四次挥手。

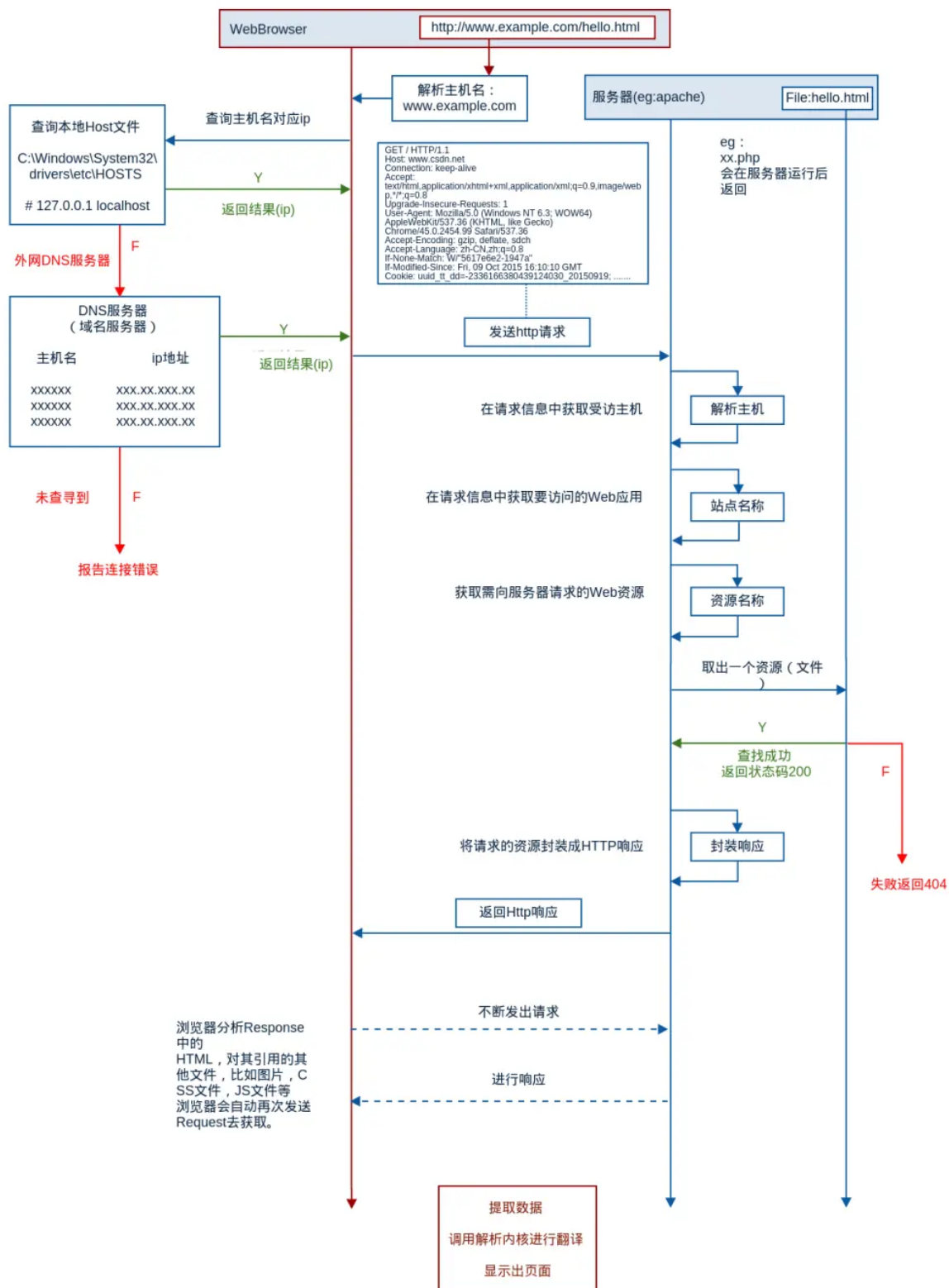
TCP 流量控制、拥塞控制

小林补充：

- 发送方不能无脑的发数据给接收方，要考虑接收方处理能力。如果一直无脑的发数据给对方，但对方处理不过来，那么就会导致触发重发机制，从而导致网络流量的无端的浪费。为了解决这种现象发生，**TCP 提供一种机制可以让「发送方」根据「接收方」的实际接收能力控制发送的数据量，这就是所谓的流量控制。**
- 一般来说，计算机网络都处在一个共享的环境。因此也有可能会因为其他主机之间的通信使得网络拥堵。在网络出现拥堵时，如果继续发送大量数据包，可能会导致数据包时延、丢失等，这时 TCP 就会重传数据，但是一重传就会导致网络的负担更重，于是会导致更大的延迟以及更多的丢包，这个情况就会进入恶性循环被不断地放大....所以，TCP 不能忽略网络上发生的事，它被设计成一个无私的协议，当网络发送拥塞时，TCP 会自我牺牲，降低发送的数据量。于是，就有了拥塞控制，目的就是避免「发送方」的数据填满整个网络。

访问一个网站流程，从http方面

小林补充：



from csdn-

更详细传输层->网络层->数据链路层->路由器的过程，看图解网络->基础篇->键入网址到网页显示期间发生了什么？。

一、前言

二、基础篇

2.1 TCP/IP 网络模型有哪几层？

2.2 键入网址到网页显示，期间发生了什么？

孤单小弟 —— HTTP

真实地址查询 —— DNS

指南好帮手 —— 协议栈

可靠传输 —— TCP

远程定位 —— IP

两点传输 —— MAC

出口 —— 网卡

送别者 —— 交换机

出境大门 —— 路由器

互相扒皮 —— 服务器与客户端

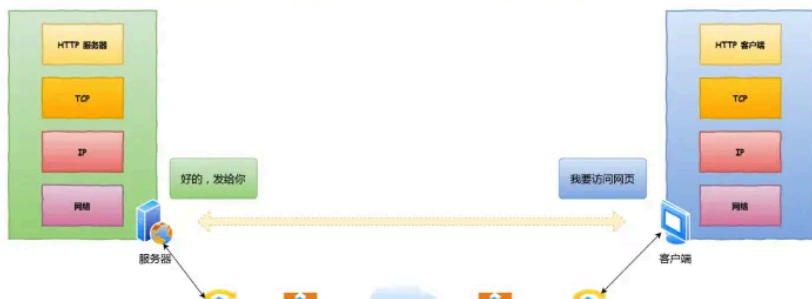
一个数据包且不要脸的威码

2.2 键入网址到网页显示，期间发生了什么？

想必不少小伙伴面试过程中，会遇到「当键入网址后，到网页显示，其间发生了什么」的面试题。

还别说，这问题真挺常问的，前几天坐在我旁边的主管电话面试应聘者的时候，也问了这个问题。

接下来以下图较简单的网络拓扑模型作为例子，探究探究其间发生了什么？



知道ip和port就可以生成tcp连接吗？连接建立的具体流程

读者回答：客户端有对方的ip和端口，然后调用connect()，服务器端也要选择所有ip都能访问。

select、poll、epoll区别

小林补充：

select 实现多路复用的方式是，将已连接的 Socket 都放到一个**文件描述符集合**，然后调用 select 函数将文件描述符集合**拷贝**到内核里，让内核来检查是否有网络事件产生，检查的方式很粗暴，就是通过**遍历**文件描述符集合的方式，当检查到有事件产生后，将此 Socket 标记为可读或可写，接着再把整个文件描述符集合**拷贝**回用户态里，然后用户态还需要再通过**遍历**的方法找到可读或可写的 Socket，然后再对其处理。

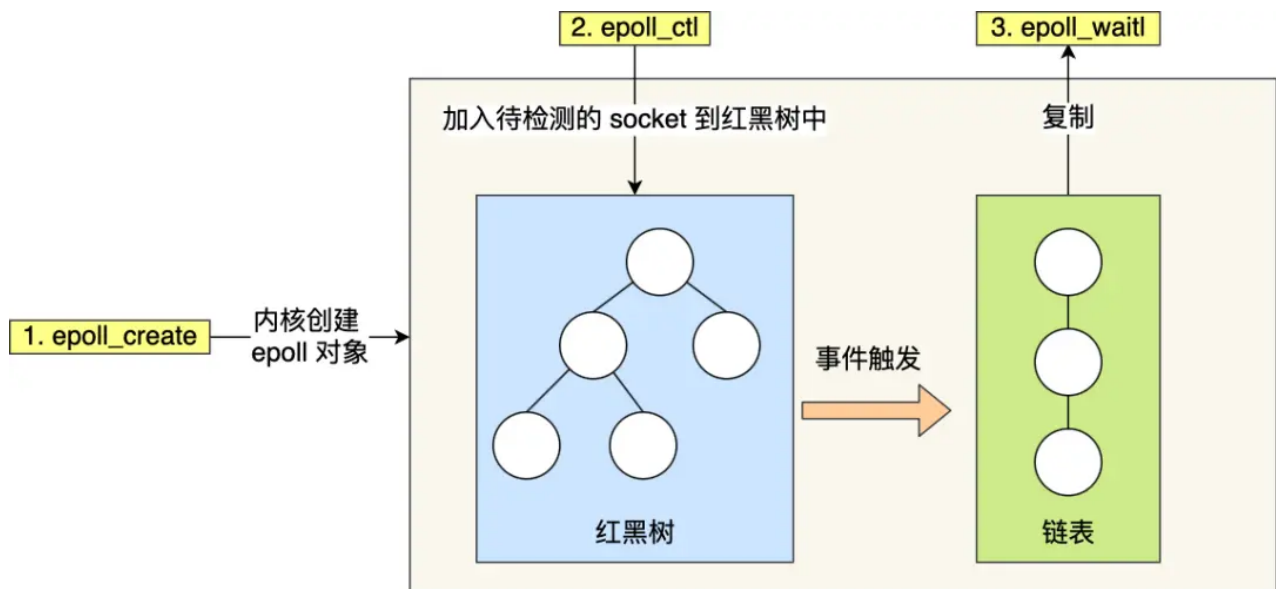
所以，对于 select 这种方式，需要进行 **2 次「遍历」文件描述符集合**，一次是在内核态里，一个次是在用户态里，而且还会发生 **2 次「拷贝」文件描述符集合**，先从用户空间传入内核空间，由内核修改后，再传出到用户空间中。

select 使用固定长度的 Bitmap，表示文件描述符集合，而且所支持的文件描述符的个数是有限制的，在 Linux 系统中，由内核中的 FD_SETSIZE 限制，默认最大值为 **1024**，只能监听 0~1023 的文件描述符。

poll 不再用 Bitmap 来存储所关注的文件描述符，取而代之用动态数组，以链表形式来组织，突破了 select 的文件描述符个数限制，当然还会受到系统文件描述符限制。

但是 poll 和 select 并没有太大的本质区别，都是使用「**线性结构**」存储进程关注的 **Socket 集合**，因此都需要遍历文件描述符集合来找到可读或可写的 **Socket**，时间复杂度为 **O(n)**，而且也需要在用户态与内核态之间**拷贝**文件描述符集合，这种方式随着并发数上来，性能的损耗会呈指数级增长。

epoll 通过两个方面解决了 select/poll 的问题。



1、epoll 在内核里使用「红黑树」来关注进程所有待检测的 Socket，红黑树是个高效的数据结构，增删改一般时间复杂度是 $O(\log n)$ ，通过对这棵黑红树的管理，不需要像 select/poll 在每次操作时都传入整个 Socket 集合，减少了内核和用户空间大量的数据拷贝和内存分配。

2、epoll 使用事件驱动的机制，内核里维护了一个「链表」来记录就绪事件，只将有事件发生的 Socket 集合传递给应用程序，不需要像 select/poll 那样轮询扫描整个集合（包含有和无事件的 Socket），大大提高了检测的效率。

epoll 具体工作流程

读者回答：先建立epoll对象，然后添加事件，然后wait等待事件发生 但他好像不是很满意这个回答。

小林补充：

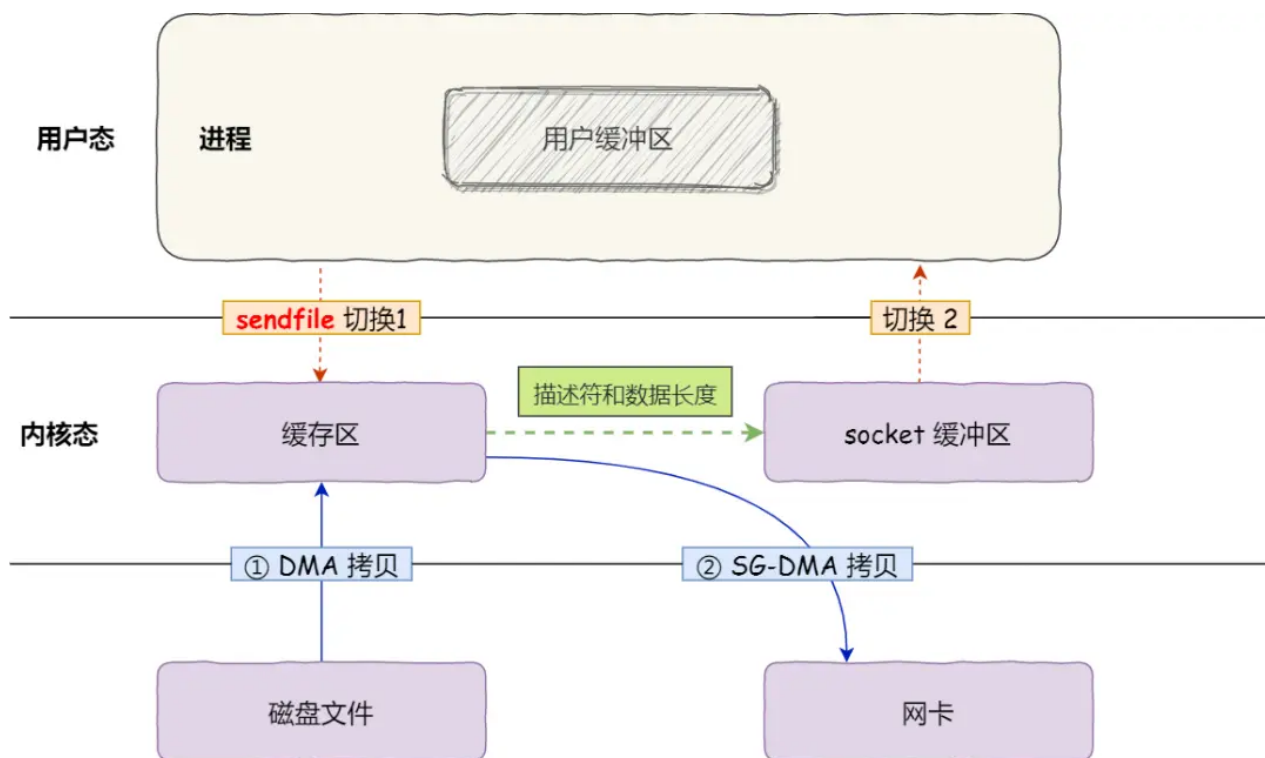
1. 通过epoll_create创建epoll对象，此时epoll对象的内核结构包含就绪链表和红黑树，就绪队列是用于保存所有读写事件到来的socket。红黑树用于保存所有待检测的socket。
2. 通过 epoll_ctl 将待检测的socket，加入到红黑树中，并注册一个事件回调函数，当有事件到来的之后，会调用这个回调函数，进而通知到 epoll 对象。
3. 调用 epoll_wait 等待事件的发生，当内核检测到事件发生后，调用该socket注册的回调函数，执行回调函数就能找到socket对应的epoll对象，然后会将事件加入到epoll对象的就绪队列中，最后将就绪队列返回给应用层。

零拷贝

读者回答：记混了，回答了DMA，说主要是为了减少拷贝操作，举了使用零拷贝的项目，比如nginx。

小林补充：

sendfile 系统调用实现了零拷贝技术，零拷贝技术的文件传输方式相比传统文件传输的方式，减少了 2 次上下文切换和数据拷贝次数，只需要 2 次上下文切换和数据拷贝次数，就可以完成文件的传输，而且 2 次的数据拷贝过程，都不需要通过 CPU，2 次都是由 DMA 来搬运，使用零拷贝的项目有nginx、kafka。



Redis

redis 优点、场景

redis 优点快。

Redis 是一种基于内存的数据库，对数据的读写操作都是在内存中完成，因此读写速度非常快，适用于服务器与数据库间的缓存，还可以用来做分布式锁、秒杀、消息队列。

redis 为什么快？

之所以 Redis 采用单线程那么快，有如下几个原因：

- Redis 的大部分操作都在内存中完成，并且采用了高效的数据结构，因此 Redis 瓶颈可能是机器的内存或者网络带宽，而并非 CPU，既然 CPU 不是瓶颈，那么自然就采用单线程的解决方案了；
- Redis 采用单线程模型可以避免多线程之间的竞争，省去了多线程切换带来的时间和性能上的开销，而且也不会导致死锁问题。
- Redis 采用了 I/O 多路复用机制处理大量的客户端 Socket 请求，IO 多路复用机制是指一个线程处理多个 IO 流，就是我们经常听到的 select/epoll 机制。简单来说，在 Redis 只运行单线程的情况下，该机制允许内核中，同时存在多个监听 Socket 和已连接 Socket。内核会一直监听这些 Socket 上的连接请求或数据请求。一旦有请求到达，就会交给 Redis 线程处理，这就实现了一个 Redis 线程处理多个 IO 流的效果。

持久化

Redis 的读写操作都是在内存中，所以 Redis 性能才会高，但是当 Redis 重启后，内存中的数据就会丢失，那为了保证内存中的数据不会丢失，Redis 实现了数据持久化的机制，这个机制会把数据存储到磁盘，这样在 Redis 重启就能够从磁盘中恢复原有的数据。

Redis 共有三种数据持久化的方式：

- **AOF 日志**：每执行一条写操作命令，就把该命令以追加的方式写入到一个文件里；
- **RDB 快照**：将某一时刻的内存数据，以二进制的方式写入磁盘；
- **混合持久化方式**：Redis 4.0 新增的方式，集成了 AOF 和 RDB 的优点；

分布式锁的实现和四个特性

小林补充：

- **互斥性**：锁的目的是获取资源的使用权，所以只让一个竞争者持有锁，这一点要尽可能保证；
- **安全性**：避免死锁情况发生。当一个竞争者在持有锁期间内，由于意外崩溃而导致未能主动解锁，其持有的锁也能够被正常释放，并保证后续其它竞争者也能加锁；
- **对称性**：同一个锁，加锁和解锁必须是同一个竞争者。不能把其他竞争者持有的锁给释放了，这又称为锁的可重入性。
- **可靠性**：需要有一定程度的异常处理能力、容灾能力。

一条语句可以完成加锁操作吗？

读者回答：可以，但是他一直反问，让我有点动摇

小林补充：

可以的。

Redis 的 SET 命令有个 NX 参数可以实现「key不存在才插入」，所以可以用它来实现分布式锁：

- 如果 key 不存在，则显示插入成功，可以用来表示加锁成功；
- 如果 key 存在，则会显示插入失败，可以用来表示加锁失败。

基于 Redis 节点实现分布式锁时，对于加锁操作，我们需要满足三个条件。

- 加锁包括了读取锁变量、检查锁变量值和设置锁变量值三个操作，但需要以原子操作的方式完成，所以，我们使用 SET 命令带上 NX 选项来实现加锁；
- 锁变量需要设置过期时间，以免客户端拿到锁后发生异常，导致锁一直无法释放，所以，我们在 SET 命令执行时加上 EX/PX 选项，设置其过期时间；
- 锁变量的值需要能区分来自不同客户端的加锁操作，以免在释放锁时，出现误释放操作，所以，我们使用 SET 命令设置锁变量值时，每个客户端设置的值是一个唯一值，用于标识客户端；

满足这三个条件的分布式命令如下：

```
SET lock_key unique_value NX PX 10000
```

- lock_key 就是 key 键；
- unique_value 是客户端生成的唯一的标识，区分来自不同客户端的锁操作；
- NX 代表只在 lock_key 不存在时，才对 lock_key 进行设置操作；
- PX 10000 表示设置 lock_key 的过期时间为 10s，这是为了避免客户端发生异常而无法释放锁。

同一个用户id两次加锁会怎么样？发生死锁了怎么解决？

- 第二次加锁时，这种情况不算他成功，不引入额外复杂性。
- 死锁有过期时间兜底。

MySQL

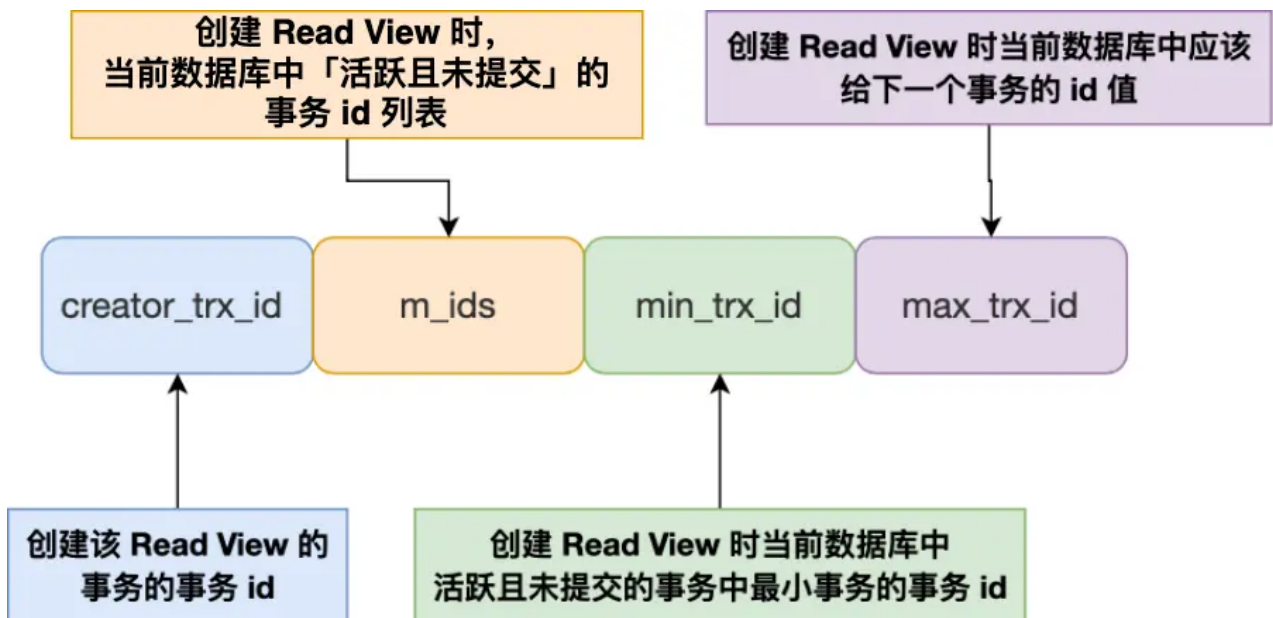
MVCC 概念？如何实现？

小林补充：

我们需要了解两个知识：

- Read View 中四个字段作用；
- 聚簇索引记录中两个跟事务有关的隐藏列；

那 Read View 到底是个什么东西？



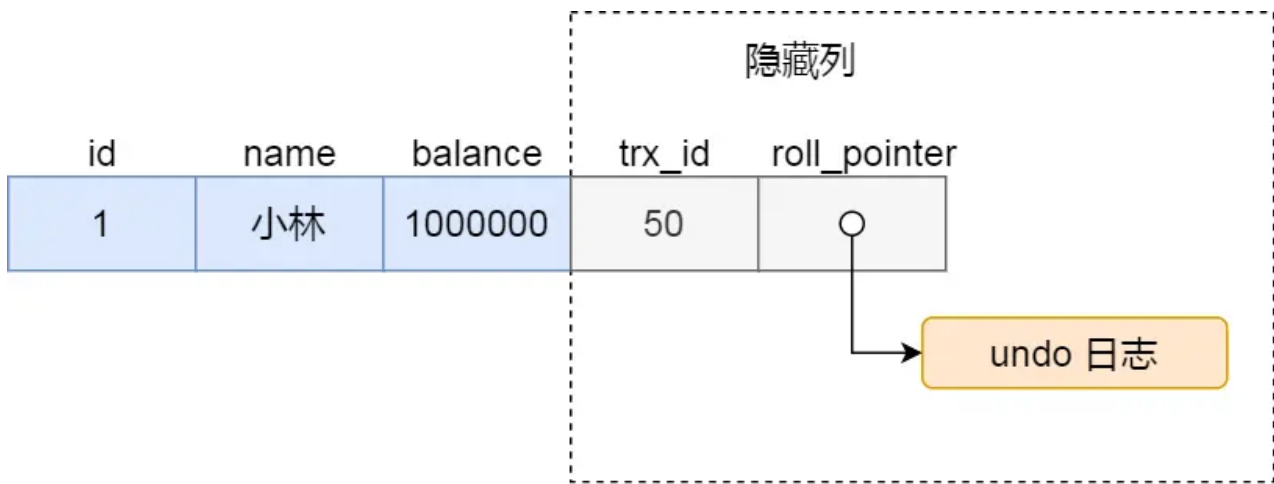
img

Read View 有四个重要的字段：

- **m_ids**：指的是在创建 Read View 时，当前数据库中「活跃事务」的事务 **id** 列表，注意是一个列表，“活跃事务”指的就是，启动了但还没提交的事务。
- **min_trx_id**：指的是在创建 Read View 时，当前数据库中「活跃事务」中事务 **id** 最小的事务，也就是 **m_ids** 的最小值。
- **max_trx_id**：这个并不是 **m_ids** 的最大值，而是创建 **Read View** 时当前数据库中应该给下一个事务的 **id** 值，也就是全局事务中最大的事务 **id** 值 + 1；
- **creator_trx_id**：指的是创建该 **Read View** 的事务的事务 **id**。

知道了 Read View 的字段，我们还需要了解聚簇索引记录中的两个隐藏列。

假设在账户余额表插入一条小林余额为 100 万的记录，然后我把这两个隐藏列也画出来，该记录的整个示意图如下：

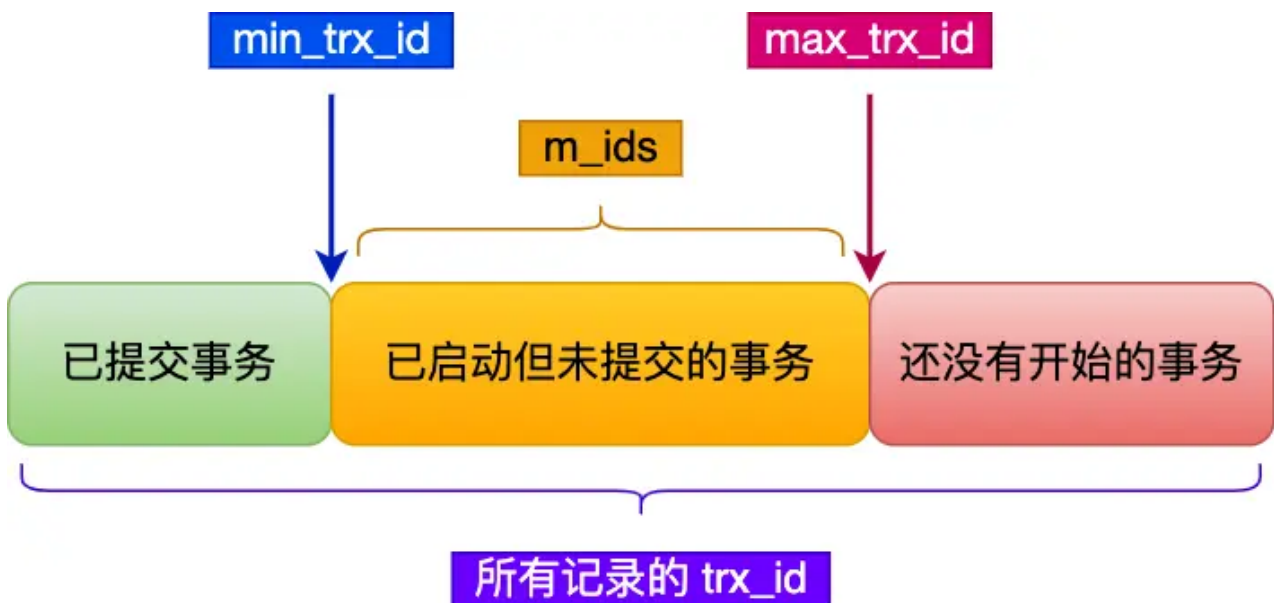


图片

对于使用 InnoDB 存储引擎的数据库表，它的聚簇索引记录中都包含下面两个隐藏列：

- `trx_id`，当一个事务对某条聚簇索引记录进行改动时，就会把该事务的事务 `id` 记录在 `trx_id` 隐藏列里；
- `roll_pointer`，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 `undo 日志` 中，然后这个隐藏列是个指针，指向每一个旧版本记录，于是就可以通过它找到修改前的记录。

在创建 Read View 后，我们可以将记录中的 `trx_id` 划分这三种情况：



img

一个事务去访问记录的时候，除了自己的更新记录总是可见之外，还有这几种情况：

- 如果记录的 `trx_id` 值小于 Read View 中的 `min_trx_id` 值，表示这个版本的记录是在创建 Read View 前已经提交的事务生成的，所以该版本的记录对当前事务可见。
- 如果记录的 `trx_id` 值大于等于 Read View 中的 `max_trx_id` 值，表示这个版本的记录是在创建 Read View 后才启动的事务生成的，所以该版本的记录对当前事务不可见。
- 如果记录的 `trx_id` 值在 Read View 的 `min_trx_id` 和 `max_trx_id` 之间，需要判断 `trx_id` 是否在 `m_ids` 列表中：
 - 如果记录的 `trx_id` 在 `m_ids` 列表中，表示生成该版本记录的活跃事务依然活跃着（还没提交事务），所以该版本的记录对当前事务不可见。

- 如果记录的 `trx_id` 不在 `m_ids` 列表中，表示生成该版本记录的活跃事务已经被提交，所以该版本的记录对当前事务可见。

这种通过「版本链」来控制并发事务访问同一个记录时的行为就叫 **MVCC**（多版本并发控制）。

可重复读概念

可重复读（*repeatable read*），指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，**MySQL InnoDB 引擎**的默认隔离级别；

可重复读下，快照是在什么时候生成的，是事务启动时，还是语句执行前

读者回答：死记硬背八股了，我回答事务启动时。

小林补充：

在 MySQL 有两种开启事务的命令，分别是：

- 第一种：begin/start transaction 命令；
- 第二种：start transaction with consistent snapshot 命令；

这两种开启事务的命令，创建 read view 的时机是不同的：

- 执行了 begin/start transaction 命令后，并不会创建 read view，只有在第一次执行 select 语句后，才会创建 read view。
- 执行了 start transaction with consistent snapshot 命令，就会马上创建 read view。

可重复读下，执行两个select语句，会生成几个快照？

读者回答：上一个问题引申的，我回答不知道。

小林补充：只会生成一次 read view。

可重复读下场景题

问题：他给的场景是 先select value where id = 1的记录此时value = 1，然后update value = 2 where id = 1，然后再select value where id = 1，查询的结果是什么，此时事务还未提交。

读者回答：回答了 value 是 1。

小林补充：value 是2，同一个事务的所有更新操作，都是可见的。事务隔离性，隔离的是其他事务，不隔离自己人。

C++

1. 指针与引用区别

- 正常回答

1. 智能指针

- 正常回答

1. 析构函数什么时候调用

- 正常回答

1. 实际运行中泄漏如何快速定位

- valgrind 内存泄漏检测工具

算法

1. Top K问题 如何实现热搜排行榜

- 追问：用大顶推还是小顶堆
- 追问：插入数据时结构变化

1. 手撕 反转链表

最后，可以看到，其实校招面试过程中，编程语言考察的占比是比较少的，大部分都是问操作系统、网络、数据库、算法。

你们觉得这次面试难度如何？



图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

① 关注公众号回复「**图解**」
获取图解系列 PDF

② 关注公众号回复「**加群**」
拉你进百人技术交流群

扫一扫，关注「小林coding」公众号

腾讯一面，没有稳住，凉了！

大家好，我是小林。

春招开展也有 1 个多月了，这段时间多给大家分享真实的面经题目。

今天这个一位读者腾讯 go 暑期实习的一面的面经。

你们学校里面学的是 go 吗，还是学的 C

答：项目中用的。一开始用的 java，后来做新项目时通过调研选择了 go 并选择深入学习下去。

你能简单说一下里操作系统保护模式和实模式，区别在哪

读者答：（1）实模式早期的那种处理器的一个工作模式，然后他只能使用一些低端的中断处理方式，很难实现一些复杂的功能。（2）保护模式的话它是相对于那个实模式来说它是一种更高级别的工作模式，它能访问的内存也更多，然后它也支持一些就是什么特权就是权限的一些东西比方说内、内核态和用户态的一些东西，更加灵活和安全吧。

小林补充：

其实问这个问题的目的，是为了引出虚拟内存，所以他后面都往虚拟内存方向问了。

- **实模式**将整个物理内存看成分段的区域，程序代码和数据位于不同区域，系统程序 and 用户程序并没有区别对待，而且每一个指针都是指向实际的物理地址。这样一来，用户程序的一个指针如果指向了系统程序区域或其他用户程序区域，并修改了内容，那么对于这个被修改的系统程序或用户程序，其后果就很可能是灾难性的。再者，随着软件的发展，1M的寻址空间已经远远不能满足实际的需求了。最后，对处理器多任务支持需求也日益紧迫，所有这些都促使新技术的出现。
- 为了克服实模式下的内存非法访问问题，并满足飞速发展的内存寻址和多任务需求，处理器厂商开发出**保护模式**。在保护模式中，除了内存寻址空间大大提高；提供了硬件对多任务的支持；**物理内存地址也不能直接被程序访问，程序内部的地址(虚拟地址)要由操作系统转化为物理地址去访问，程序对此一无所知**。至此，进程(程序的运行态)有了严格的边界，任何其他进程根本没有办法访问不属于自己的物理内存区域，甚至在自己的虚拟地址范围内也不是可以任意访问的，因为有一些虚拟区域已经被放进一些公共系统运行库。这些区域也不能随便修改，若修改就会有出现linux中的段错误，或Windows中的非法内存访问对话框。

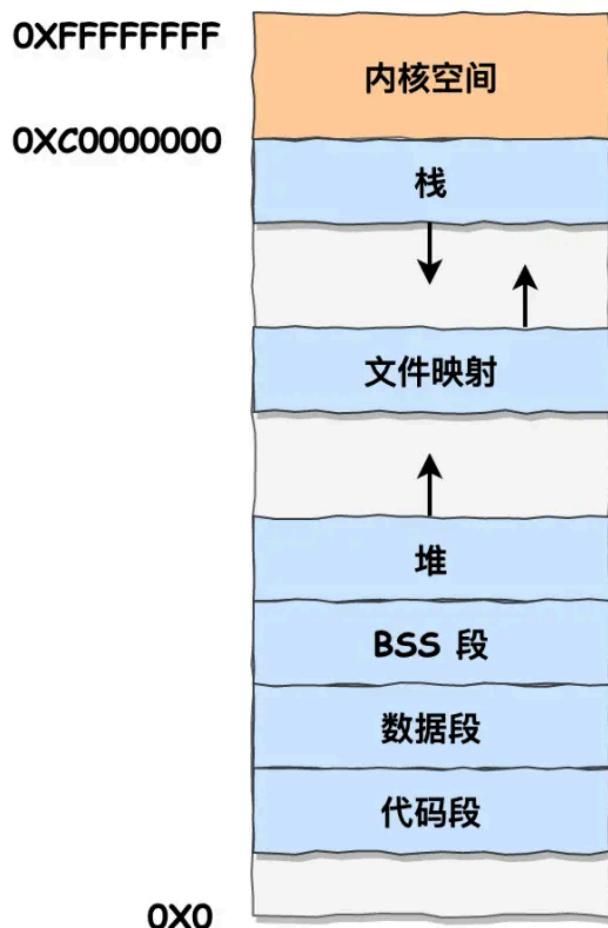
虚拟内存，它的内存布局大概是什么样子的

读者答：（1）首先这个程序在运行的时候他使用的那个地址是虚拟内存的地址，然后虚拟内存和实际

物理内存它是有的地址上是有一个映射的。（2）然后关于这个虚拟内存的布局的话，可以根据就在我在学 c 语言的时候，我们就知道这个里面有什么比方说代码段、数据段，还有什么堆空间站空间这些东西，应该是在分别对应于不同的内存地址中。

小林补充：

我们看看用户空间分布的情况，以 32 位系统为例，我画了一张图来表示它们的关系：



虚拟内存空间划分

通过这张图你可以看到，用户空间内存，从低到高分别是 6 种不同的内存段：

- 代码段，包括二进制可执行代码；
- 数据段，包括已初始化的静态常量和全局变量；
- BSS 段，包括未初始化的静态变量和全局变量；
- 堆段，包括动态分配的内存，从低地址开始向上增长；
- 文件映射段，包括动态库、共享内存等，从低地址开始向上增长。
- 栈段，包括局部变量和函数调用的上下文等。栈的大小是固定的，一般是 8 MB。当然系统也提供了参数，以便我们自定义大小；

上图中的内存布局可以看到，代码段下面还有一段内存空间的（灰色部分），这一块区域是「保留区」，之所以要有保留区这是因为在大多数的系统里，我们认为比较小数值的地址不是一个合法地址，例如，我们通常在 C 的代码里会将无效的指针赋值为 NULL。因此，这里会出现一段不可访问的内存保留区，防止程序因为出现 bug，导致读或写了一些小内存地址的数据，而使得程序跑飞。

在这 7 个内存段中，堆和文件映射段的内存是动态分配的。比如说，使用 C 标准库的 `malloc()` 或者 `mmap()`，就可以分别在堆和文件映射段动态分配内存。

怎么申请堆空间

读者答：用 c 语言的话一般是用 malloc 函数去去申请，如果是 C++ 的话一般是用 new 去申请。而栈空间的话程序的临时变量就普通声明的时候他就是用的是栈空间。

malloc 是一个系统调用么

读者答：应该是吧

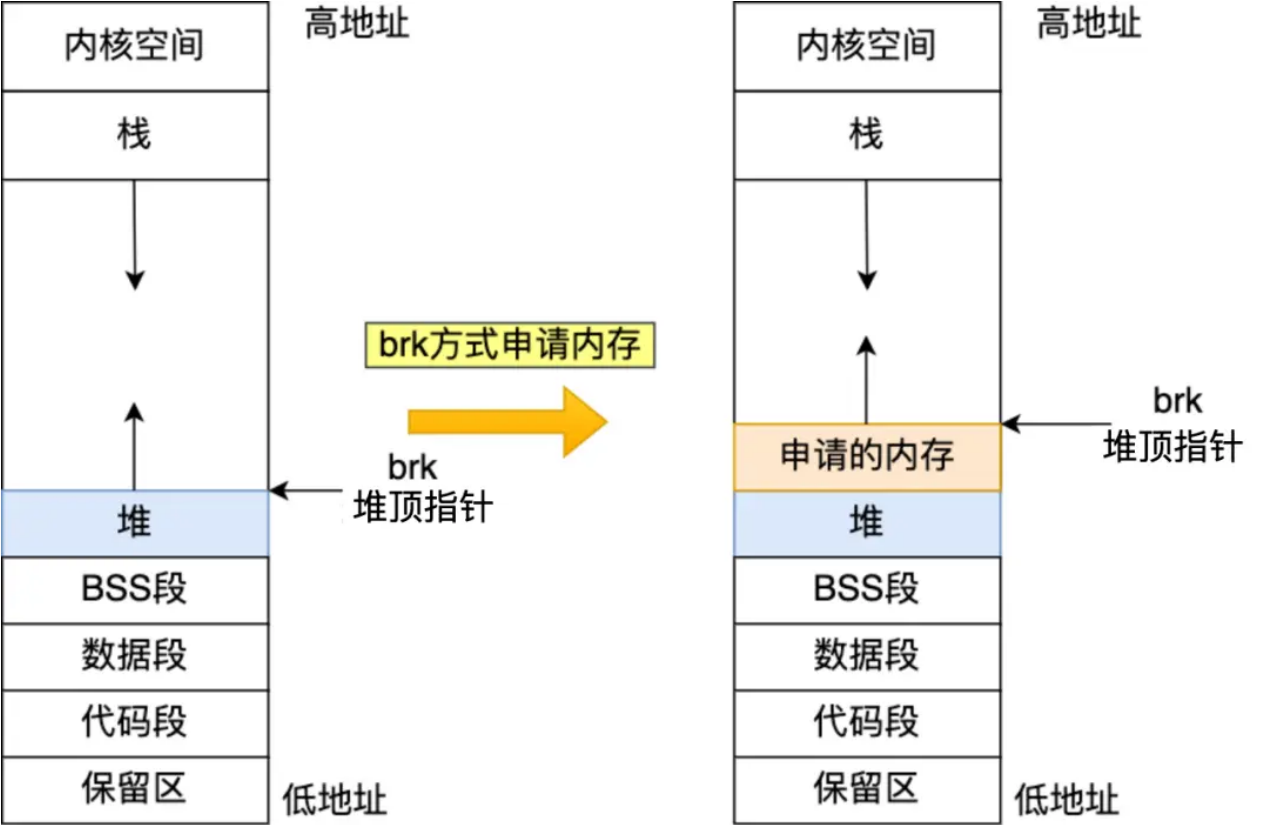
小林补充：

实际上，malloc() 并不是系统调用，而是 C 库里的函数，用于动态分配内存。

malloc 申请内存的时候，会有两种方式向操作系统申请堆内存。

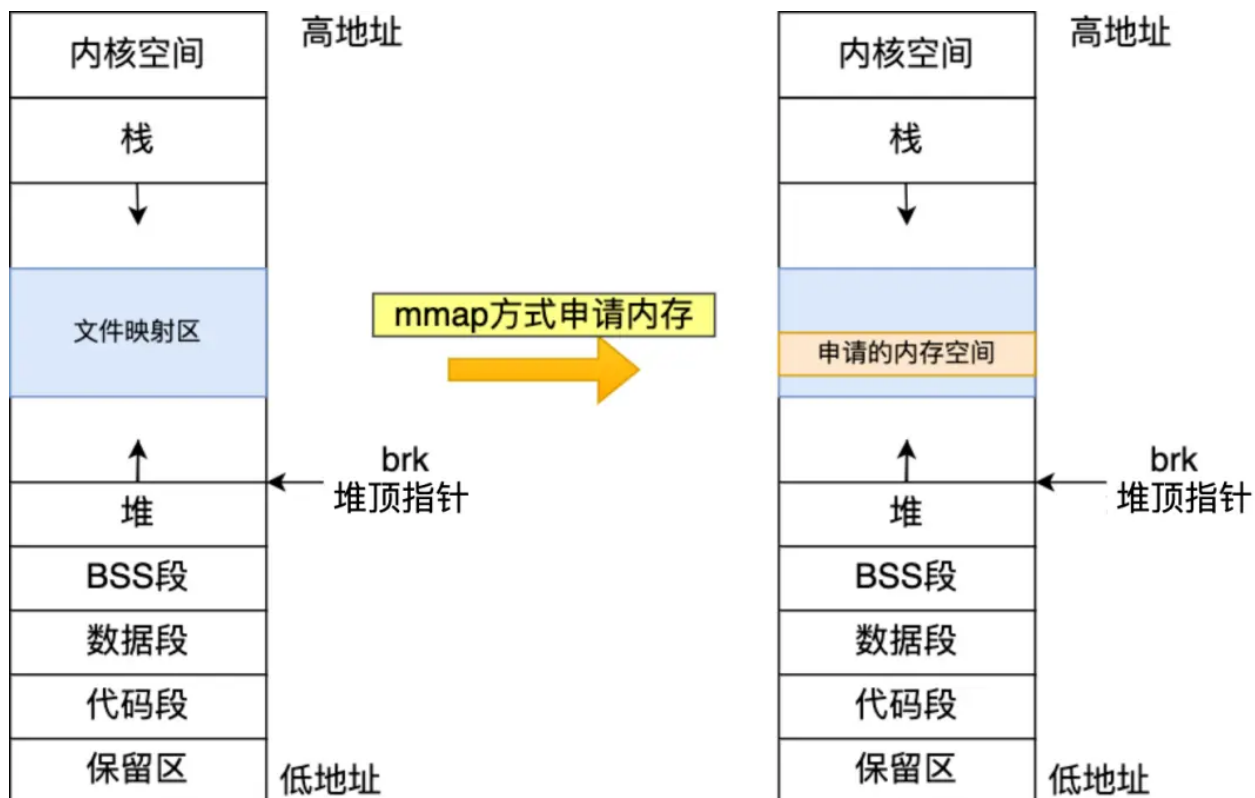
- 方式一：通过 brk() 系统调用从堆分配内存
- 方式二：通过 mmap() 系统调用在文件映射区域分配内存；

方式一实现的方式很简单，就是通过 brk() 函数将「堆顶」指针向高地址移动，获得新的内存空间。如下图：



img

方式二通过 mmap() 系统调用中「私有匿名映射」的方式，在文件映射区分配一块内存，也就是从文件映射区“偷”了一块内存。如下图：



img

栈空间地址增长顺序（从低到高还是从高到低）

没答好，答案反复改了好几次，先说从高到底，又改成从低到高，然后又改成从高到低。

读者答：从高到低，然后站顶个指针他的他在移动的时候他会越变越小，他始终是一个最小的值。

小林补充：

栈空间地址增长顺序是高 --> 底。



go 里面有协程，操作系统里面有进程和线程，简单说一下他们的区别

读者答：（1）从操作系统层面，分线程和进程，在操作系统中没有协程的概念。进程是当前运行的整个程序，有自己的空间和数据资源。线程是进程的一部分，一个进程可以开很多个线程，但是这些线程都共享同一块内存地址（应该是资源当时太紧张说错了）。（2）协程是 go 提出来的概念，是 go 程序运行时维护的一个轻量级的线程，go 有 GMP 模型，M 对应着线程，p 是个调度机器，g 是协程。在程序运行的时候通过 p 把协程调度到 M 线程上去。线程执行完毕后，会让 p 再调度新的协程到线程上执行。

小林补充：

线程和协程区别，要从栈的大小、调度、切换这三个角度来讲。

进程可以理解为一个动态的程序，进程是操作系统资源分配的基本单位，而线程是操作系统调度的基本单位，进程独占一个虚拟内存空间，而进程里的线程共享一个进程虚拟内存空间。线程的粒度更小

协程可以理解为用户态线程，跟线程的区别主要有三个方面

1. 大小，协程到栈为2k，可以动态扩容，而线程大小为2m,协程更轻量
2. 线程切换需要用户态到内核态的切换，而协程的切换不用，只在用户态完成，切换消耗更小
3. 线程的调度由操作系统完成，而协程的调度有运行时的调度器完成

go 里的空结构体可以用来做什么事

我一开始只想到 channel，面试官提醒说可不可以实现个集合容器我又结合 map 说了一下

答：空结构体在使用的时候表示它的类型，然后这个东西一般在 channel 里面比较常用，就是说用它来表示表示你需要用的数据类型。你在同步操作时，需要的一个具体的一个元素的类型，但是

因为你只要保持同步操作，而不是说真的要传数据所以说我们只需要用个空的这个结构体就可以了。

追问：那这个空结构体会占内存空间吗

答：应该不会占内存空间。

面试官提示：还有一个用法就是，如果在 go 里面要去实现一个 set 就是一个集合的话，如果用空结构体的话应该是怎么去实现？可以和其他的数据结构一起去用。

答：比方说你用 map 去实现一个 set 集合，map 不是 key 和 value 对应吗，我想判断某个元素是否出现，我把这个元素的类型作为他的 key，然后他如果出现的话我需要往 map 里插入，但是我只知道我只需要知道的是他出现过，所以说他的那个值 value 类型就可以设置为空就用 Struct，这样就可以通过一个 map 实现一个集合的类型，然后要做查询操作的话你就是使用就是从 map 中取值判断这个值是否存在就可以了。

小林补充：

struct{}不占据任何内存空间

一般有三个用途：

1. 实现set集合
2. 和channel配合使用，不具备任何意义，但除用作goroutine之间通知
3. 实现一个不带字段，仅包含方法的结构体

go 里面有哪些锁，怎么使用，用过么

读者答：操作系统信号量的一些锁。他也有 channel 这个东西这两个东西都可以实现并发的这个操作。然后如果用锁的话就是说他有互斥锁、读写的锁，或者是信号量就刚才提到信号量。然后具体使用的话，需要我举个例子吗？

小林补充：

互斥锁和读写锁

MySQL 常见的存储引擎是哪个？他的索引有哪些

读者答：InnoDB，InnoDB 比较常见的索引一个是 B 树，还有一个是倒排表，然后它也支持哈希索引和那个一个东西叫空间索引好像是。

小林补充：

MySQL 常见的存储引擎 InnoDB、MyISAM 和 Memory。InnoDB 是在 MySQL 5.5 之后成为默认的 MySQL 存储引擎，B+Tree 索引类型也是 MySQL 存储引擎采用最多的索引类型。

索引类型	InnoDB 引擎	MyISAM 引擎	Memory 引擎
B+Tree 索引	Yes	Yes	Yes
HASH 索引	No (不支持hash索引, 但是在内存结构中有一个自适应hash索引)	No	Yes
Full-Text 索引	Yes (MySQL 5.6 版本后支持)	Yes	No

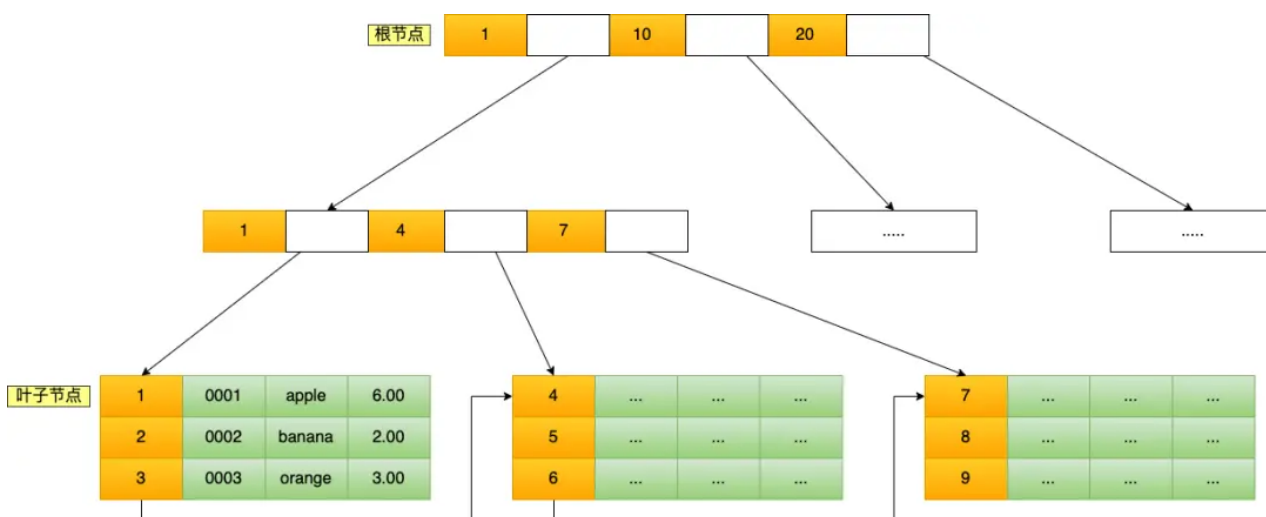
InnoDB 它如果是存储一张表的话它是怎么去存储的

读者答：如果使用 B+树的话这个他的每一条信息他都是存到这个 B+ 树的叶子节点上，然后他的索引和他的那个具体数据是在一起的。

小林补充：

表的数据是存储在聚簇索引，聚簇索引的 B+Tree 的叶子节点存放的是实际数据，所有完整的用户记录都存放在主键索引的 B+Tree 的叶子节点里。

主键索引的 B+Tree 如图所示（图中叶子节点之间我画了单向链表，但是实际上是双向链表，原图我找不到了，修改不了，偷个懒我不重画了，大家脑补成双向链表就行）：



如说你创建一个索引的话，比如一个字一个表里面有 a b c 三个字段，对 b 和 c 这两个字段去做一个索引，顺序的话是先 b 或后 c，那这时候我需要去查 c 这个字段能用上这个索引么？

不知怎么答糊涂了，把问题都搞错了，又重新答了一遍

读者答：如果对 c 要对 c 查询的话他无法使用到这个索引，因为我们刚才建的那个索引它是一个复合类型的，就是你既有 b 有 c，然后你只有在要你的查询条件包含 b c 的时候你才能按最左匹配原则去进行查询，而如果你只查 c 的话是无法使用这个索引的需要为 c 单独建立索引才可以。

小林补充：

联合索引(b,c)，如果查询条件是有c字段，是无法利用走该联合索引的，因为联合索引的b+树是先按b排序，再按c排序，所以如果查询条件只有一个c 字段， 它就无法在这颗联合索引的b+树进行二分查找，这就是联合索引有最左匹配规则的来源。

算法题：反转链表

用三个指针很快写完，面试官问我可不可以用两个指针，我不知道，我又用头插法写了一遍。

之后让我对比了这两个方法。

感觉面试官像是已经决定要挂我了随便出的算法题走个流程。

说一下数据库隔离级别。

读者答：读未提交，读提交，可重复读、可串行化。

这个读可重复读和那个读提交这两个有什么区别

我举了两个例子，读提交的流程说成了读未提交的，经过面试官提醒才发现并改过来

读者答：对于可重复读来说，就是在 a 进入这个事务以后，那个他的这个数据在他的视图来说就是已经是固定了，如果说在 a 这个事务提交之前 B 的这个事务修改了那个数据，在 A 是看不到的

于读提交的情况来说，就是说还是 a b 两个事务，b 事务修改一个数据然后并且提交以后，但 a a 还没有提交，然后 a 这个时候去读那个数据，就会读到 b 已经修改的数据。

小林补充：

读提交，指一个事务提交之后，它做的变更才能被其他事务看到。

可重复读，指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，MySQL InnoDB 引擎的默认隔离级别。

对于「读提交」和「可重复读」隔离级别的事务来说，它们是通过 MVCC 来实现的，它们的区别在于创建 Read View 的时机不同，大家可以把 Read View 理解成一个数据快照，就像相机拍照那样，定格某一时刻的风景。

「读提交」隔离级别是在「每个语句执行前」都会重新生成一个 Read View，而「可重复读」隔离级别是「启动事务时」生成一个 Read View，然后整个事务期间都在用这个 Read View。

反问

有哪些需要深入学习的地方？

答：如果想往后端开发方向发展的话，就数据库这一块可能你需要再再多看一看，他的一些内部原理相关的书或者课程。

在公司这块的话就是实习的过程中做 go 的相关的东西多吗，还是说需要转语言什么的。

答：基本上内部现在大部分都是往 go 上面去转的，所以这个这个这个没有什么问题。一般来说不会需要去转语言的。

总结

读者总结

感觉：

1. 有很多地方说了又改，给人印象很不好。
2. 谈吐不自然，说话磕磕绊绊。

不足之处：

1. 对基础知识掌握不牢固，遇到拿不准的地方喜欢瞎说。
2. 对自己的项目描述不是很完善，需要详细补充内容。

小林总结

- 面试的回答的时候，最好先抛出结论，再举例子，而不是先举例子，再说结论。因为面试其实说的是关键词，只要你说的几个技术词，命中了面试官心里的预期，他就会认为你知道这个知识的了。
- 如果你面后端，mysql确实得补一补，像索引和事务是最常见的两个问题，可以到我的图解mysql网站补一补这两个知识。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

美团面试，问的都是基础啊！

大家好，我是小林。

今天分享一位读者的春招面经，美团基础架构的面经。

问的全是基础，一个编程语言的问都没有。

问题记录

MySQL-MVCC

读者答：InnoDB是通过 MVCC 实现可重复读的隔离级别的，MVCC 就是多版本并发控制，它其实记录了历史版本的数据，解决了读写并发冲突问题。有一个版本编码，然后它进入了各种操作下的数据状态，能够根据当前这个指令的状态来读取不同时期的数据快照。主要实现方法的话就是通过事务版本号，读取视图还有undo日志进行完善的。

小林补充：具体的实现原理过程，可以去 xiaolincoding.com 网站->图解MySQL->事务隔离级别是怎么实现的？这篇文章学习。

MySQL-原子性怎么实现的

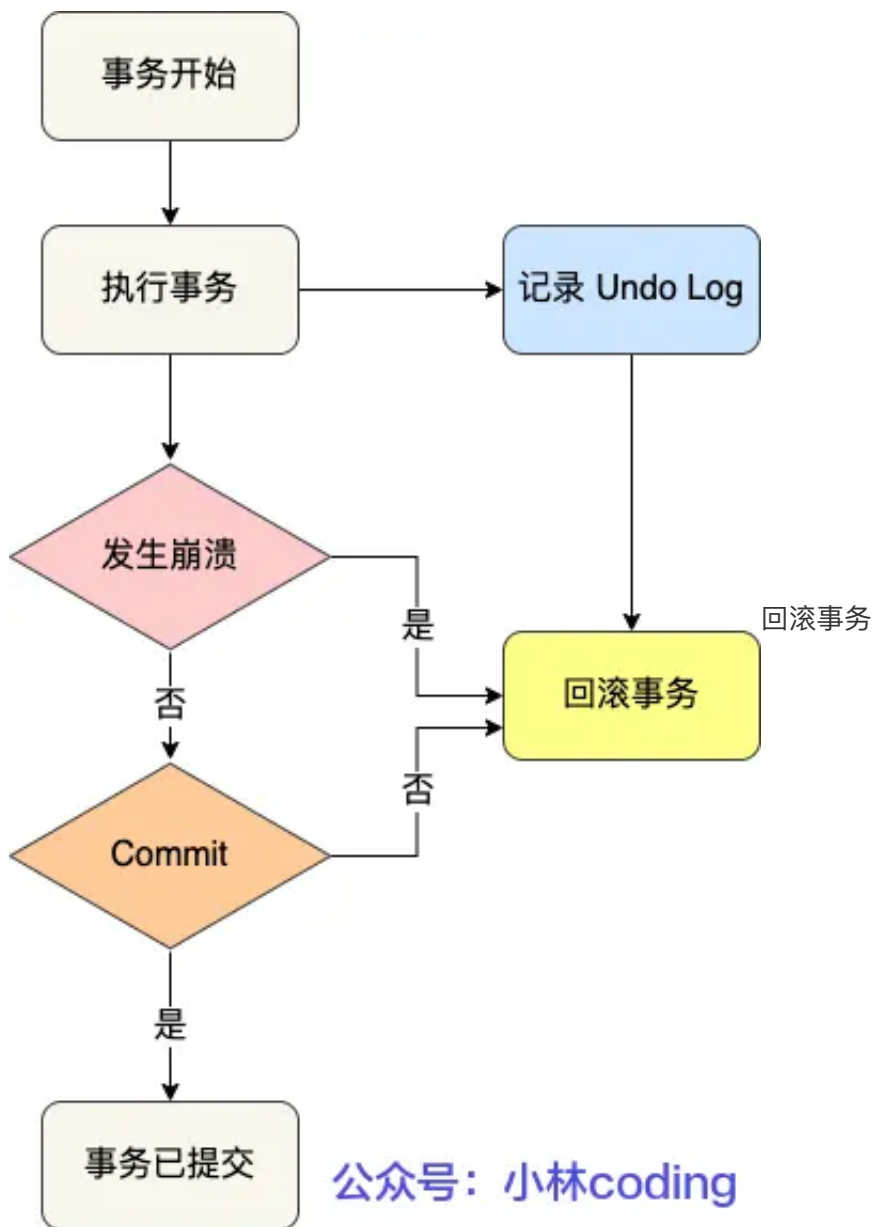
说错了，说成redoLog了。应该是undoLog。

读者答：原子性的话会在写数据之前有一个，就是WAL的过程，就是写一个 redo log，然后如果数据没有写完或者是执行操作失败的话，可以恢复所有已提交的事务或者回滚。

小林补充：

事务的原子性是通过 undo log 实现的。

undo log 是一种用于撤销回退的日志。在事务没提交之前，MySQL 会先记录更新前的数据到 undo log 日志文件里面，当事务回滚时，可以利用 undo log 来进行回滚。如下图：



每当 InnoDB 引擎对一条记录进行操作（修改、删除、新增）时，要把回滚时需要的信息都记录到 undo log 里，比如：

- 在插入一条记录时，要把这条记录的主键值记下来，这样之后回滚时只需要把这个主键值对应的记录删掉就好了；
- 在删除一条记录时，要把这条记录中的内容都记下来，这样之后回滚时再把由这些内容组成的记录插入到表中就好了；
- 在更新一条记录时，要把被更新的列的旧值记下来，这样之后回滚时再把这些列更新为旧值就好了。

在发生回滚时，就读取 undo log 里的数据，然后做原先相反操作。比如当 delete 一条记录时，undo log 中会把记录中的内容都记下来，然后执行回滚操作的时候，就读取 undo log 里的数据，然后进行 insert 操作。

不同的操作，需要记录的内容也是不同的，所以不同类型的操作（修改、删除、新增）产生的 undo log 的格式也是不同的，具体的每一个操作的 undo log 的格式我就不详细介绍了，感兴趣的可以自己去看看。

MySQL-持久性是怎么实现的

读者答：通过 redo log 保证持久化。buffer pool 中有 undo 页，对 undo 页的修改也都会记录到 redo log。redo log 会每秒刷盘，提交事务时也会刷盘，数据页和 undo 页都是靠这个机制保证持久化的。

通过两次写来实现，当缓冲池的脏页刷新时，并不直接写磁盘，而是会通过memcpy函数将脏页先拷贝到内存中的doublewrite buffer，之后通过doublewrite buffer再分两次，每次写入1MB到共享表空间的物理磁盘上，然后马上调用fsync函数，同步磁盘，进行数据持久化。

小林补充：

事务的持久性是通过 redo log 实现的。

我们修改某条记录，其实该记录并不是马上刷入磁盘的，而是将 Innodb 的 Buffer Pool 标记为脏页，等待后续的异步刷盘。

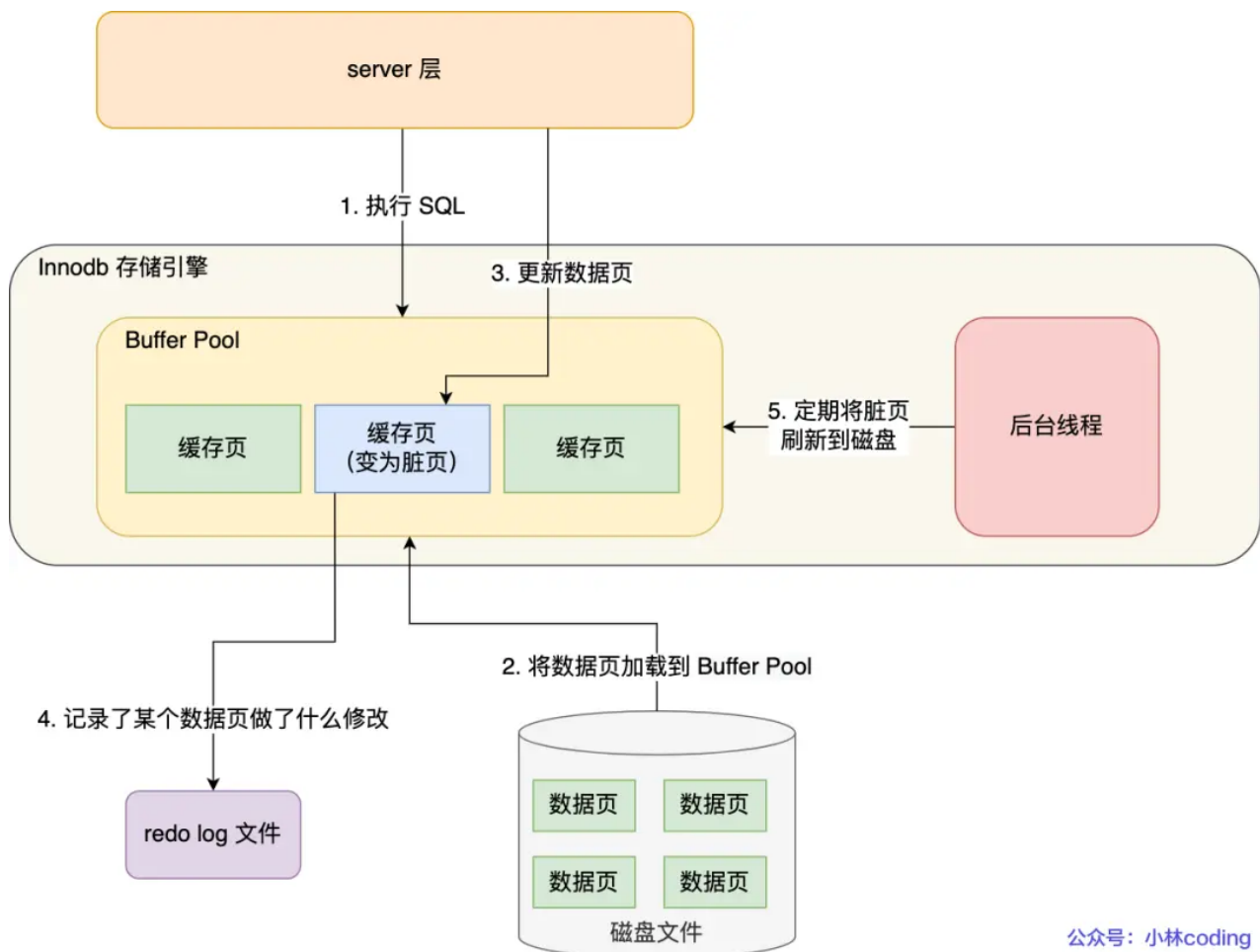
Buffer Pool 是提高了读写效率没错，但是问题来了，Buffer Pool 是基于内存的，而内存总是不可靠，万一断电重启，还没来得及落盘的脏页数据就会丢失。

为了防止断电导致数据丢失的问题，当有一条记录需要更新的时候，InnoDB 引擎就会先更新内存（同时标记为脏页），然后将本次对这个页的修改以 redo log 的形式记录下来，**这个时候更新就算完成了。**

后续，InnoDB 引擎会在适当的时候，由后台线程将缓存在 Buffer Pool 的脏页刷新到磁盘里，这就是 **WAL（Write-Ahead Logging）** 技术。

WAL 技术指的是，MySQL 的写操作并不是立刻写到磁盘上，而是先写日志，然后在合适的时间再写到磁盘上。

过程如下图：



公众号：小林coding

redo log 是物理日志，记录了某个数据页做了什么修改，比如对 **XXX** 表空间中的 **YYY** 数据页 **ZZZ** 偏移量的地方做了 **AAA** 更新，每当执行一个事务就会产生这样的一条或者多条物理日志。

在事务提交时，只要先将 redo log 持久化到磁盘即可，可以不需要等到将缓存在 Buffer Pool 里的脏页数据持久化到磁盘。

当系统崩溃时，虽然脏页数据没有持久化，但是 redo log 已经持久化，接着 MySQL 重启后，可以根据 redo log 的内容，将所有数据恢复到最新的状态。

操作系统-死锁怎么产生的

读者答：死锁会产生的话一般会出现就是嗯资源就是互相占用，但是没有办法解锁，形成循环这样的情况，比如说 a 线程有一部分 b 线程需要的资源，b 线程有一部分 a 需要的资源，那他两个人互相的互斥等待形成了死锁，两个线程都没有办法完成任务。

小林补充：

死锁问题的产生是由两个或者以上线程并行执行的时候，争夺资源而互相等待造成的。

死锁只有同时满足互斥、持有并等待、不可剥夺、环路等待这四个条件的时候才会发生。

所以要避免死锁问题，就是要破坏其中一个条件即可，最常用的方法就是使用资源有序分配法来破坏环路等待条件。

操作系统-怎么避免死锁

读者答：

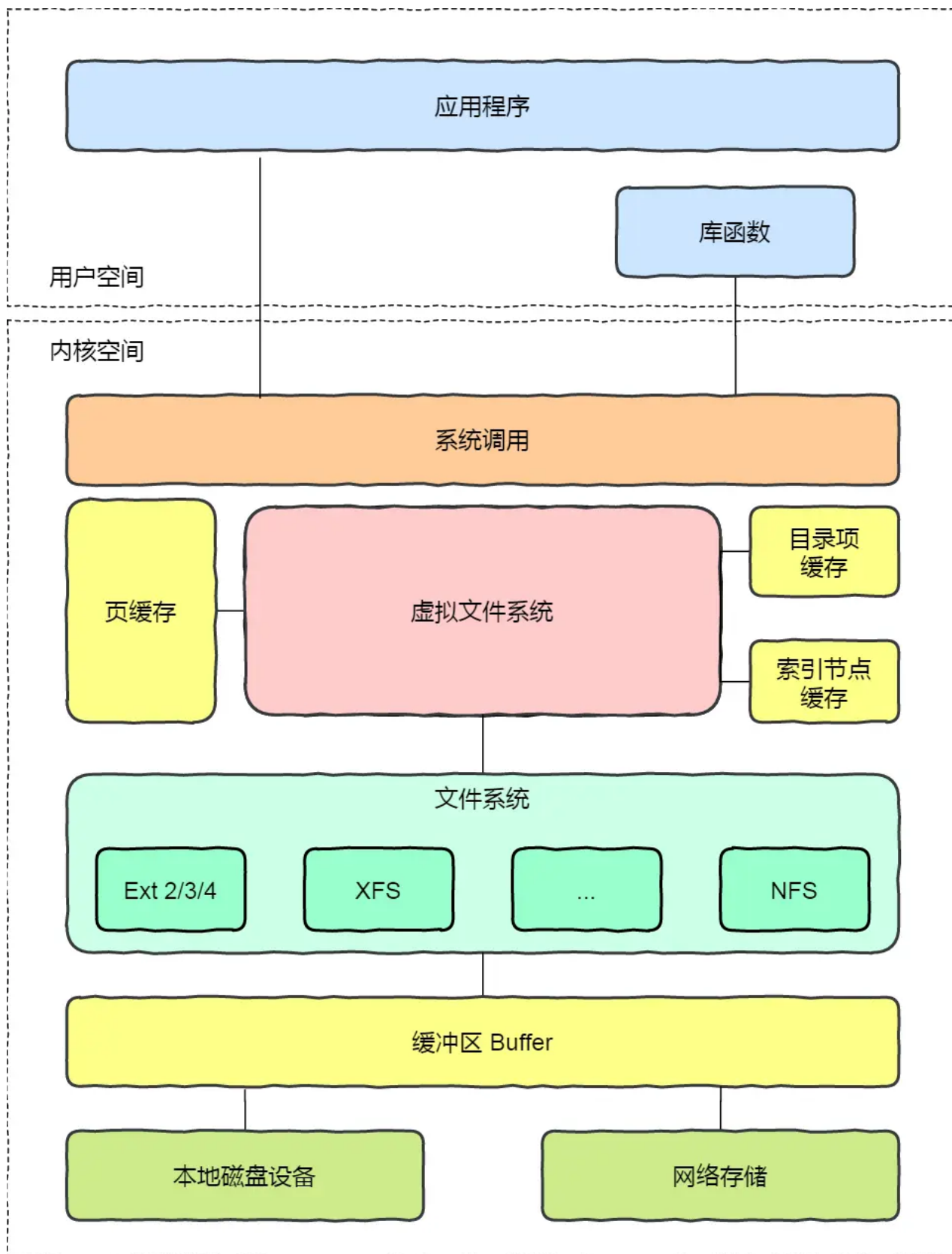
- 避免死锁的话可以手动的 kill 掉某一个进程来结束当前的死锁状态。
- 也可以说设置一些抢占的规则。如果这个进程占用的时间非常长的话，通过上下文切换给另外一个进程运行的机会，
- 也可以在分配资源的时候进行预先的设计，就是有一个银行家算法来进行一个不会发生死锁的进程运行的调度

操作系统-pageCache是什么

读者答：缓存一些比较常访问的文件到缓存中，这样子的话它就能减少两次从内核空间拷贝的过程，就是来减少查询这个内容的时间。

小林补充：

为了提升对文件的读写效率，Linux 内核会以页大小（4KB）为单位，将文件划分为多数据块。当用户对文件中的某个数据块进行读写操作时，内核首先会申请一个内存页（称为 页缓存）与文件中的数据块进行绑定。如下图所示：



如上图所示，当用户对文件进行读写时，实际上是对文件的 **页缓存** 进行读写。所以对文件进行读写操作时，会分以下两种情况进行处理：

- 当从文件中读取数据时，如果要读取的数据所在的页缓存已经存在，那么就直接把页缓存的数据拷贝给用户即可。否则，内核首先会申请一个空闲的内存页（页缓存），然后从文件中读取数据到页缓存，并且把页缓存的数据拷贝给用户。
- 当向文件中写入数据时，如果要写入的数据所在的页缓存已经存在，那么直接把新数据写入到页缓存。

存即可。否则，内核首先会申请一个空闲的内存页（页缓存），然后从文件中读取数据到页缓存，并且把新数据写入到页缓存中。对于被修改的页缓存，内核会定时把这些页缓存刷新到文件中。

计算机网络-TCP的可靠性和顺序性怎么实现的

读者答：TCP 它实现可靠性和有序性的操作的话，是通过快重传或者是回退 n 这样子的设计来实现。如果报文在传递的过程中丢失之后能够进行重传。而会怎么能发现这个报文丢失呢？主要是根据一些序列号和 ACK 的配合来帮助两个服务之间知道当前传递的信息会丢失。

计算机网络-怎么进行流量控制的？

回答成拥塞控制了；

读者答：内部维护了一个能接收消息的一个窗口的大小，如果他出现就是消息丢失的情况，然后这个消息窗口的大小会减半。启动的时候采用慢启动的方式，从0开始指数级增加窗口大小，直到到达阈值之后线性增加窗口大小。

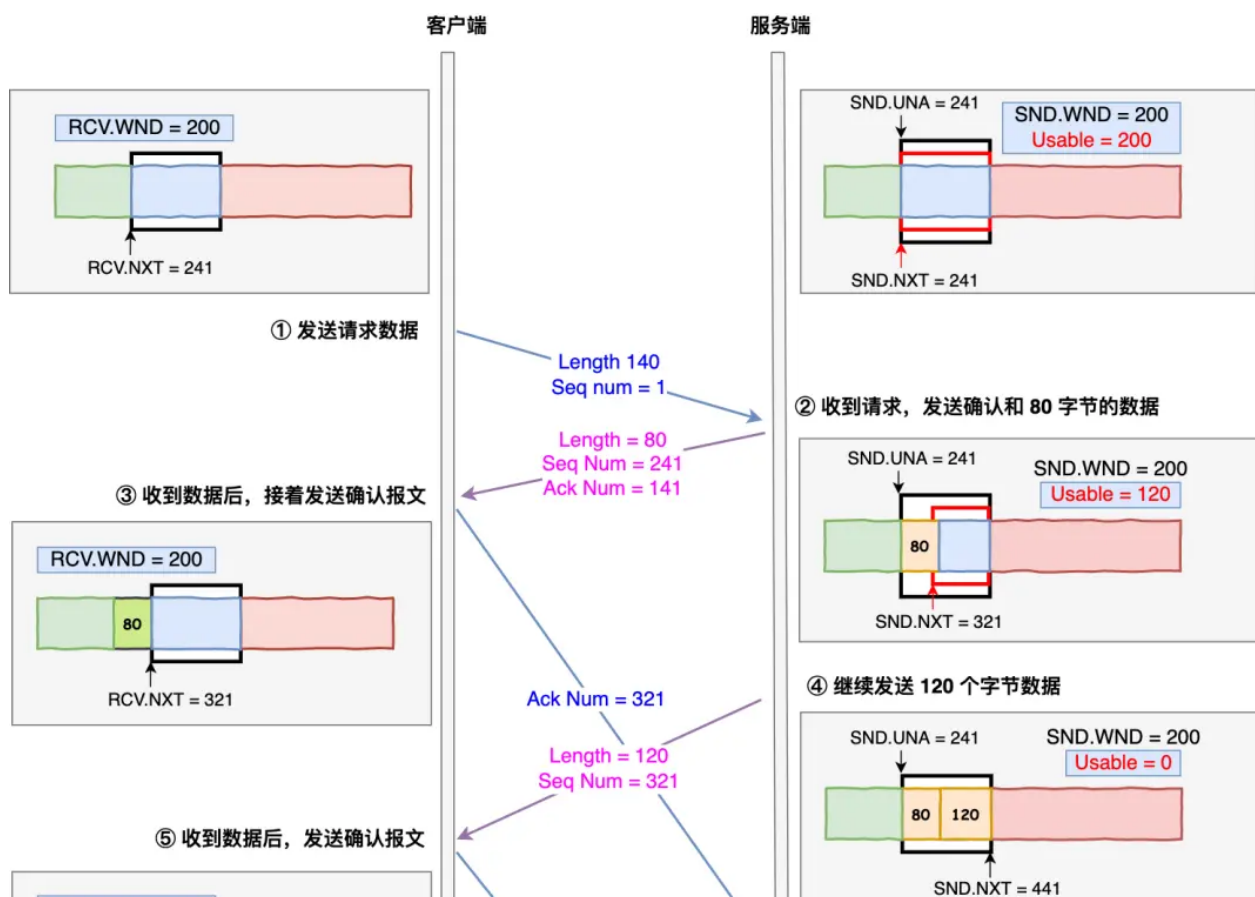
小林补充：

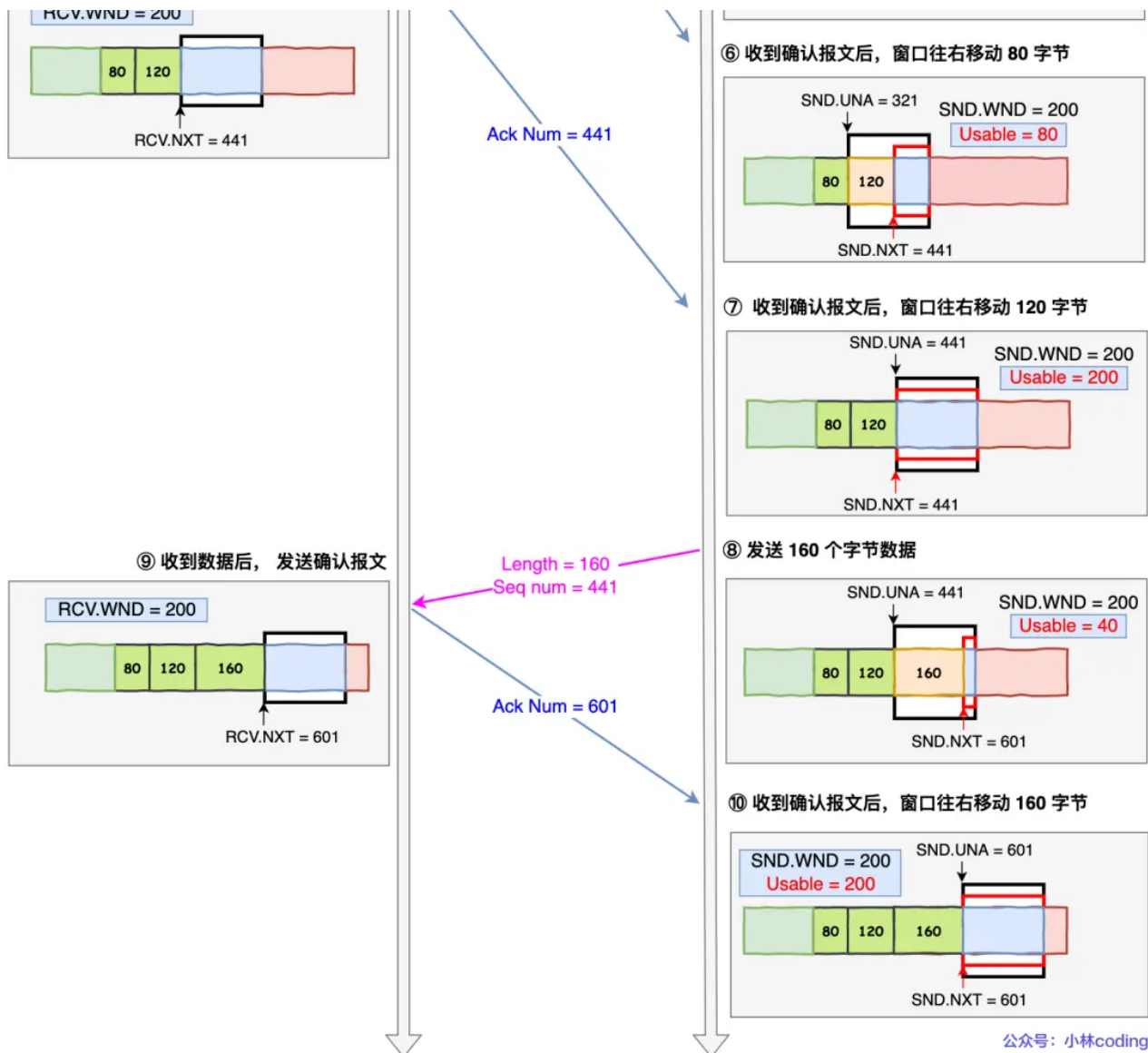
流量控制主要是可以让「发送方」根据「接收方」的实际接收能力控制发送的数据量。

实现的方式，接收方会有一个接收缓冲区，如果内核接收到了数据，没有被应用读取的话，接收窗口就会收缩，然后会在tcp报文携带接收窗口的大小，发送发收到后，就会控制的发送流量。

下面举个栗子，为了简单起见，假设以下场景：

- 客户端是接收方，服务端是发送方
- 假设接收窗口和发送窗口相同，都为 200
- 假设两个设备在整个传输过程中都保持相同的窗口大小，不受外界影响





流量控制

根据上图的流量控制，说明下每个过程：

1. 客户端向服务端发送请求数据报文。这里要说明下，本次例子是把服务端作为发送方，所以没有画出服务端的接收窗口。
2. 服务端收到请求报文后，发送确认报文和 80 字节的数据，于是可用窗口 Usable 减少为 120 字节，同时 SND.NXT 指针也向右偏移 80 字节后，指向 321，这意味着下次发送数据的时候，序列号是 321。
3. 客户端收到 80 字节数据后，于是接收窗口往右移动 80 字节，RCV.NXT 也就指向 321，这意味着客户端期望的下一个报文的序列号是 321，接着发送确认报文给服务端。
4. 服务端再次发送了 120 字节数据，于是可用窗口耗尽为 0，服务端无法再继续发送数据。
5. 客户端收到 120 字节的数据后，于是接收窗口往右移动 120 字节，RCV.NXT 也就指向 441，接着发送确认报文给服务端。
6. 服务端收到对 80 字节数据的确认报文后，SND.UNA 指针往右偏移后指向 321，于是可用窗口 Usable 增大到 80。
7. 服务端收到对 120 字节数据的确认报文后，SND.UNA 指针往右偏移后指向 441，于是可用窗口 Usable 增大到 200。
8. 服务端可以继续发送了，于是发送了 160 字节的数据后，SND.NXT 指向 601，于是可用窗口 Usable 减少到 40。

9. 客户端收到 160 字节后，接收窗口往右移动了 160 字节，`RCV.NXT` 也就是指向了 601，接着发送确认报文给服务端。
10. 服务端收到对 160 字节数据的确认报文后，发送窗口往右移动了 160 字节，于是 `SND.UNA` 指针偏移了 160 后指向 601，可用窗口 `Usable` 也就增大至了 200。

Redis-怎么持久化的数据

读者答：Redis 的话，它其实提供了两种持久化数据的方法，一种是 AOF，一种是 RDB。然后 AOF 的话它是一种，就是说每一条操作信息它都会进行追加记录这样的一种持久化的方式。当那个数据库重新启动的时候，它会根据 AOF 里面记录的数据操作，然后来进行一个数据库内容的重建。而 RDB 的话，它是做快照，也就是说在数据库运行的过程中，它可能会另开一个 IO 的线程来进行数据库的快照记录，这样子的话来记录它某一个时间段的数据情况，这样子它进行恢复，数据库再次启动的时候就可以直接根据 RDB 文件来进行恢复这两个操作。

这样一执行的话就可以看出来，AOF 的话，它虽然就是在执行的过程中性能的损耗是小的，但是如果数据库要进行重新启动的话，那它需要的耗时是比较长的。而 RDB 的话，它虽然重新启动的耗时小，但是说它在过程中会有一定的性能损耗。而且如果是在两个快照创建的中间就是数据库宕机，或者是这样子没有做成快照的话，会造成一部分数据的缺失。

Redis-集群是怎么做的？就是数据怎么分片的，然后它的集群的高可用是什么？怎么部署的，这个有没有了解过？

读者答：我了解到它是有一个主从模型的，从它从模型的话就是复制一份主节点的备份，然后如果主节点宕机的情况下，从节点是可以成为主节点来提供服务的，别的就没有什么了解的。

后续查资料补充add：在应对数据量扩容时，虽然增加内存这种纵向扩展的方法简单直接，但是会造成数据库的内存过大，导致性能变慢。Redis 切片集群提供了横向扩展的模式，也就是使用多个实例，并给每个实例配置一定数量的哈希槽，数据可以通过键的哈希值映射到哈希槽，再通过哈希槽分散保存到不同的实例上。这样做的好处是扩展性好，不管有多少数据，切片集群都能应对。

分布式-分布式事务是什么

读者答：我只知道分布式事务中的 2 阶段提交和 3 阶段提交这样一个概念

- 2 阶段提交：准备阶段和提交阶段
- 3 阶段提交：比 2PC 多了一个阶段，即把原来 2PC 的准备阶段拆分成 CanCommit 和 PreCommit 两个阶段，同时引入超时机制来解决 2PC 的同步阻塞问题。

后续查资料补充add：实际使用都是使用消息队列+本地消息表保证最终一致性，2PC 这种强一致性用在一些金融业务中，实现很麻烦。

分布式-paxos和raft的区别

读者答：Paxos 在一个节点当选为就是 leader 节点之后，其他的从节点如果不满意主节点的那个投票策略的话，是可以对主节点的投票就是进行否决的。Paxos 就是三阶段提交。但是 raft 的话就是只要集群中存在 leader 节点的话，从节点就是会按照主节点的策略来进行一致性的执行。

分布式-为什么就是分布式的共识算法都需要要求多数派提交才能完成它的分布式一致性？

读者答：有作恶节点，消息可能到的顺序不一样，扯了拜占庭问题

场景题

- 设计一个线程池
- LRU 缓存设计

面试总结

感觉

面试官想多要些人填志愿，基础知识没有深挖，所有的知识点都考察了一下

不足之处

1. 基础知识还要再扎实
2. 场景题需要结合实际开发，有些过于书本知识了，实际不用



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

① 关注公众号回复「**图解**」
获取图解系列 PDF

② 关注公众号回复「**加群**」
拉你进百人技术交流群

字节面试体验拉满！

大家好，我是小林。

今天分享一位读者春招的字节二面面经，岗位是后端开发。

一个编程语言都没问，都是问网络+项目+mysql+redis。

问题记录

使用消息中间件降低消息持久化的压力是怎么做的，为什么可以降低？

读者答：在突发大量消息的情况下可以做到流量削峰，在消费者消费能力达不到生产者产生消息的速度时也能够正常运行。

怎么解决消息队列上的消息堆压？

读者答：

- （1）自身场景下，消息堆压是暂时的，消息堆压只是突发状况，就算不额外处理，随着时间流逝也可消费完毕。
- （2）假如存在持续性消息堆压，可以考虑临时增加消费者的数量，提升消费者的消费能力。

小林补充：

如果是线上突发问题，要临时扩容，增加消费端的数量，与此同时，降级一些非核心的业务。通过扩容和降级承担流量，这是为了表明你对应急问题的处理能力。其次，才是排查解决异常问题，如通过监控，日志等手段分析是否消费端的业务逻辑代码出现了问题，优化消费端的业务处理逻辑。

在并发编程时，在需要加锁时，不加锁会有什么问题？

读者答：两个线程使用同一个全局变量会有不一致的问题，比如a线程把全局变量加1，b线程读的时候，如果还是从缓存中读的，那么会没有发现这个更新，就会产生不一致的问题。

如何避免出现死锁？

读者答：在使用之前，考虑死锁产生的条件：互斥访问、占有并保持、循环等待。

针对以上几点，可以：资源一次性分配、占有时可被打断、考虑资源分配顺序。

HTTP状态码有哪些？

读者答：

- 1xx信息的响应，比如100表示临时响应。
- 2xx成功。
- 3xx重定向，301永久更改，308临时更改（是308么？）哦不，是307，记得不是很清楚了。
- 4xx客户端问题，404 not found找不到资源，403 客户端没有访问权限。
- 5xx服务端问题，502网关问题。

面试官：307其实是临时没错，他比较新的版本，旧的是 302，现在业界还是旧的用的比较多。

小林补充：

五大类 HTTP 状态码

	具体含义	常见的状态码
1xx	提示信息，表示目前是协议处理的中间状态，还需要后续的操作；	
2xx	成功，报文已经收到并被正确处理；	200、204、206
3xx	重定向，资源位置发生变动，需要客户端重新发送请求；	301、302、304
4xx	客户端错误，请求报文有误，服务器无法处理；	400、403、404
5xx	服务器错误，服务器在处理请求时内部发生了错误。	500、501、502、503

五大类 HTTP 状态码

1xx 类状态码属于**提示信息**，是协议处理中的一种中间状态，实际用到的比较少。

2xx 类状态码表示服务器**成功**处理了客户端的请求，也是我们最愿意看到的状态。

- 「**200 OK**」是最常见的成功状态码，表示一切正常。如果是非 **HEAD** 请求，服务器返回的响应头都会有 body 数据。
- 「**204 No Content**」也是常见的成功状态码，与 200 OK 基本相同，但响应头没有 body 数据。
- 「**206 Partial Content**」是应用于 HTTP 分块下载或断点续传，表示响应返回的 body 数据并不是资源的全部，而是其中的一部分，也是服务器处理成功的状态。

3xx 类状态码表示客户端请求的资源发生了变动，需要客户端用新的 URL 重新发送请求获取资源，也就是**重定向**。

- 「**301 Moved Permanently**」表示永久重定向，说明请求的资源已经不存在了，需改用新的 URL 再次访问。
- 「**302 Found**」表示临时重定向，说明请求的资源还在，但暂时需要用另一个 URL 来访问。

301 和 302 都会在响应头里使用字段 **Location**，指明后续要跳转的 URL，浏览器会自动重定向新的 URL。

- 「**304 Not Modified**」不具有跳转的含义，表示资源未修改，重定向已存在的缓冲文件，也称缓存重定向，也就是告诉客户端可以继续使用缓存资源，用于缓存控制。

4xx 类状态码表示客户端发送的**报文有误**，服务器无法处理，也就是错误码的含义。

- 「**400 Bad Request**」表示客户端请求的报文有错误，但只是个笼统的错误。
- 「**403 Forbidden**」表示服务器禁止访问资源，并不是客户端的请求出错。
- 「**404 Not Found**」表示请求的资源在服务器上不存在或未找到，所以无法提供给客户端。

5xx 类状态码表示客户端请求报文正确，但是**服务器处理时内部发生了错误**，属于服务器端的错误码。

- 「**500 Internal Server Error**」与 400 类型，是个笼统通用的错误码，服务器发生了什么错误，我们并不知道。
- 「**501 Not Implemented**」表示客户端请求的功能还不支持，类似“即将开业，敬请期待”的意

思。

- 「**502 Bad Gateway**」通常是服务器作为网关或代理时返回的错误码，表示服务器自身工作正常，访问后端服务器发生了错误。
- 「**503 Service Unavailable**」表示服务器当前很忙，暂时无法响应客户端，类似“网络服务正忙，请稍后重试”的意思。

HTTP1.0和2.0的区别，或者2.0和3.0的区别？

读者答：

- （1）每次请求他都会与服务器建立一个tcp的连接。请求处理完以后他会立刻断开tcp，他有短暂的一个tcp链接。在1.1中，他稍微有些改善。
- （2）2.0提供了一个多路复用的功能。还有二进制分帧和首部压缩。

小林补充：

HTTP/2 相比 HTTP/1.1 性能上的改进：

- 头部压缩
- 二进制格式
- 并发传输
- 服务器主动推送资源

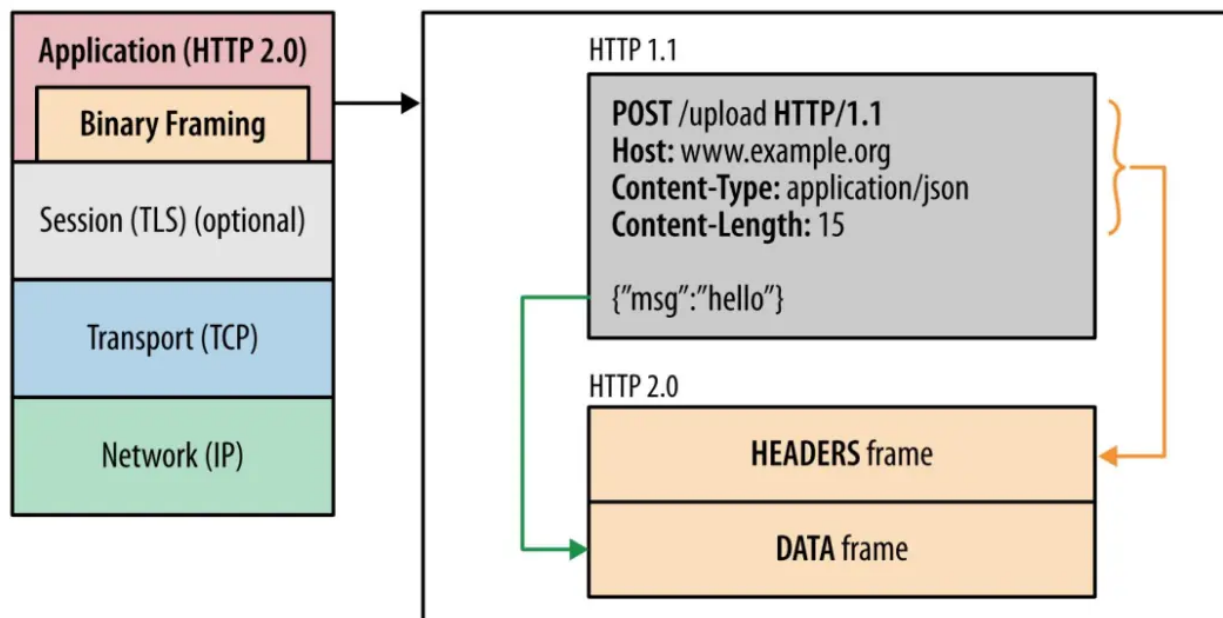
1. 头部压缩

HTTP/2 会**压缩头**（Header）如果你同时发出多个请求，他们的头是一样的或是相似的，那么，协议会帮你**消除重复的部分**。

这就是所谓的 **HPACK** 算法：在客户端和服务器同时维护一张头信息表，所有字段都会存入这个表，生成一个索引号，以后就不发送同样字段了，只发送索引号，这样就**提高速度**了。

2. 二进制格式

HTTP/2 不再像 HTTP/1.1 里的纯文本形式的报文，而是全面采用了**二进制格式**，头信息和数据体都是二进制，并且统称为帧（frame）：**头信息帧（Headers Frame）**和**数据帧（Data Frame）**。



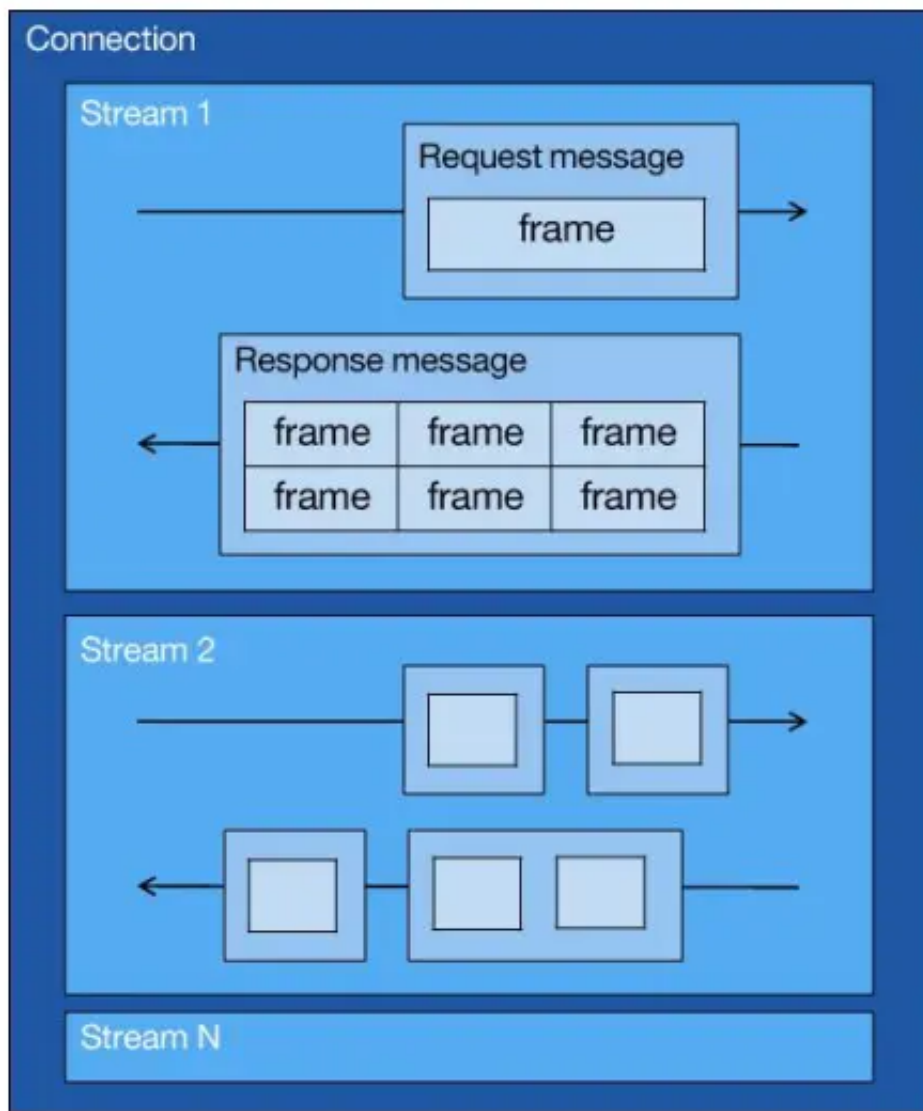
HTTP/1 与 HTTP/2

这样虽然对人不友好，但是对计算机非常友好，因为计算机只懂二进制，那么收到报文后，无需再将明文报文转成二进制，而是直接解析二进制报文，这增加了数据传输的效率。

3. 并发传输

我们都知道 HTTP/1.1 的实现是基于请求-响应模型的。同一个连接中，HTTP 完成一个事务（请求与响应），才能处理下一个事务，也就是说在发出请求等待响应的过程中，是没办法做其他事情的，如果响应迟迟不来，那么后续的请求是无法发送的，也造成了队头阻塞的问题。

而 HTTP/2 就很牛逼了，引出了 Stream 概念，多个 Stream 复用在一条 TCP 连接。

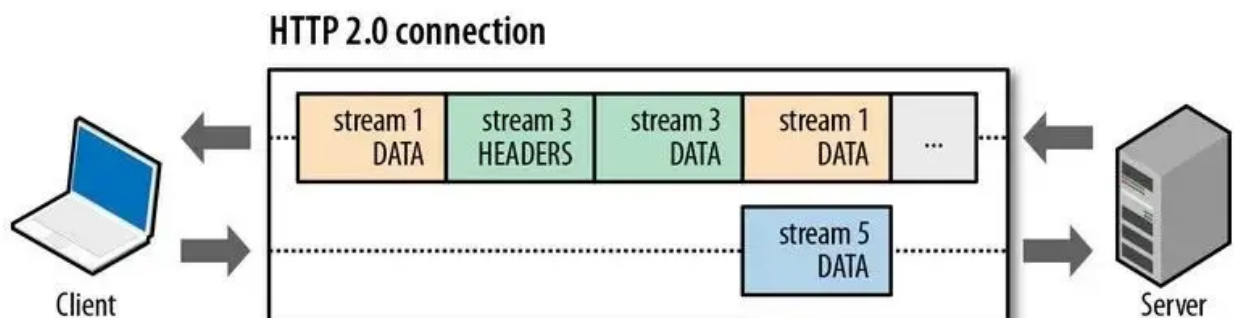


img

从上图可以看到，1 个 TCP 连接包含多个 Stream，Stream 里可以包含 1 个或多个 Message，Message 对应 HTTP/1 中的请求或响应，由 HTTP 头部和包体构成。Message 里包含一条或者多个 Frame，Frame 是 HTTP/2 最小单位，以二进制压缩格式存放 HTTP/1 中的内容（头部和包体）。

针对不同的 HTTP 请求用独一无二的 **Stream ID** 来区分，接收端可以通过 **Stream ID** 有序组装成 HTTP 消息，不同 **Stream** 的帧是可以乱序发送的，因此可以并发不同的 **Stream**，也就是 HTTP/2 可以并行交错地发送请求和响应。

比如下图，服务端**并行交错地**发送了两个响应：Stream 1 和 Stream 3，这两个 Stream 都是跑在一个 TCP 连接上，客户端收到后，会根据相同的 Stream ID 有序组装成 HTTP 消息。



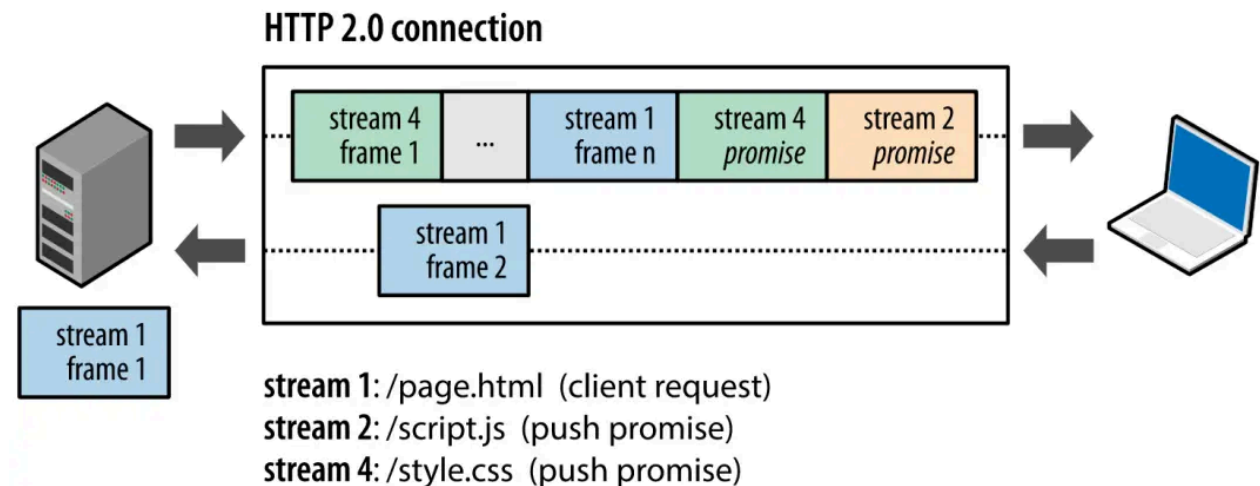
img

4、服务器推送

HTTP/2 还在一定程度上改善了传统的「请求 - 应答」工作模式，服务端不再是被动地响应，可以主动向客户端发送消息。

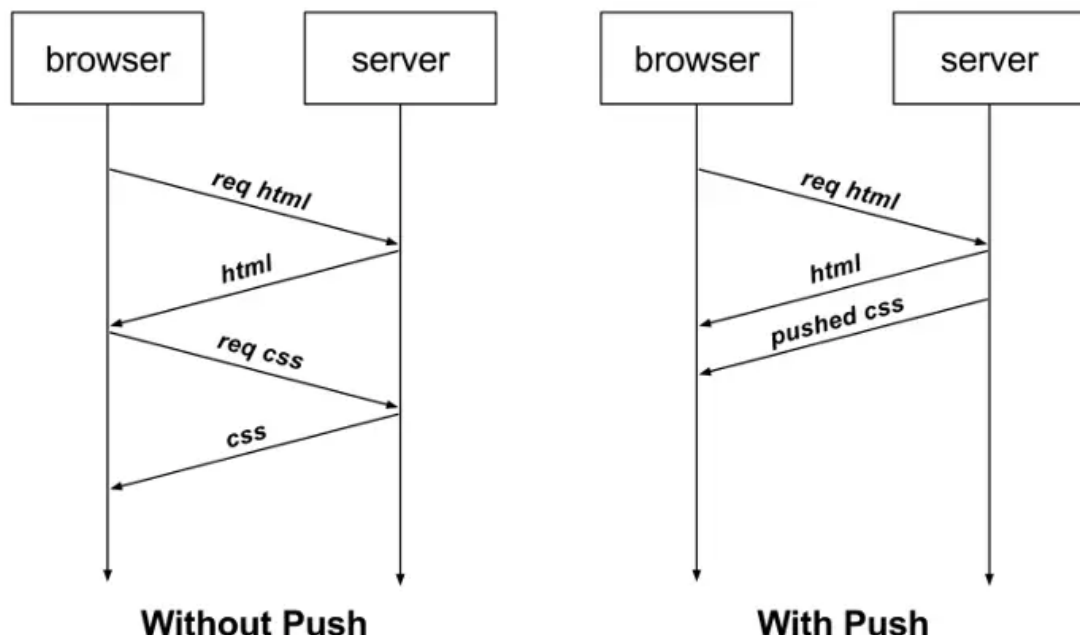
客户端和服务端双方都可以建立 **Stream**，Stream ID 也是有区别的，客户端建立的 Stream 必须是奇数号，而服务器建立的 Stream 必须是偶数号。

比如下图，Stream 1 是客户端向服务端请求的资源，属于客户端建立的 Stream，所以该 Stream 的 ID 是奇数（数字 1）；Stream 2 和 4 都是服务端主动向客户端推送的资源，属于服务端建立的 Stream，所以这两个 Stream 的 ID 是偶数（数字 2 和 4）。



img

再比如，客户端通过 HTTP/1.1 请求从服务器那获取到了 HTML 文件，而 HTML 可能还需要依赖 CSS 来渲染页面，这时客户端还要再发起获取 CSS 文件的请求，需要两次消息往返，如下图左边部分：



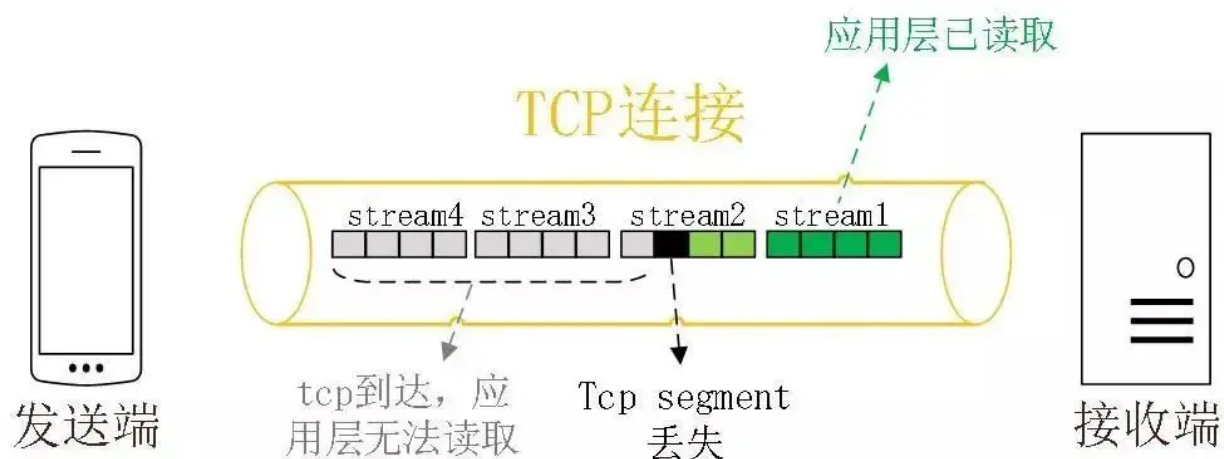
img

如上图右边部分，在 HTTP/2 中，客户端在访问 HTML 时，服务器可以直接主动推送 CSS 文件，减少了消息传递的次数。

HTTP/2 有什么缺陷？

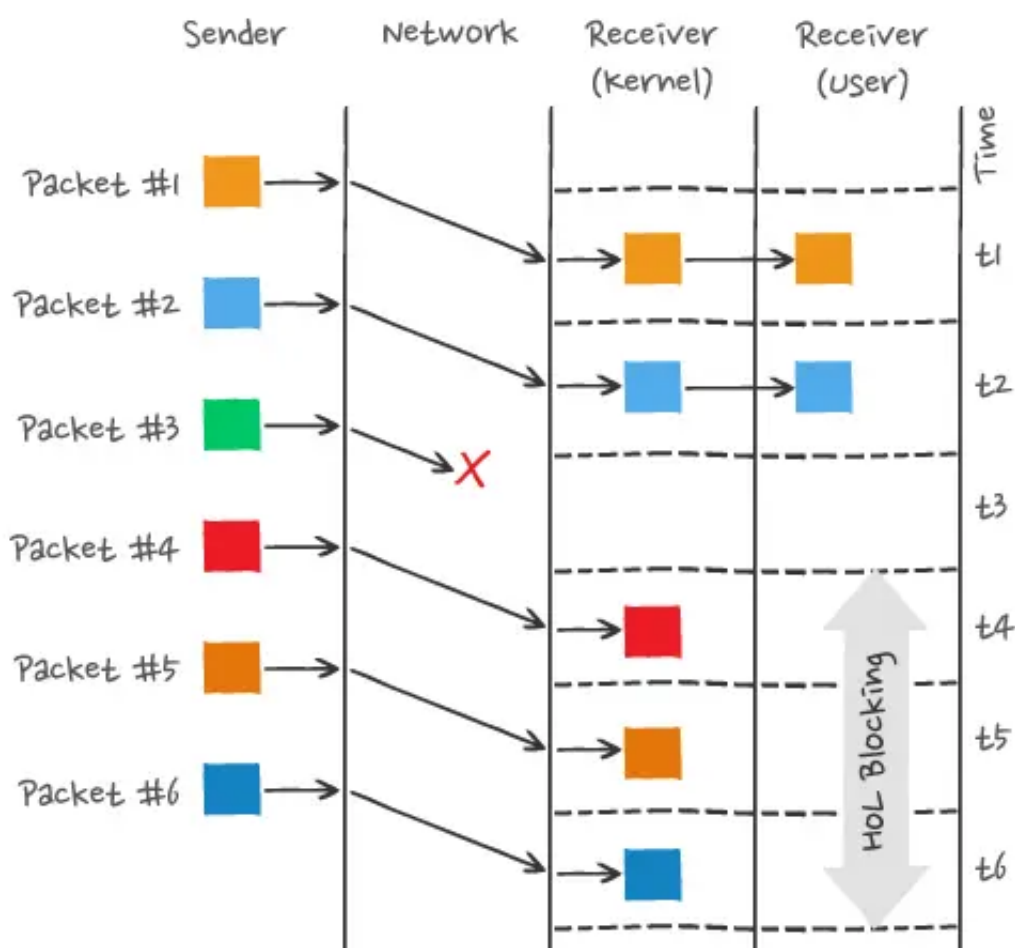
HTTP/2 通过 Stream 的并发能力，解决了 HTTP/1 队头阻塞的问题，看似很完美了，但是 HTTP/2 还是存在“队头阻塞”的问题，只不过问题不是在 HTTP 这一层面，而是在 TCP 这一层。

HTTP/2 是基于 TCP 协议来传输数据的，TCP 是字节流协议，TCP 层必须保证收到的字节数据是完整且连续的，这样内核才会将缓冲区里的数据返回给 HTTP 应用，那么当「前 1 个字节数据」没有到达时，后收到的字节数据只能存放在内核缓冲区里，只有等到这 1 个字节数据到达时，HTTP/2 应用层才能从内核中拿到数据，这就是 HTTP/2 队头阻塞问题。



img

举个例子，如下图：



img

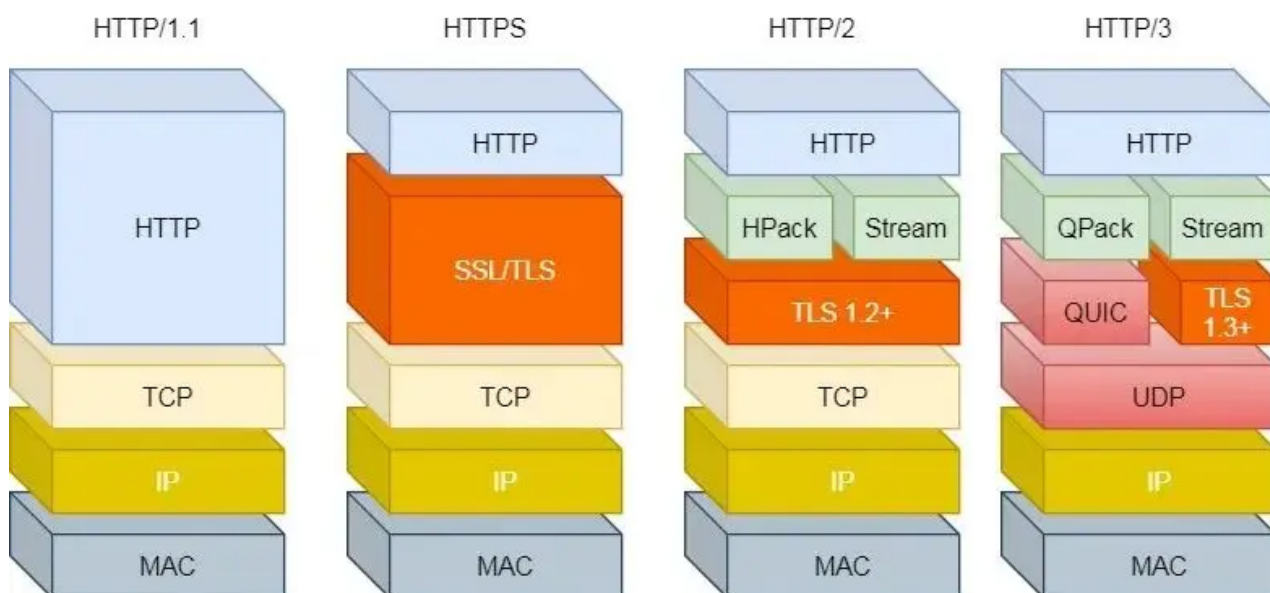
图中发送方发送了很多个 packet，每个 packet 都有自己的序号，你可以认为是 TCP 的序列号，其中 packet 3 在网络中丢失了，即使 packet 4-6 被接收方收到后，由于内核中的 TCP 数据不是连续的，于是接收方的应用层就无法从内核中读取到，只有等到 packet 3 重传后，接收方的应用层才可以从内核中读取到数据，这就是 HTTP/2 的队头阻塞问题，是在 TCP 层面发生的。

所以，一旦发生了丢包现象，就会触发 TCP 的重传机制，这样在一个 TCP 连接中的所有的 HTTP 请求都必须等待这个丢了包被重传回来。

前面我们知道了 HTTP/1.1 和 HTTP/2 都有队头阻塞的问题：

- HTTP/1.1 中的管道（pipeline）虽然解决了请求的队头阻塞，但是没有解决响应的队头阻塞，因为服务端需要按顺序响应收到的请求，如果服务端处理某个请求消耗的时间比较长，那么只能等响应完这个请求后，才能处理下一个请求，这属于 HTTP 层队头阻塞。
- HTTP/2 虽然通过多个请求复用同一个 TCP 连接解决了 HTTP 的队头阻塞，但是一旦发生丢包，就会阻塞住所有的 HTTP 请求，这属于 TCP 层队头阻塞。

HTTP/2 队头阻塞的问题是因为 TCP，所以 HTTP/3 把 HTTP 下层的 TCP 协议改成了 UDP！



HTTP/1 ~ HTTP/3

UDP 发送是不管顺序，也不管丢包的，所以不会出现像 HTTP/2 队头阻塞的问题。大家都知道 UDP 是不可靠传输的，但基于 UDP 的 **QUIC 协议** 可以实现类似 TCP 的可靠性传输。

QUIC 有以下 3 个特点。

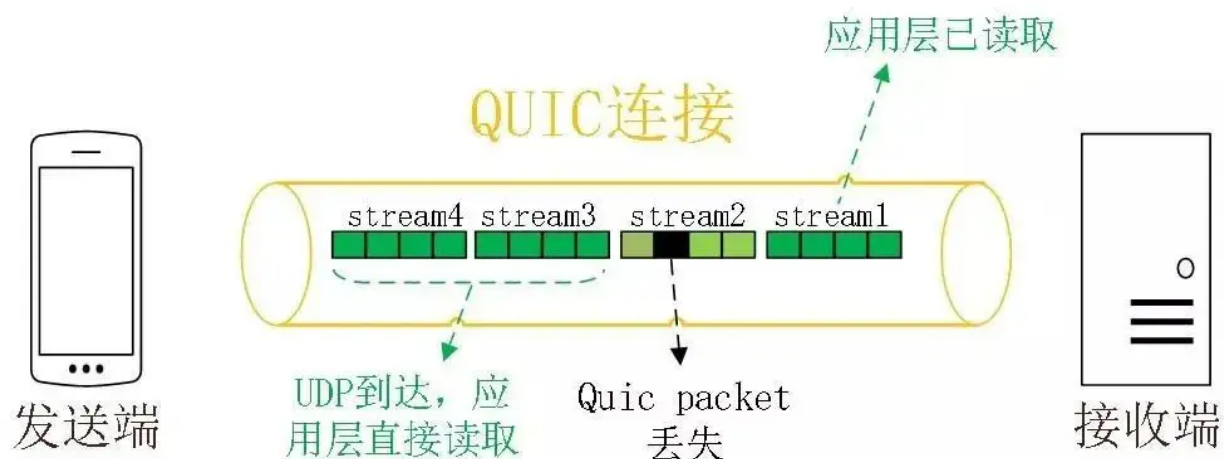
- 无队头阻塞
- 更快的连接建立
- 连接迁移

1、无队头阻塞

QUIC 协议也有类似 HTTP/2 Stream 与多路复用的概念，也是可以在同一条连接上并发传输多个 Stream，Stream 可以认为就是一条 HTTP 请求。

QUIC 有自己的一套机制可以保证传输的可靠性的。当某个流发生丢包时，只会阻塞这个流，其他流不会受到影响，因此不存在队头阻塞问题。这与 HTTP/2 不同，HTTP/2 只要某个流中的数据包丢失了，其他流也会因此受影响。

所以，QUIC 连接上的多个 Stream 之间并没有依赖，都是独立的，某个流发生丢包了，只会影响该流，其他流不受影响。



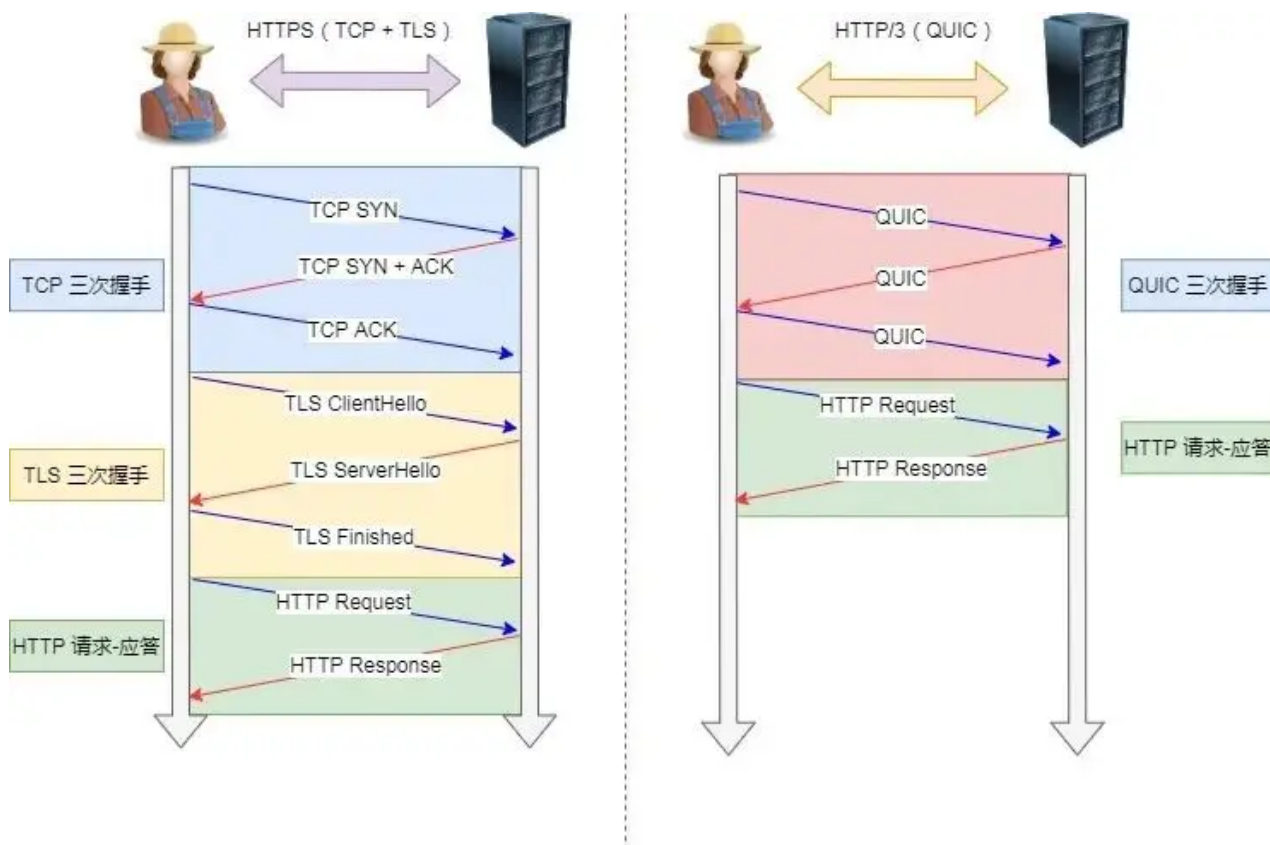
img

2、更快的连接建立

对于 HTTP/1 和 HTTP/2 协议，TCP 和 TLS 是分层的，分别属于内核实现的传输层、openssl 库实现的表示层，因此它们难以合并在一起，需要分批次来握手，先 TCP 握手，再 TLS 握手。

HTTP/3 在传输数据前虽然需要 QUIC 协议握手，但是这个握手过程只需要 1 RTT，握手的目的是为确认双方的「连接 ID」，连接迁移就是基于连接 ID 实现的。

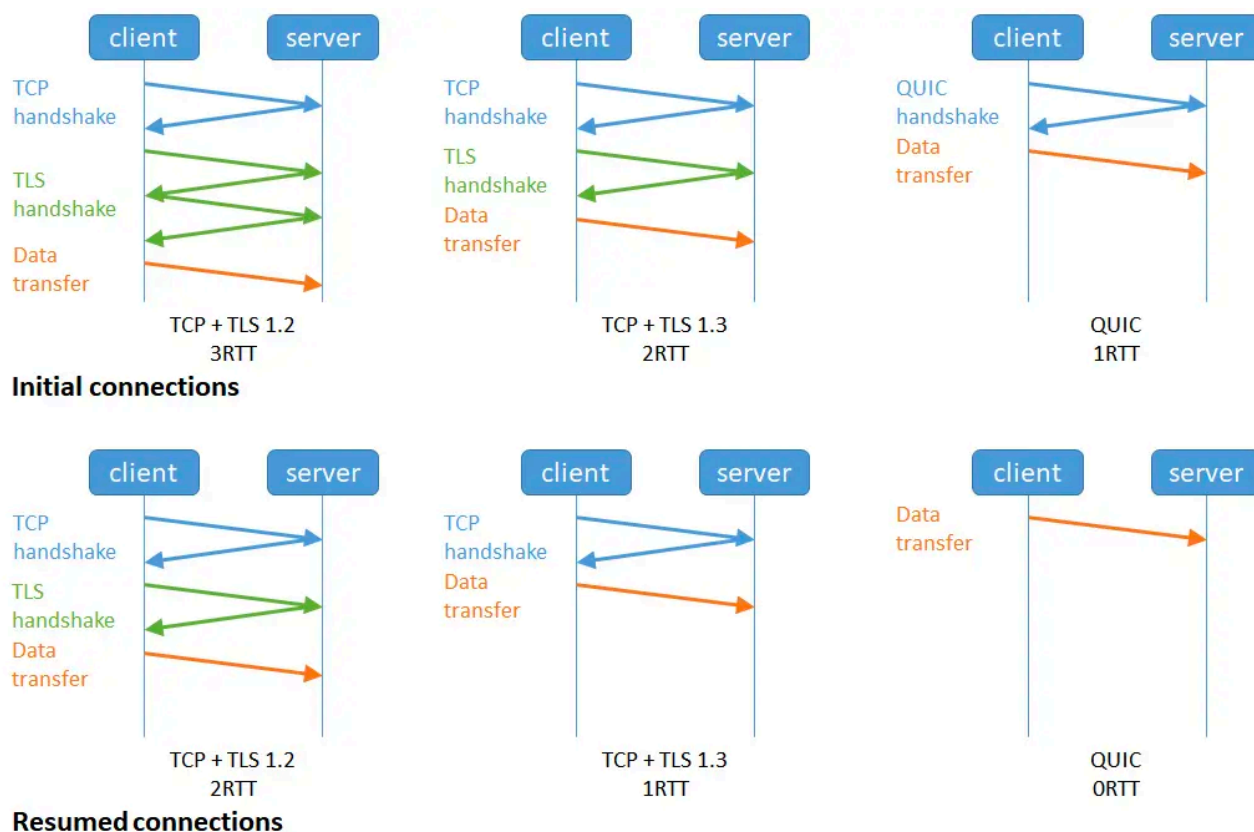
但是 HTTP/3 的 QUIC 协议并不是与 TLS 分层，而是 QUIC 内部包含了 TLS，它在自己的帧会携带 TLS 里的“记录”，再加上 QUIC 使用的是 TLS/1.3，因此仅需 1 个 RTT 就可以「同时」完成建立连接与密钥协商，如下图：



TCP HTTPS (TLS/1.3) 和 QUIC HTTPS

甚至，在第二次连接的时候，应用数据包可以和 QUIC 握手信息（连接信息 + TLS 信息）一起发送，达到 0-RTT 的效果。

如下图右边部分，HTTP/3 当会话恢复时，有效负载数据与第一个数据包一起发送，可以做到 0-RTT（下图的右下角）：



img

3、连接迁移

基于 TCP 传输协议的 HTTP 协议，由于是通过四元组（源 IP、源端口、目的 IP、目的端口）确定一条 TCP 连接。



TCP 四元组

那么当移动设备的网络从 4G 切换到 WIFI 时，意味着 IP 地址变化了，那么就必须断开连接，然后重新建立连接。而建立连接的过程包含 TCP 三次握手的时延，以及 TCP 慢启动的减速过程，给用户的感受就是网络突然卡顿了一下，因此连接的迁移成本是很高的。

而 QUIC 协议没有用四元组的方式来“绑定”连接，而是通过**连接 ID** 来标记通信的两个端点，客户端和服务端可以各自选择一组 ID 来标记自己，因此即使移动设备的网络变化后，导致 IP 地址变化了，只要仍保有上下文信息（比如连接 ID、TLS 密钥等），就可以“无缝”地复用原连接，消除重连的成本，没有丝毫卡顿感，达到了**连接迁移**的功能。

口述回答题目：MySQL联合索引

一开始给了个写sql语句的题，我说我平时用orm框架访问数据库，很少写sql，于是换了个索引的问题。

题目：

index(abc)

1. select * from T where a=x and b=y and c=z
2. select * from T where a=x and b>y and c=z
3. select * from T where c=z and a=x and b=y
4. select (a,b) from T where a=x and b>y
5. select count(*) from T where a=x
6. select count(*) from T where b=y
7. select count(*) from T

读者答：

- （1）根据最左匹配原则，走索引，先匹配a再匹配b再匹配c。
- （2）a走索引，b因为在索引中有序，依旧可以走索引，c需要回表查询。
- （3）因为mysql查询器优化，和（1）一样。
- （4）a走索引，b也走索引，因为ab都在索引上，最后不用回表。
- （5）走索引。
- （6）只有b，无法使用联合索引，需要回表查询。
- （7）不需要回表。

小林补充：

1. a、b、c三个字段都可以走联合索引
2. a和b都会走联合索引，但是由于最左匹配原则，范围查找后面的字段是无法走联合索引的，但是在mysql 5.6 版本后，c 字段虽然无法走联合索引，但是因为有索引下推的特性，c 字段在 innodb 层过滤完满足查询条件的记录后，才返回给server 层进行回表，相比没有索引下推，减少了回表的次数。
3. 查询条件的顺序不影响，优化器会优化，所以a、b、c三个字段都可以走联合索引
4. a和b都会走联合索引，查询是覆盖索引，不需要回表
5. a 可以走联合索引
6. 只有b，无法使用联合索引，由于表存在联合索引，所以 count(*) 选择的扫描方式是扫描联合索引来统计个数，扫描的方式是type=index
7. 由于表存在联合索引，所以 count(*) 选择的扫描方式是扫描联合索引来统计个数，扫描的方式是 type=index

关于 count(*) 为什么选择扫描联合索引（二级索引），而不扫描聚簇索引的原因：这是因为相同数量的二级索引记录可以比聚簇索引记录占用更少的存储空间，所以二级索引树比聚簇索引树小，这样遍历二级索引的 I/O 成本比遍历聚簇索引的 I/O 成本小，因此「优化器」优先选择的是二级索引。

Redis怎么实现分布式锁

读者答：应用程序获取redis的连接，然后发送一个命令获取锁。（面试官：什么命令？）SETNX。因为这个指令是Redis里的原子命令，可以把一个key设置其对应的value。然后这个时候这个应用程序就就开始去临界区进行一些操作。如果说如果说他看来运他执行那个命令失败了。就说明这个锁已经被其他的户端持有了，这样的话他就需要等待。成功获取锁并且执行完临界区操作之后，便可以用del命令在redis删除这个key，实现释放锁的目的。

小林补充：

Redis 的 SET 命令有个 NX 参数可以实现「key不存在才插入」，所以可以用它来实现分布式锁：

- 如果 key 不存在，则显示插入成功，可以用来表示加锁成功；
- 如果 key 存在，则会显示插入失败，可以用来表示加锁失败。

基于 Redis 节点实现分布式锁时，对于加锁操作，我们需要满足三个条件。

- 加锁包括了读取锁变量、检查锁变量值和设置锁变量值三个操作，但需要以原子操作的方式完成，所以，我们使用 SET 命令带上 NX 选项来实现加锁；
- 锁变量需要设置过期时间，以免客户端拿到锁后发生异常，导致锁一直无法释放，所以，我们在 SET 命令执行时加上 EX/PX 选项，设置其过期时间；
- 锁变量的值需要能区分来自不同客户端的加锁操作，以免在释放锁时，出现误释放操作，所以，我们使用 SET 命令设置锁变量值时，每个客户端设置的值是一个唯一值，用于标识客户端；

满足这三个条件的分布式命令如下：

```
SET lock_key unique_value NX PX 10000
```

- lock_key 就是 key 键；
- unique_value 是客户端生成的唯一的标识，区分来自不同客户端的锁操作；
- NX 代表只在 lock_key 不存在时，才对 lock_key 进行设置操作；
- PX 10000 表示设置 lock_key 的过期时间为 10s，这是为了避免客户端发生异常而无法释放锁。

而解锁的过程就是将 lock_key 键删除（del lock_key），但不能乱删，要保证执行操作的客户端就是加锁的客户端。所以，解锁的时候，我们要先判断锁的 unique_value 是否为加锁客户端，是的话，才将 lock_key 键删除。

可以看到，解锁是有两个操作，这时就需要 Lua 脚本来保证解锁的原子性，因为 Redis 在执行 Lua 脚本时，可以以原子性的方式执行，保证了锁释放操作的原子性。

```
// 释放锁时，先比较 unique_value 是否相等，避免锁的误释放
if redis.call("get",KEYS[1]) == ARGV[1] then
    return redis.call("del",KEYS[1])
else
    return 0
end
```

这样一来，就通过使用 SET 命令和 Lua 脚本在 Redis 单节点上完成了分布式锁的加锁和解锁。

思考题：抛硬币，先抛到正面算赢，否则轮流抛。问先抛的人获胜的概率。

读者答：阿巴阿巴.....考虑一轮，（正，-）（反，正）（反，反），其中，（正，-）可以还原成（正，正）（正，）两份，而（反，反）相当于再来一次，这样（正，-）占2份，（反，正）占1份，赢的概率就算2/3

算法题：寻找峰值

给定一个输入数组nums，其中 $\text{nums}[i] \neq \text{nums}[i+1]$ ，找到峰值元素并返回其索引。

数组可能包含多个峰值，在这种情况下，返回任何一个峰值所在位置即可。

你可以假设 $\text{nums}[-1] = \text{nums}[n] = -\infty$ 。

读者答：

- （1）遍历递增序列，遇到第一个非递增的位置返回，时间复杂度为 $O(n)$ 。
- （2）寻找最大值，时间复杂度依旧为 $O(n)$ 。
- （3）二分查找变体，在二分查找时，取中间位置m，并与它相邻位置m+1进行比较，如果m大于m+1，说明峰值应该在左侧，否则应该在右侧，移动对应的左右边界。

反问

（1）在面试的过程中，我存在什么问题，需要在哪些方面加强学习？或者说想要从事后端开发，我还欠缺什么知识？

面试官：数据库的部分你可以多研究一下。微服务的部分可能也可以研究一下，因为像我刚刚其实问你不能讲微服务，我其实是希望你能懂的，你懂的话我就问这方面的问题。

（2）假如我有机会加入进行实习，大概会做些什么样的内容？

面试官：后端这块，我们做的B2B的，我们的产品形态会跟数据库有一点像。如果能够深度的去，了解一下数据库的实现、怎么设计的，那可能会比较好一点。

面试总结

感觉

感觉整体沟通比较顺畅，面试官很和蔼，在我回答有问题或者卡壳的时候能很好的引导，给人感觉很舒服。

不足之处

1. 有些没准备的问题回答的不是很好，比如对比http1.0和2.0,答的很没有条理。
2. 回答过程中不必要的语气词和废话过多，对于自己不确定的问题给人感觉印象很差。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

这次轮到阿里了！

大家好，我是小林。

最近分享了很多字节、腾讯、美团的春招后端面经，有同学问有没有阿里巴巴的？

这不来了嘛，今天就补上了。

这次是一位读者阿里巴巴春招的后端面经，问了比较多的计算机基础和数据库的内容。

操作系统

一个操作系统，我们在衡量它的内存占用时候，它一般会有哪些内存的部分？

读者答：堆和栈

小林补充：

这个其实是问你对free命令的理解。

free 命令有什么信息？

```
[root@registry ~]#  
[root@registry ~]# free  
              total        used        free      shared  buff/cache   available  
Mem:        16267000      7268364      1122432       361040       7876204      8145512  
Swap:       33550332       990332      32560000  
[root@registry ~]#  
[root@registry ~]#  
[root@registry ~]#
```

第一列

Mem 内存的使用信息

Swap 交换空间的使用信息

第一行

total 系统总的可用物理内存大小

used 已被使用的物理内存大小

free 还有多少物理内存可用

shared 被共享使用的物理内存大小

buff/cache 被 buffer 和 cache 使用的物理内存大小

available 还可以被 *应用程序* 使用的物理内存大小

free 与 available 的区别

free 是真正尚未被使用的物理内存数量。

available 是应用程序认为可用内存数量， $\text{available} = \text{free} + \text{buffer} + \text{cache}$ (注：只是大概的计算方法)

Linux 为了提升读写性能，会消耗一部分内存资源缓存磁盘数据，对于内核来说，buffer 和 cache 其实都属于已经被使用的内存。但当应用程序申请内存时，如果 free 内存不够，内核就会回收 buffer 和 cache 的内存来满足应用程序的请求。

主机的内存做一些清理的动作。你知道这里面会涉及到对哪些内存区域进行操作吗？

读者答：不了解

小林补充：

系统内存紧张的时候，就会进行回收内存的工作，那具体哪些内存是可以被回收的呢？

主要有两类内存可以被回收，而且它们的回收方式也不同。

- **文件页 (File-backed Page)**：内核缓存的磁盘数据 (Buffer) 和内核缓存的文件数据 (Cache) 都叫作文件页。大部分文件页，都可以直接释放内存，以后有需要时，再从磁盘重新读取就可以了。而那些被应用程序修改过，并且暂时还没写入磁盘的数据 (也就是脏页)，就得先写入磁盘，然后才能进行内存释放。所以，回收干净页的方式是直接释放内存，回收脏页的方式是先写回磁盘后再释放内存。
- **匿名页 (Anonymous Page)**：这部分内存没有实际载体，不像文件缓存有硬盘文件这样一个载体，比如堆、栈数据等。这部分内存很可能还要再次被访问，所以不能直接释放内存，它们回收的方式是通过 **Linux 的 Swap 机制**，Swap 会把不常访问的内存先写到磁盘中，然后释放这些内存，给其他更需要的进程使用。再次访问这些内存时，重新从磁盘读入内存就可以了。

文件页和匿名页的回收都是基于 LRU 算法，也就是优先回收不常访问的内存。LRU 回收算法，实际上维护着 active 和 inactive 两个双向链表，其中：

- **active_list** 活跃内存页链表，这里存放的是最近被访问过（活跃）的内存页；
- **inactive_list** 不活跃内存页链表，这里存放的是很少被访问（非活跃）的内存页；

越接近链表尾部，就表示内存页越不常访问。这样，在回收内存时，系统就可以根据活跃程度，优先回收不活跃的内存。

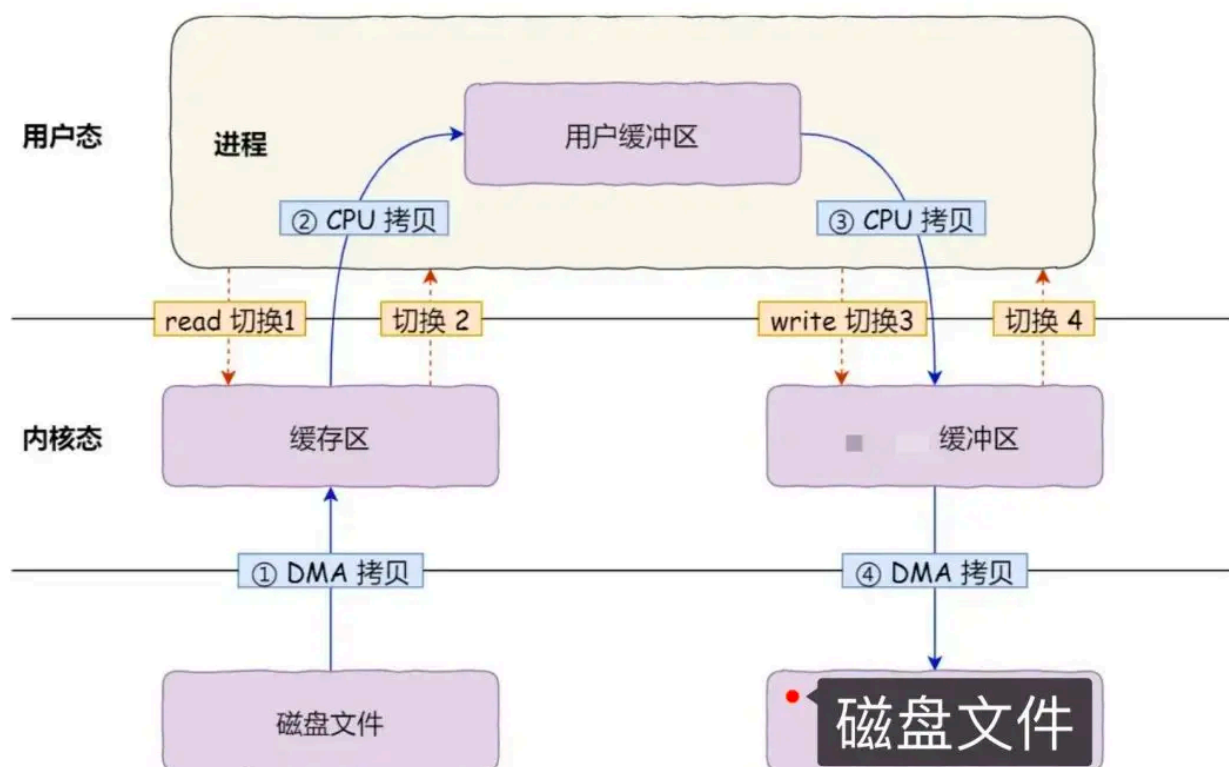
从 a 文件 copy 到另外一个目录， b 作为一个从 a 目录 copy 到一个 b 目录这样的文件，操作过程中包含了哪些系统调用？这里面执行了多少次拷贝的动作？

读者答：不知道

小林补充：

简单理解这个过程就是，在另外一个目录创建a文件，然后通过read读取原来a文件的数据，然后write写入到新目录的a文件。

发生了 4 次数据拷贝，其中两次是 DMA 的拷贝，另外两次则是通过 CPU 拷贝的。



下面说一下这个过程：

- 第一次拷贝，把磁盘上的数据拷贝到操作系统内核的缓冲区里，这个拷贝的过程是通过 DMA 搬运的。
- 第二次拷贝，把内核缓冲区的数据拷贝到用户的缓冲区里，于是我们应用程序就可以使用这部分数据了，这个拷贝到过程是由 CPU 完成的。
- 第三次拷贝，把刚才拷贝到用户的缓冲区里的数据，再拷贝到内核缓冲区里，这个过程依然还是由 CPU 搬运的。

- 第四次拷贝，把内核缓冲区里的数据，拷贝到磁盘，这个过程又是由 DMA 搬运的。

数据库

NoSQL和关系型数据库的区别

读者答：

他们主要的差异在于：关系型数据库它是有比较统一的格式组成，表与表之间是会有关系的映射的。但是NoSQL基本上键值对的形式，能够在不同的场景中提供不一样的业务。比如像MongoDB，它属于文档式的存储，还有一些图数据库相关的数据库，它就没有关系型数据库这么强的关系性。然后主要的一个差距就是在这里。

小林补充：

NoSQL和关系型数据库是两种不同的数据库类型，它们具有一些显著的区别。以下是从多个角度分析这两种数据库类型的区别：

1. 数据模型：关系型数据库采用表格模型来存储数据，表格中的每行都代表一个记录，每列代表一个属性或字段。而NoSQL数据库则采用不同的数据模型，如文档、键值对、图形和列式等。
2. 数据结构：关系型数据库的数据结构严格，要求每个表都必须有一个预定义的模式（schema），包括数据类型、大小、关系等。而NoSQL数据库没有这种限制，允许数据结构的动态变化。
3. 可扩展性：NoSQL数据库通常具有很好的可扩展性，可以通过添加更多的节点来水平扩展，以应对大规模数据处理和高并发访问的需求。关系型数据库也可以进行垂直扩展，即增加更多的处理能力和存储空间，但这种方式的扩展存在一定的局限性。
4. ACID特性：关系型数据库通常具有ACID特性（原子性、一致性、隔离性和持久性），它们可以确保数据的完整性和一致性。而NoSQL数据库通常不支持完全的ACID特性，但提供了更高的可用性和性能。
5. 查询语言：关系型数据库通常使用SQL语言来查询和操作数据，这种语言简单易学，能够完成复杂的数据查询和统计分析。而NoSQL数据库则使用不同的查询语言，如MongoDB的Mongo Query和Cassandra的CQL等。

MySQL表里的数据很多，如何加快数据的查询？

读者答：可以建立索引来提高数据的查询速度。

小林补充：

当你想查阅书中某个知识的内容，你会选择一页一页的找呢？还是在书的目录去找呢？

傻瓜都知道时间是宝贵的，当然是选择在书的目录去找，找到后再翻到对应的页。书中的目录，就是充当索引的角色，方便我们快速查找书中的内容，所以索引是以空间换时间的设计思想。

那换到数据库中，索引的定义就是帮助存储引擎快速获取数据的一种数据结构，形象的说就是**索引是数据的目录**。

所以，要加快数据的查询，就是通过建立索引来提高查询的速度。

索引是怎么实现的？为什么它就能加快查询的？

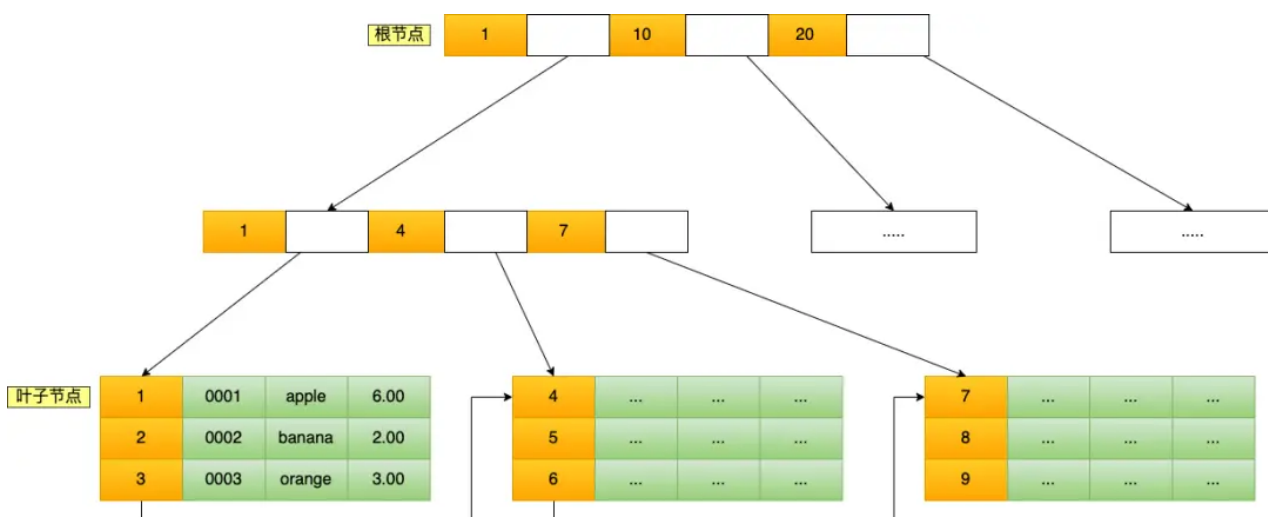
读者答：我个人理解，索引是一种数据的组织结构，它能够根据场景的不同，采取不同的数据结构存储数据，加快数据的查询。对于索引，如果内存够大，或追求极致的查找效率，当然哈希索引会是很快的。如果B+树的结构适合存到磁盘上。这样子的索引结构是比较适合存在磁盘像 SSD 这样子的磁盘结构上的，因为它提供了一个它一个叶子节点，像跟磁盘的结构差不多，一个叶子节点大概有 16 KB 的大小，跟磁盘的一个页大小是基本上能够整数倍吻合。同时也能够提供一些范围查找，加快数据的读取速度。

小林补充：

Innodb 存储引擎的索引是 B+ 树结构，我以 B+ 树作为例子。

B+Tree 是一种多叉树，叶子节点才存放数据，非叶子节点只存放索引，而且每个节点里的数据是按主键顺序存放的。每一层父节点的索引值都会出现在下层子节点的索引值中，因此在叶子节点中，包括了所有的索引值信息，并且每一个叶子节点都有两个指针，分别指向下一个叶子节点和上一个叶子节点，形成一个双向链表。

主键索引的 B+Tree 如图所示（图中叶子节点之间我画了单向链表，但是实际上是双向链表，原图我找不到了，修改不了，偷个懒我不重画了，大家脑补成双向链表就行）：



主键索引 B+Tree

比如，我们执行了下面这条查询语句：

```
select * from product where id= 5;
```

这条语句使用了主键索引查询 id 号为 5 的商品。查询过程是这样的，B+Tree 会自顶向下逐层进行查找：

- 将 5 与根节点的索引数据 (1, 10, 20) 比较，5 在 1 和 10 之间，所以根据 B+Tree 的搜索逻辑，找到第二层的索引数据 (1, 4, 7)；
- 在第二层的索引数据 (1, 4, 7) 中进行查找，因为 5 在 4 和 7 之间，所以找到第三层的索引数据 (4, 5, 6)；
- 在叶子节点的索引数据 (4, 5, 6) 中进行查找，然后我们找到了索引值为 5 的行数据。

正是因为 B+ 树的索引是有序存放的，所以我们可以通过类似于二分查找算法快速找到对应的数据。

B+树具体实现？

读者答：B+树它是一种树形结构，在这棵树里它是能支持多个节点的（多叉树）。也就是相比于平衡二叉树来说，它一层能够存储的节点数量是更多的。B+树它比较特殊的地方是在于，除了叶子节点以外，它存的基本上都是路径信息，index 信息。只有在最后一层页节点才会存储具体的数据信息。

B+和红黑树，B树的区别

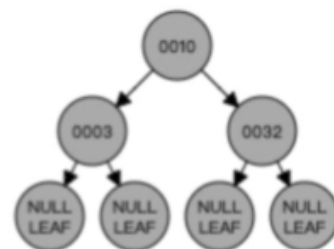
读者答：

- 红黑树，它其实也是二叉树的一种，它叶节点只有 red 和 black 两种类型。所以相比于B+数来说，它整个树的结构，深度高度是会更高的。对于查询来说，高度更高意味着就是查的效率会更低这样子。所以B+树会比红黑树更加的矮胖，它一层能存储的数据会更多一点。
- B树呢，也是个多叉树，但是在一些数据扫描的情况下，b 树可能会有一个回溯的过程，但是B+树就可以直接的通过遍历的方式来查找。

小林补充：

红黑树，它也是通过一些约束条件来达到自平衡，不过红黑树的约束条件比较复杂，不是本篇的重点重点，大家可以看《数据结构》相关的书籍来了解红黑树的约束条件。

下面是红黑树插入节点的过程，这左旋右旋的操作，就是为了自平衡。



图片

不管平衡二叉查找树还是红黑树，都会随着插入的元素增多，而导致树的高度变高，这就意味着磁盘 I/O 操作次数多，会影响整体数据查询的效率。

虽然平衡二叉查找树和红黑树能保持查询操作的时间复杂度在 $O(\log n)$ ，但是因为它本质上是一个二叉树，每个节点只能有 2 个子节点，那么当节点个数越来越多的时候，树的高度也会相应变高，这样就会增加磁盘的 I/O 次数，从而影响数据查询的效率。

为了解决降低树的高度的问题，后面就出来了 B 树和 B+，它不再限制一个节点就只能是 2 个子节点，而是允许 M 个子节点 ($M > 2$)，从而降低树的高度，这样就可以减少磁盘的 I/O 次数。

B 树和 B+ 都是通过多叉树的方式，会将树的高度变矮，所以这两个数据结构非常适合检索存于磁盘中的数据。但是 MySQL 默认的存储引擎 InnoDB 采用的是 B+ 作为索引的数据结构，原因有：

- B+ 树的非叶子节点不存放实际的记录数据，仅存放索引，因此数据量相同的情况下，相比存储即存索引又存记录的 B 树，B+ 树的非叶子节点可以存放更多的索引，因此 B+ 树可以比 B 树更「矮胖」，查询底层节点的磁盘 I/O 次数会更少。
- B+ 树有大量的冗余节点（所有非叶子节点都是冗余索引），这些冗余索引让 B+ 树在插入、删除的

效率都更高，比如删除根节点的时候，不会像 B 树那样会发生复杂的树的变化；

- B+ 树叶子节点之间用链表连接了起来，有利于范围查询，而 B 树要实现范围查询，因此只能通过树的遍历来完成范围查询，这会涉及多个节点的磁盘 I/O 操作，范围查询效率不如 B+ 树。

我们什么时候会用到事务？

读者答：有多个操作，并且想要这个操作是原子性，要不这多个操作一起成功，要不就一起失败。这样的场景会需要事务。

小林补充：

这是我的钱包，共有 100 万元。

小林的钱包

¥1,000,000.00

图片

今天我心情好，我决定给你的转账 100 万，最后的结果肯定是我的余额变为 0 元，你的余额多了 100 万元，是不是想到就很开心？

转账这一动作在程序里会涉及到一系列的操作，假设我向你转账 100 万的过程是有下面这几个步骤组成的：

1. 从数据库读取我的余额
2. 将我的余额**减去**转账的金额
3. 将我修改后的余额更新到数据库里

图片

4. 从数据库读取你的余额
5. 将你的余额**加上**转账的金额
6. 将你修改后的余额更新到数据库里

可以看到这个转账的过程涉及到了两次修改数据库的操作。

假设在执行第三步骤之后，服务器忽然掉电了，就会发生一个蛋疼的事情，我的账户扣了 100 万，但是钱并没有到你的账户上，也就是说这 **100 万消失了！**

要解决这个问题，就要保证转账业务里的所有数据库的操作是不可分割的，要么全部执行成功，要么全部失败，不允许出现中间状态的数据。

数据库中的「**事务 (*Transaction*)**」就能达到这样的效果。

我们在转账操作前先开启事务，等所有数据库操作执行完成后，才提交事务，对于已经提交的事务来说，该事务对数据库所做的修改将永久生效，如果中途发生中断或错误，那么该事务期间对数据库所做的修改将会被回滚到没执行该事务之前的状态。

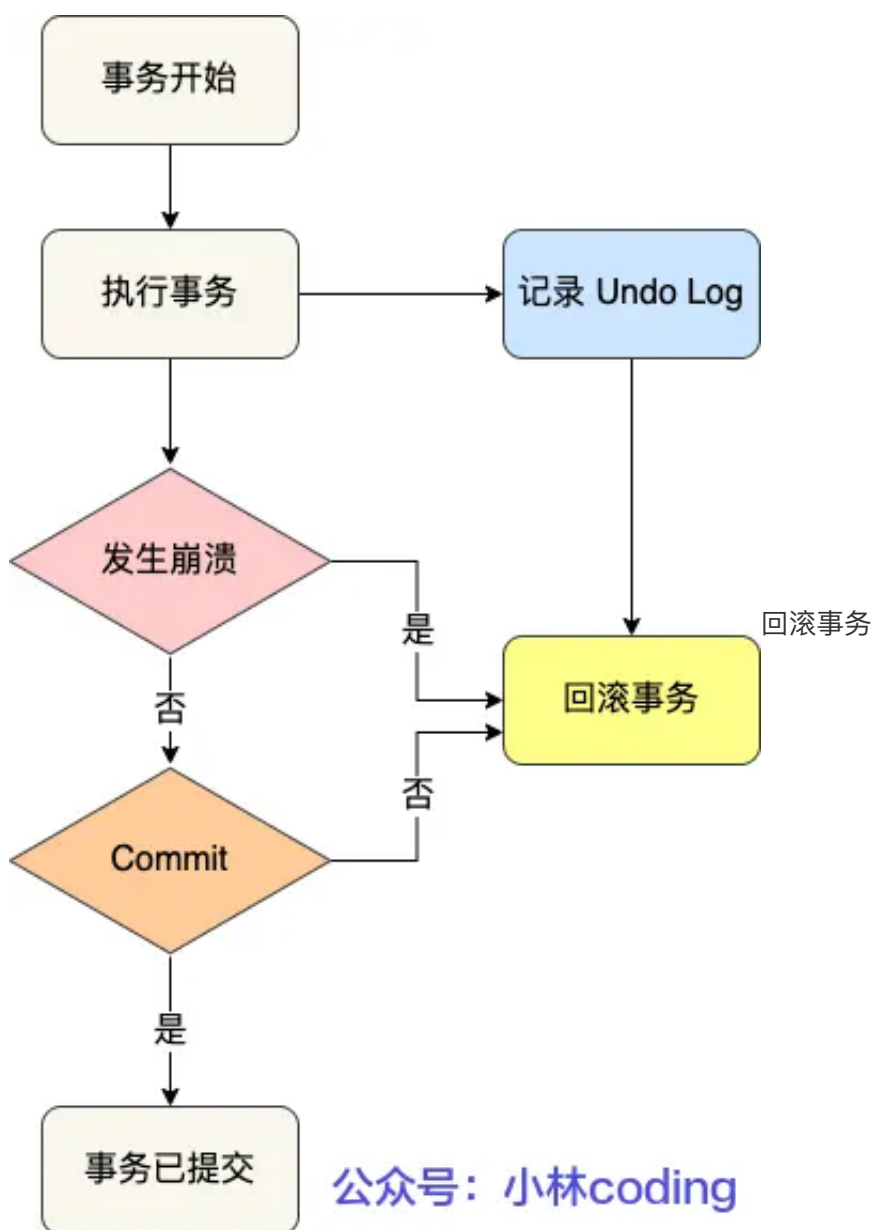
一起成功，一起失败的原子性怎么保证的

读者答：如果执行失败就回滚。如果执行成功，就持久化，使用undo log来进行保证原子性的。

小林补充：

undo log（回滚日志）保证了事务的 ACID 特性中的原子性（Atomicity）。

undo log 是一种用于撤销回退的日志。在事务没提交之前，MySQL 会先记录更新前的数据到 undo log 日志文件里面，当事务回滚时，可以利用 undo log 来进行回滚。如下图：



每当 InnoDB 引擎对一条记录进行操作（修改、删除、新增）时，要把回滚时需要的信息都记录到 undo log 里，比如：

- 在插入一条记录时，要把这条记录的主键值记下来，这样之后回滚时只需要把这个主键值对应的记录删掉就好了；

- 在删除一条记录时，要把这条记录中的内容都记下来，这样之后回滚时再把由这些内容组成的记录插入到表中就好了；
- 在更新一条记录时，要把被更新的列的旧值记下来，这样之后回滚时再把这些列更新为旧值就好了。

在发生回滚时，就读取 undo log 里的数据，然后做原先相反操作。比如当 delete 一条记录时，undo log 中会把记录中的内容都记下来，然后执行回滚操作的时候，就读取 undo log 里的数据，然后进行 insert 操作。

假如我们现在对已经插入了一条数据，我们接下来去修正另外一条数据。被查入的这条数据在这个时候能够被查询到吗？

读者答：要看数据库的隔离级别，如果它的隔离级别比较高，这个事务没有完成之前，他对数据的修改，另一个事务是没有办法感知到的。但是如果他事务的隔离级别比较低，是可以能够查到的。

一般有哪些隔离级别

读者答：大概有 4 种隔离级别读未提交，读已提交，可重复读和串行化情况。

小林补充：

SQL 标准提出了四种隔离级别来规避这些现象，隔离级别越高，性能效率就越低，这四个隔离级别如下：

- 读未提交 (read uncommitted)，指一个事务还没提交时，它做的变更就能被其他事务看到；
- 读提交 (read committed)，指一个事务提交之后，它做的变更才能被其他事务看到；
- 可重复读 (repeatable read)，指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，MySQL InnoDB 引擎的默认隔离级别；
- 串行化 (serializable)；会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生了读写冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行；

按隔离水平高低排序如下：



图片

针对不同的隔离级别，并发事务时可能发生的现象也会不同。



图片

也就是说：

- 在「读未提交」隔离级别下，可能发生脏读、不可重复读和幻读现象；
- 在「读提交」隔离级别下，可能发生不可重复读和幻读现象，但是不可能发生脏读现象；
- 在「可重复读」隔离级别下，可能发生幻读现象，但是不可能脏读和不可重复读现象；
- 在「串行化」隔离级别下，脏读、不可重复读和幻读现象都不可能发生。

所以，要解决脏读现象，就要升级到「读提交」以上的隔离级别；要解决不可重复读现象，就要升级到「可重复读」的隔离级别，要解决幻读现象不建议将隔离级别升级到「串行化」。

不同的数据库厂商对 SQL 标准中规定的 4 种隔离级别的支持不一样，有的数据库只实现了其中几种隔离级别，我们讨论的 **MySQL** 虽然支持 4 种隔离级别，但是与 **SQL** 标准中规定的各级隔离级别允许发生的现象却有些出入。

MySQL 在「可重复读」隔离级别下，可以很大程度上避免幻读现象的发生（注意是很大程度避免，并不是彻底避免），所以 MySQL 并不会使用「串行化」隔离级别来避免幻读现象的发生，因为使用「串行化」隔离级别会影响性能。

Go

Why Go?

读者答：go支持高并发，在我的研究领域基本都是用go，有比较好的生态。

小林补充：

云原生像k8s,docker都是go实现，对go也有比较好的api支持，然后go在并发生而也有天然的优势，语法也简单

为什么Goroutine支持高并发

读者答：相比于线程来说，它所占用的资源更小的。一个线程可以有多个协程；线程进程可以看作是同步机制，而协程非阻塞模式。和channel的组合能让开发的过程中不会太过于去考虑事物的并发控制，因为通过 channel 可以进行控制，因为是协程，他们其实共享了一些资源，所以他们其实可以通过共享内存的方式去进行相关的交互。相对比起线程的加锁机制来说，协程的效率会更高一些。

小林补充：

1. 协程大小相对于县城来说更笑，大小只有2k，内存中可以允许有更多的携程来处理并发任务
2. 协程的上下文切换不哦那个由用户态切换到内核态，且保存的寄存器值更少，所以切成切换的开销更小，效率更高
3. go语言在语言层面go语言的runtime内置了协程调度器，整体通过gmp模型高效的调度goroutine的运行

开销大的话，用线程池不就能解决吗？

读者答：可以解决，但是需要用户自己建线程池也需要开销，还需要对所有的线程手动管理。

小林补充：

线程池虽然可以控制线程的数量到那时线程的上下文切换开销相比于协程更大，需要用户态到内核态的切换。并且还需要自己实现和维护线程池，实现起来复杂

开销是指什么？你觉得线程池需要做哪些管理？

读者答：维护过期的线程，具体的不清楚了。

小林补充：

开销指线程切换的开销，用户态到内核态的切换，以及执行上下文的保存

1、corePoolSize（核心线程数量）

向线程池提交一个任务，此时，若线程池已创建的线程数小于corePoolSize，即便此时存在空闲线程，也会通过创建一个新线程来执行该任务，直到已创建的线程数大于或等于corePoolSize时，（除了利用提交新任务来创建和启动线程（按需构造），也可以通过 prestartCoreThread() 或 prestartAllCoreThreads() 方法来提前启动线程池中的基本线程。）

2、maximumPoolSize（最大线程数量）

线程池所允许的最大同时可执行线程数量。当队列满了，且已创建的线程数小于maximumPoolSize，则线程池会创建新的线程来执行任务。另外，对于无界队列，可忽略该参数。

3、keepAliveTime（线程存活保持时间）

当线程池中线程数大于核心线程数时，线程的空闲时间如果超过线程存活时间，那么这个线程就会被销毁，直到线程池中的线程数小于等于核心线程数。

4、workQueue（任务队列）

用于传输和保存等待执行任务的阻塞队列。

5、threadFactory（线程工厂）

用于创建新线程。threadFactory创建的线程也是采用new Thread()方式，threadFactory创建的线程名都具有统一的风格：pool-m-thread-n（m为线程池的编号，n为线程池内的线程编号）。

6、handler（线程饱和策略）

当线程池和队列都满了，再加入线程会执行此策略。

Goroutine调度

读者答：基本上是一个 GMP 的模式。goroutine 会缓存在一个队列里面，等待 p processor 处理者去进行相关的调度。实际执行者是m，是一个线程，也就是 m 的多少基本上取决于CPU。建议数据量和CPU内核差不多一致。

面试总结

感觉

操作系统的知识结束的很快，没有关注底层的技术原理

后面挂了

不足之处

1. 操作系统的底层知识掌握的不行
2. get不到面试官的启发点



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

硬钢百度面试！

大家好，我是小林。

今天分享一位百度春招面经，读者的技术栈是C++。

这次的面经，主要都是问操作系统、网络编程、C++ 这三大方向。

能明显感觉到，C++面试和Java或者Go面试重点，Java/Go主要是问MySQL、Redis。

一、介绍一下webserver项目

1. 服务器开始运行，创建（初始化）线程池（IO密集型，线程数 $n+1$ ）；
2. 创建 epoll 对连接进行监听
3. 监听到连接事件，调用线程池线程处理 http 请求
4. 读取 http 请求并对其进行解析（空格，`\r\n`字段提取）
5. 返回解析结果

二、select、poll、epoll的选择

select缺点：

1. select() 检测数量有限制，最大值通常为 1024 (bit)，每一个比特位对应一个监听的文件描述符
2. fd_set被内核修改后，不可以重用，每次都需要重置
3. 每次调用select，都需要把fd集合从用户态拷贝到内核态，这个开销在fd很多时会很大
4. 每次调用select都需要在内核遍历传递进来的所有fd，这个开销在fd很多时也很大（时间复杂度是 $O(n)$ ）

poll缺点：select第三四条缺点没有解决

1. 每次调用select，都需要把**fd集合从用户态拷贝到内核态，这个开销在fd很多时会很大
2. 每次调用select都需要在内核遍历传递进来的所有fd，这个开销在fd很多时也很大（时间复杂度是 $O(n)$ ）

epoll优点：epoll底层数据结构

- 红黑树增删改综合效率高
- 就绪的描述符的链表。当有的连接就绪的时候，内核会把就绪的连接放到 rdllist 链表里。这样应用进程只需要判断链表就能找出就绪进程，而不用去遍历整棵树。

三、线程和进程的区别？使用线程的心得？

1. 进程是资源（包括内存、打开的文件等）分配的单位，线程是 CPU 调度的单位；
2. （关键词：进程独立空间、线程之前共享空间资源）进程拥有一个独立完整的资源平台，不和其他进程共享；而线程只独享必不可少的资源，如寄存器和栈，而一个进程里可以有多个线程，彼此共享同一个地址空间。
3. 线程同样具有就绪、阻塞、执行三种基本状态，同样具有状态之间的转换关系；
4. 线程能减少并发执行的时间和空间开销

对于，线程相比进程能减少开销，体现在：

- （1. 创建时间少）线程的创建时间比进程快，因为进程在创建的过程中，还需要资源管理信息，比如内存、文件管理信息切换虚拟地址空间，切换内核栈和硬件上下文，页表切换开销很大，而线程在创建的过程中，不会涉及这些信息，而是共享它们，只需保存和设置少量寄存器内容，因此开销很小；
- （2. 终止时间少）线程的终止时间比进程快，因为线程释放的资源相比进程少很多；
- （3. 不需要切换页表，切换时间块）同一个进程内的线程切换比进程切换快，因为线程具有相同的地址空间（虚拟内存共享），这意味着同一个进程的线程都具有同一个页表，那么在切换的时候不需要切换页表。而对于进程之间的切换，切换的时候要把页表给切换掉，而页表的切换过程开销是比较大的；

- (4. 共享、线程之间数据传递效率高) 由于同一进程的各线程间共享内存和文件资源，那么在线程之间数据传递的时候，就不需要经过内核了，这就使得线程之间的数据交互效率更高了；

所以，不管是时间效率，还是空间效率线程比进程都要高

心得：线程使用有一定难度，需要处理数据一致性问题，比如要使用互斥锁和条件变量等同步机制保证线程安全（原子性操作）

四、C++ 空类的大小？一个只包含int 变量的空class和只包含int变量的空struct的内存各占多大？

关键词：空类和空结构体都大小为1，这样可以确保两个不同的对象，拥有不同的地址。

1.空类

```
class A {};  
int main(){  
    cout<<sizeof(A)<<endl;// 输出 1;  
    A a;  
    cout<<sizeof(a)<<endl;// 输出 1;  
    return 0;  
}
```

1. C++空类的大小不为0，不同编译器设置不一样，**vs和lg++都是设置为1**；
2. C++标准指出，不允许一个对象（当然包括类对象）的大小为0，不同的对象不能具有相同的地址；
3. 带有虚函数的C++类大小不为1，**因为每一个对象会有一个vpitr指向虚函数表**，具体大小根据指针大小确定；
4. C++中要求对于类的每个实例都必须有独一无二的地址,那么编译器自动为空类分配一个字节大小，这样便保证了每个实例均有独一无二的内存地址。

在C++中空类会占一个字节，这是为了让对象的实例能够相互区别。具体来说，空类同样可以被实例化，并且每个实例在内存中都有独一无二的地址，因此，编译器会给空类隐含加上一个字节，这样空类实例化之后就会拥有独一无二的内存地址。当该空白类作为基类时，该类的大小就优化为0了，子类的大小就是子类本身的大小。这就是所谓的空白基类最优化。

空类的实例大小就是类的大小，所以sizeof(a)=1字节,如果a是指针，则sizeof(a)就是指针的大小，即4字节。

2.含有虚函数的类的大小

```
class A { virtual Fun(){} };  
int main(){  
    cout<<sizeof(A)<<endl;// 输出 4(32位机器)/8(64位机器);  
    A a;  
    cout<<sizeof(a)<<endl;// 输出 4(32位机器)/8(64位机器);  
    return 0;  
}
```

因为有虚函数的类对象中都有一个虚函数表指针 `_vpitr`，其大小是4字节

3.只含有一个int成员变量的类的大小 (4)

```
class A { int a; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 4;
    A a;
    cout<<sizeof(a)<<endl;// 输出 4;
    return 0;
}
```

只是一个int变量的大小——4字节

4.只含有一个静态成员变量的类的大小 (1)

```
class A { static int a; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 1;
    A a;
    cout<<sizeof(a)<<endl;// 输出 1;
    return 0;
}
```

静态成员存放在静态存储区, 不占用类的大小, 普通函数也不占用类大小

```
class A { static int a; int b; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 4;
    A a;
    cout<<sizeof(a)<<endl;// 输出 4;
    return 0;
}
```

静态成员a不占用类的大小, 所以类的大小就是b变量的大小 即4个字节

五、为什么一般构造函数定义为虚函数？析构函数不定义为虚函数？

为什么析构函数一般写为虚函数？

如果析构函数不被声明成虚函数, 则编译器实施静态绑定, 在删除基类指针时, 只会调用基类的析构函数而不调用派生类析构函数, 这样就会造成派生类对象析构不完全, 造成内存泄漏。

所以在实现多态时, 当用基类操作派生类, 在析构时防止只析构基类而不析构派生类的状况发生, 要将基类的析构函数声明为虚函数。

为什么构造函数不写为虚函数？

从存储空间角度：虚函数对应一个vtable，可是这个vtable其实是存储在对象的内存空间的。问题出来了，如果构造函数是虚的，就需要通过vtable来调用，可是对象还没有实例化，也就是内存空间还没有，无法找到vtable，所以构造函数不能是虚函数。

从使用角度：虚函数的作用在于通过父类的指针或者引用来调用它的时候能够变成调用子类的那个成员函数。而构造函数是在创建对象时自动调用的，不可能通过父类的指针或者引用去调用，因此也就规定构造函数不能是虚函数。

六、static的作用（作用域限制）

static

- 不考虑类的情况
 - 有时候希望某些全局变量或者函数只在本文件中被使用，而不能被其他外部文件引用，这个时候可以在全局变量前加一个static说明，这样不同的人编写不同的变量或者函数时不用担心重名的问题，即使重名了也互不干扰
 - 默认初始化为0，包括未初始化的全局静态变量与局部静态变量，都存在全局未初始化区
 - 静态变量在函数内定义，始终存在，且只进行一次初始化，具有记忆性，其作用范围与局部变量相同，函数退出后仍然存在，但不能使用
- 考虑类的情况
 - static成员变量：只与类关联，不与类的对象关联。定义时要分配空间，不能在类声明中初始化，必须在类定义体外部初始化，初始化时不需要标示为static；可以被非static成员函数任意访问。
 - static成员函数：不具有this指针，无法访问类对象的非static成员变量和非static成员函数；不能被声明为const、虚函数和volatile；可以被非static成员函数任意访问

静态局部变量：

1. 静态局部变量属于静态存储类别，在静态存储区内分配存储单元，在整个程序运行期间始终存在。
2. 静态局部变量只初始化一次，并且之后再次调用函数时不再重新分配空间和赋初值，而保留上次函数调用结束时的值（而普通局部变量每调用一次就会重新分配空间并赋一次初值）
3. 静态局部变量默认初始化为0
4. 函数调用结束之后静态局部变量依然存在，但是只能在该函数内进行使用该静态局部变量，

extern的作用（作用域扩展）

1. 将全局变量的作用域扩展到其定义之前：如果全局变量不在文件的开头定义，其作用范围只限定于从定义处到文件结尾，如果在定义点之前的函数想引用该变量，就应该在引用之前使用extern关键字对该变量进行声明，之后该全局变量的作用域就从声明处一直到文件结尾了
2. 将某一个源文件中全局变量的作用域扩展到其他源文件中：一个C++项目很多情况是由多个源文件构成，如果在一个文件中想引用另一个文件中已定义的全局变量，比如现在两个文件都要使用到同一个全局变量int a，正确的做法应该是：在一个文件中定义变量a，而在另一个文件中使用extern int a；对该变量进行声明，这样就可以两个文件同时使用同一个变量了

const

- 不考虑类的情况
 - **const**常量在定义时必须初始化，之后无法更改
 - **const**形参可以接收**const**和非**const**类型的实参，例如// i 可以是 int 型或者 const int 型void fun(const int& i){ //...}
- 考虑类的情况
 - **const**成员变量：不能在类定义外部初始化，只能通过构造函数初始化列表进行初始化，并且必须有构造函数；不同类对其const数据成员的值可以不同，所以不能在类中声明时初始化。
 - const成员函数：const对象不可以调用非const成员函数；非const对象都可以调用；不可以改变非mutable（用该关键字声明的变量可以在const成员函数中被修改）数据的值。

七、C++ sort()函数实现

sort()源码中采用的是一种叫做IntroSort内省式排序的混合式排序算法，

- 1.首先进行判断排序的元素个数是否大于stl_threshold，stl_threshold是一个常量值是16，意思就是说我传入的元素规模小于我们的16的时候直接采用插入排序。（为什么用插入排序？因为插入排序在面对“几近排序”的序列时，表现更好，而快排是通过递归实现的，会为了极小的子序列产生很多的递归调用在区间长度小的时候经常不如插入排序效率高）
- 2.如果说我们的元素规模大于16，那就需要去判断是不是能采用快速排序，怎么判断呢？快排是使用递归来实现的，如果说我们进行判断我们的递归深度有没有到达递归深度的限制阈值 $2 * \lg(n)$ ，如果递归深度没达到阈值就使用快速排序来进行排序
- 3.如果说大于我们的最深递归深度阈值的话，这个时候说明快排复杂度退化了（比如很不巧基准元素多次选取到了当前区间中最小或最大的元素。这种情况下，每次划分只能将区间缩小1个元素，造成递归深度过深），就会采用我们的堆排序，堆排序是可以保证稳定 $O(n \log n)$ 的时间复杂度的。

难度上来了！阿里面的问题好多啊

大家好，我是小林。

之前分享的面经，比较少 Java 问题，有同学问我有没有相关的 Java 面经。

这不来了嘛，今天分享的是一位读者的阿里实习一面的面经，阿里毕竟是 Java 大厂，这次的面试重点是以 Java 为主，MySQL+网络为辅。

感觉问的问题还挺多的，面试时长有 1小时+。

问题记录

MySQL

1. 问对哪个项目熟悉，为什么想要用java去写关系型数据库？
2. 这个数据库体现了Spring的事务传播机制吗？
3. 可重复读，已提交读，这两个隔离级别表现的现象是什么，区别是什么样的？
4. 数据管理里，数据文件大体分成哪几种数据文件？

5. 日志文件是分成了哪几种?
6. 说下MVCC机制的原理?
7. 索引的类型有哈希索引, B+树索引, 而hash索引的时间复杂度是 $O(1)$, 那为什么我们一般情况下不使用哈希索引, 而使用b+树索引呢?
8. 对一个慢sql怎么去排查?
9. 索引字段是不是建的越多越好?

Java

1. java这一块对框架都是熟悉的吧?
2. MyBatis运用了哪些常见的设计模式?
3. MyBatis中创建了一个Mapper接口, 在写一个xml文件, java的接口是要实现的, 为什么这没有实现呢?
4. 与传统的JDBC相比, MyBatis的优点?
5. 还记得JDBC连接数据库的步骤吗?
6. 怎么理解SpringIOC?
7. 如果让你设计一个SpringIOC, 你觉得会从哪些方面考虑这个设计?
8. Spring给我们提供了很多扩展点, 这些有了解吗?
9. servlet有写过简单的代码吗?
10. 大致了解SpringMVC的处理流程吗?
11. HandlerMapping 和 handlerAdapter有了解吗?
12. SpringAOP主要想解决什么问题
13. SpringAOP的原理了解吗
14. 动态代理和静态代理的区别
15. 代理模式和适配器模式有什么区别?
16. java线程的生命周期有了解吗?
17. 使用多线程要注意哪些问题?
18. 保证数据的一致性有哪些方案呢?
19. 线程池有了解吗? 线程池大概的原理?
20. ArrayList和LinkedList有什么区别
21. 对面向对象的理解?

网络

1. http协议的报文的格式有了解吗?
2. http常用的状态码?

其他

1. 读书中遇到最难的技术是什么, 怎么克服的?
2. 有没有什么强项在面试中还没有展现的?
3. 反问

面试总结

感觉

面试官有引导，大多问的是八股，会的就回答的比较流畅，不熟悉的就磕磕巴巴，面试官给的反馈基础还行，但是深度不够，对刚刚设计模式的对比回答不满意，和mybatis的原理回答也不太满意

不足之处

对框架还是不够熟练，回答不够全面，经常被面试官问还有呢，但就回答不上来了，网络这方面看了就忘。



腾讯面试比之前的要难，开始会胡说八道了

今天又来分享面经了，这次腾讯春招实习的面经，岗位是 java 后端开发。

读者的背景是985硕，根据读者的面试感受说是，腾讯面试比之前的要难，但是增加自信了，遇到不会的问题，开始会胡说八道了。

说一下BIO、NIO和AIO

读者答：

- BIO是阻塞IO。在上一个线程的任务执行完之前，该线程必须阻塞等待上一个线程执行完毕。
- NIO是非阻塞IO。一旦是响应事件发生了，该线程就会将对应的响应事件交给对应的事件处理器进行处理。
- AIO是异步IO。主线程接收到请求后，可以分发给其他线程进行异步处理，主线程继续接收其他请求。

小林补充：

BIO（Blocking IO）、NIO（Non-Blocking IO）和AIO（Asynchronous IO）是Java中常用的IO模式。它们之间的主要区别在于IO的处理方式和效率。

BIO是同步阻塞IO，在进行IO操作时，必须等待IO操作完成后才能进行下一步操作，这时线程会被阻塞。BIO适用于连接数比较小且固定的架构，由于线程阻塞等待IO操作，所以并发处理能力不强。

NIO是同步非阻塞IO，可以支持多个连接同时进行读写操作，因此可以用较少的线程来处理大量的连接。NIO通过Selector来监听多个Channel的状态，当Channel中有数据可读或可写时，Selector会通知程序进行读写操作。NIO适用于连接数多且连接时间较短的场景。

AIO是异步非阻塞IO，与NIO不同的是，AIO不需要用户线程等待IO操作完成，而是由操作系统来完成IO操作，操作系统完成IO操作后会通知用户线程处理。AIO适用于连接数较多且连接时间较长的场景，如高性能网络服务器等。

你说一下NIO是如何实现同步非阻塞的？主线程是只有一个嘛？

读者答：

NIO底层是用Selector、Channel和ByteBuffer来实现的。主线程在循环使用select方法进行阻塞等待，当有acceptable、readable或者writable事件发生的时候，循环就会往下走，将对应的事件交给对应的事件处理器进行处理。

他可以多线程的，可以有多个accept()线程和多个worker线程。

小林补充：

在NIO中，使用了多路复用器Selector来实现同步非阻塞的IO操作。Selector是一个可以监控多个通道（Channel）是否有数据可读或可写的对象，当一个或多个Channel准备好读或写时，Selector会通知程序进行读写操作，而不是像BIO一样阻塞等待IO操作完成。

在NIO中，主线程通常只有一个，但是可以使用Selector来管理多个Channel，实现多个连接的非阻塞读写操作。当有多个Channel需要进行IO操作时，Selector会轮询这些Channel，检查它们的状态是否可读或可写，如果有可读或可写的Channel，就将其加入到一个已选择键集合中，等待程序处理。这样，一个线程就可以同时处理多个Channel，提高了系统的并发处理能力。

你用过哪些设计模式

单例模式，观察者模式，责任链模式

讲一下观察者模式

读者答：

观察者模式就是他有多个观察者，有一个观察管理者，观察者一开始会都注册到观察管理者的列表当中，当对应的位置发生了相应的事件呢，就会由观察管理者调用相应的观察者的方法执行相应的动作。

小林补充：

观察者模式（Observer Pattern）是一种设计模式，它定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象，当主题对象状态发生变化时，它的所有观察者都会收到通知并自动更新。

在观察者模式中，有两个核心角色：Subject（主题）和Observer（观察者）。主题是被观察的对象，它维护了一个观察者列表，可以动态添加或删除观察者。当主题状态发生变化时，它会通知所有观察者，并调用它们的更新方法。观察者是接收主题通知的对象，它定义了一个更新方法，使主题在状态发生变化时能够及时通知到它。

观察者模式可以实现松耦合的设计，主题对象和观察者对象之间没有直接的耦合关系，它们之间通过抽象的接口进行通信，可以方便地增加或删除观察者，而不需要修改主题对象的代码。观察者模式在很多场景中都有应用，比如GUI事件处理、消息队列、发布订阅系统等。

java 代码示例：

当然，以下是一个简单的Java代码实例，演示了观察者模式的基本实现：

```
import java.util.ArrayList;
import java.util.List;

// 主题 (Subject) 接口
interface Subject {
    void registerObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

// 观察者 (Observer) 接口
interface Observer {
    void update(String message);
}

// 具体主题 (ConcreteSubject) 实现
class ConcreteSubject implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private String message;

    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers() {
```

```

        for (Observer observer : observers) {
            observer.update(message);
        }
    }

    public void setMessage(String message) {
        this.message = message;
        notifyObservers();
    }
}

// 具体观察者 (ConcreteObserver) 实现
class ConcreteObserver implements Observer {
    private String name;

    public ConcreteObserver(String name) {
        this.name = name;
    }

    @Override
    public void update(String message) {
        System.out.println(name + " received message: " + message);
    }
}

// 测试类
public class ObserverPatternDemo {
    public static void main(String[] args) {
        ConcreteSubject subject = new ConcreteSubject();

        Observer observer1 = new ConcreteObserver("Observer1");
        Observer observer2 = new ConcreteObserver("Observer2");
        Observer observer3 = new ConcreteObserver("Observer3");

        subject.registerObserver(observer1);
        subject.registerObserver(observer2);
        subject.registerObserver(observer3);

        subject.setMessage("Hello, everyone!");

        subject.removeObserver(observer2);

        subject.setMessage("How are you doing?");
    }
}

```

运行上述代码，输出如下：

```
Observer1 received message: Hello, everyone!
Observer2 received message: Hello, everyone!
Observer3 received message: Hello, everyone!
Observer1 received message: How are you doing?
Observer3 received message: How are you doing?
```

可以看到，当主题对象状态发生变化时，它会通知所有观察者，并调用它们的更新方法。观察者可以根据接收到的消息进行相应的处理。

java内存结构

读者答：

JVM内存结构分为5大区域，程序计数器、虚拟机栈、本地方法栈、堆内存、方法区。

方法区：

用来存储加载的类信息、常量、静态变量、编译后的代码等数据。

堆内存：

堆内存可以细分为：老年代、新生代(Eden、From Survivor、To Survivor)。JVM启动时创建，用来存放对象的实例。堆内存是垃圾收集器管理的主要区域。

虚拟机栈：

线程私有的。虚拟机栈由多个栈帧组成。一个线程会执行一个或多个方法，一个方法对应一个栈帧。每一次方法调用都会有一个对应的栈帧被压入栈中，每一个方法调用结束后，都会有一个栈帧被弹出。栈帧内容包含：局部变量表、操作数栈、动态链接、方法返回地址等信息。

本地方法栈：

和虚拟机栈功能类似，虚拟机栈是为虚拟机执行JAVA方法而准备的，本地方法栈是为虚拟机使用Native本地方法而准备的。

程序计数器(Program Counter Register)：

线程私有的，程序计数器主要有两个作用：

1. 作为当前线程所执行的字节码的行号指示器，通过它实现代码的流程控制，如：顺序执行、分支、循环、异常处理。
2. 在多线程的情况下，程序计数器用于记录当前线程执行的位置，当线程被切换回来的时候可以通过程序计数器中的信息获取上次执行的位置，然后继续执行。

说一下数据库事务的四大特性

读者答：

事务有ACID四大特性。就是原子性、一致性、隔离性和持久性。原子性就是指事务中的操作要么全做要么全不做，不存在中间态。一致性是指事务执行前后数据库的完整性不被破坏，保持一致。隔离性是指多个事务并发执行时事务之间互不影响。持久性是指事务执行成功后，事务对于数据库的操作会永久的保存在磁盘上，永不丢失。

从一定程度上讲，AID是手段，C是目的，就是通过原子性、隔离性和持久性来保证一致性。

char和varchar的区别

读者答：

char是固定长度的字符串类型，varchar是可变长度的字符串类型。拿char(128)和varchar(128)举例来说。char(128)是无论字符串大小，都会在磁盘上分配128个字符的内存空间。而varchar(128)会根据字符本身的长短来分配内存空间。

小林补充：

在MySQL中，`CHAR` 和 `VARCHAR` 都是用于存储字符类型数据的数据类型，它们的区别在于存储方式和使用场景。

`CHAR` 类型用于存储固定长度的字符串，其长度在定义表时就已经固定，且最大长度为255个字符。当存储的字符串长度小于定义的长度时，MySQL会在其后面补充空格使其长度达到定义的长度。由于存储的长度是固定的，因此 `CHAR` 类型的读取速度比 `VARCHAR` 类型更快。

`VARCHAR` 类型则用于存储可变长度的字符串，其长度可以在存储数据时动态地改变，但最大长度也为255个字符。当存储的字符串长度小于定义的长度时，MySQL不会在其后面补充空格。由于存储的长度是可变的，因此 `VARCHAR` 类型的存储空间相对更小，但读取速度比 `CHAR` 类型稍微慢一些。

那与varchar相比，char字段是不是一无是处呢？

大部分情况，是的，最好使用varchar。不过考虑一个极端的场景：某个字段的最大长度是100字节，但是会频繁修改。如果使用 `char(100)`，则插入记录后就分配了100个字节，后续修改不会造成页分裂、页空隙等问题，而 `varchar(100)` 由于没有提前分配存储空间，后续修改时可能出现页分裂，进而导致性能下降。

说一下外键约束

读者答：

举例来说，某一个字段是表b的主键，但是它也是表a中的字段，表a中该字段的使用范围取决于表b。外键约束主要是用来维护两个表的一致性。

小林补充：

外键约束的作用是维护表与表之间的关系，确保数据的完整性和一致性。让我们举一个简单的例子：

假设你有两个表，一个是学生表，另一个是课程表，这两个表之间有一个关系，即一个学生可以选修多门课程，而一门课程也可以被多个学生选修。在这种情况下，我们可以在学生表中定义一个指向课程表的外键，如下所示：

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course_id INT,  
    FOREIGN KEY (course_id) REFERENCES courses(id)  
);
```


这里，`students` 表中的 `course_id` 字段是一个外键，它指向 `courses` 表中的 `id` 字段。这个外键约束确保了每个学生所选的课程在 `courses` 表中都存在，从而维护了数据的完整性和一致性。

如果没有定义外键约束，那么就有可能出现学生选了不存在的课程或者删除了一个课程而忘记从学生表中删除选修该课程的学生情况，这会破坏数据的完整性和一致性。因此，使用外键约束可以帮助我们避免这些问题。

说一下binlog

读者答：

binlog是二进制日志文件。他主要用来做主从同步。他有statement格式和row格式。statement记录了执行的SQL语句，Row 格式保存哪条记录被修改。binlog事务提交的时候才写入的。也可以用来做归档。

小林补充：

binlog日志是MySQL数据库的一种日志记录机制，用于记录数据库的修改操作（如插入、更新、删除等），以便在需要进行数据恢复、数据复制和数据同步等操作。

binlog日志的实现以下功能：

- 数据恢复：binlog日志可以用于回滚到之前的某个时间点，从而恢复数据。
- 数据复制：binlog日志可以用于在主从数据库之间复制数据，从而实现数据的高可用和负载均衡等功能。

MySQL的binlog日志有三种格式，分别是Statement格式、Row格式和Mixed格式。它们之间的区别如下：

- STATEMENT：每一条修改数据的 SQL 都会被记录到 binlog 中（相当于记录了逻辑操作，所以针对这种格式，binlog 可以称为逻辑日志），主从复制中 slave 端再根据 SQL 语句重现。但 STATEMENT 有动态函数的问题，比如你用了 `uuid` 或者 `now` 这些函数，你在主库上执行的结果并不是你在从库执行的结果，这种随时在变的函数会导致复制的数据不一致；
- ROW：记录行数据最终被修改成什么样了（这种格式的日志，就不能称为逻辑日志了），不会出现 STATEMENT 下动态函数的问题。但 ROW 的缺点是每行数据的变化结果都会被记录，比如执行批量 `update` 语句，更新多少行数据就会产生多少条记录，使 binlog 文件过大，而在 STATEMENT 格式下只会记录一个 `update` 语句而已；
- MIXED：包含了 STATEMENT 和 ROW 模式，它会根据不同的情况自动使用 ROW 模式和 STATEMENT 模式；

说一下分库分表

读者答：

我可能知道的就是想我简历上调研过的这个mycat组件，他是根据业务字段的hash值来确定分片的，比如`user_id`不同的用户信息就会存储到不同分片当中，他是多个分片同时提供服务。

小林补充：

当数据量过大造成事务执行缓慢时，就要考虑分表，因为减少每次查询数据总量是解决数据查询缓慢的主要原因。你可能会问：“查询可以通过主从分离或缓存来解决，为什么还要分表？”但这里的查询是指事务中的查询和更新操作。

为了应对高并发，一个数据库实例撑不住，即单库的性能无法满足高并发的要求，就把并发请求分散到多个实例中去，这种就是分库。

总的来说，分库分表使用的场景不一样：分表是因为数据量比较大，导致事务执行缓慢；分库是因为单库的性能无法满足要求。

遇到过数据库死锁吗

读者答：

事务A通过数据修改操作占用着资源A,事务B通过数据修改操作占用着资源B,而他们又同时请求对方的资源，互不退让就造成了死锁。如果没有终止一个事务或者回滚过一段时间或超时。

小林补充：

假设有两事务，一个事务要插入订单 1007，另外一个事务要插入订单 1008，因为需要对订单做幂等性校验，所以两个事务先要查询该订单是否存在，不存在才插入记录，过程如下：

事务 A	事务 B
begin;	begin;
//检查 1007 订单是否存在 select id from t_order where order_no = 1007 for update;	
	//检查 1008 订单是否存在 select id from t_order where order_no = 1008 for update;
//如果没有，则插入订单记录 insert into t_order (order_no, create_date) values (1007, now()); 阻塞等待....	
	//如果没有，则插入订单记录 insert into t_order (order_no, create_date) values (1008, now()); 阻塞等待....

img

可以看到，两个事务都陷入了等待状态（前提没有打开死锁检测），也就是发生了死锁，因为都在相互等待对方释放锁。

死锁的四个必要条件：**互斥、占有且等待、不可强占用、循环等待**。只要系统发生死锁，这些条件必然成立，但是只要破坏任意一个条件就死锁就不会成立。

TCP和UDP的区别

读者答：

- 1.TCP是面向连接的协议，建立和释放连接需要进行三次握手和四次挥手。UDP是面向无连接的协议，无需进行三次握手和四次挥手。说明udp比TCP实时性更强。
- 2.TCP 是流式传输，没有边界，但保证顺序和可靠。UDP 是一个包一个包的发送，是有边界的，但可能会丢包和乱序。
- 3.TCP连接的可靠性强，UDP的可靠性不强。
- 4.TCP只能一对一，UDP支持一对多和多对多。
- 5.TCP的头部开销比UDP大。TCP 首部长度较长，会有一定的开销，首部在没有使用「选项」字段时是 20 个字节，如果使用了「选项」字段则会变长的。UDP 首部只有 8 个字节，并且是固定不变的，开销较小。

TCP是如何保证可靠的？

读者答：

- tcp的序列号可以避免乱序的问题，保证收到的tcp报文都是有序的。
- 在 TCP 中，当发送端的数据到达接收主机时，接收端主机返回一个确认应答消息，表示已收到消息。
- TCP 针对数据包丢失的情况，会用重传机制解决。
- 用快重传解决个别报文段的丢失问题。
- 使用滑动窗口实现流量控制。使用接收方确认报文中的窗口字段来控制发送方发送窗口大小，进而控制发送方的发送速率，使得接收方来得及接收。
- 使用基于窗口的拥塞控制，来尽量避免避免网络拥塞。

流量控制是使用什么数据结构来实现的？

读者答：

流量控制是使用滑动窗口来实现的。接收方确认报文中的窗口字段可以用来控制发送方窗口的大小。如果窗口的值为0，则发送方停止发送数据，但是发送方会定期的向接收方发送窗口探测报文以得到窗口的大小。

小林补充：

TCP传输协议中，流量控制是使用滑动窗口（Sliding Window）来实现的。滑动窗口是一种基于数据流的、动态调整的、可变大小的窗口，它通过协商双方的接收窗口和发送窗口大小，控制数据的传输速率。

在TCP协议中，每个数据包都有一个序号，接收方通过序号来确认是否收到了正确的数据包。发送方将数据分成若干个数据段，每个数据段的大小不超过发送窗口的大小，然后将这些数据段发送给接收方。接收方会确认已经收到的数据，同时告诉发送方自己的接收窗口大小。发送方根据接收方的窗口大小，动态调整自己的发送窗口大小，从而控制数据的传输速率。

滑动窗口的大小是可以动态调整的，它可以根据网络状况和双方的能力来自适应地调整，从而实现流量控制的功能。如果接收方的接收窗口变小，发送方会相应地减小自己的发送窗口，以避免过多的数据堆积在网络中导致拥塞。如果接收方的接收窗口变大，发送方会相应地增加自己的发送窗口，以提高数据传输速率。

分块传输

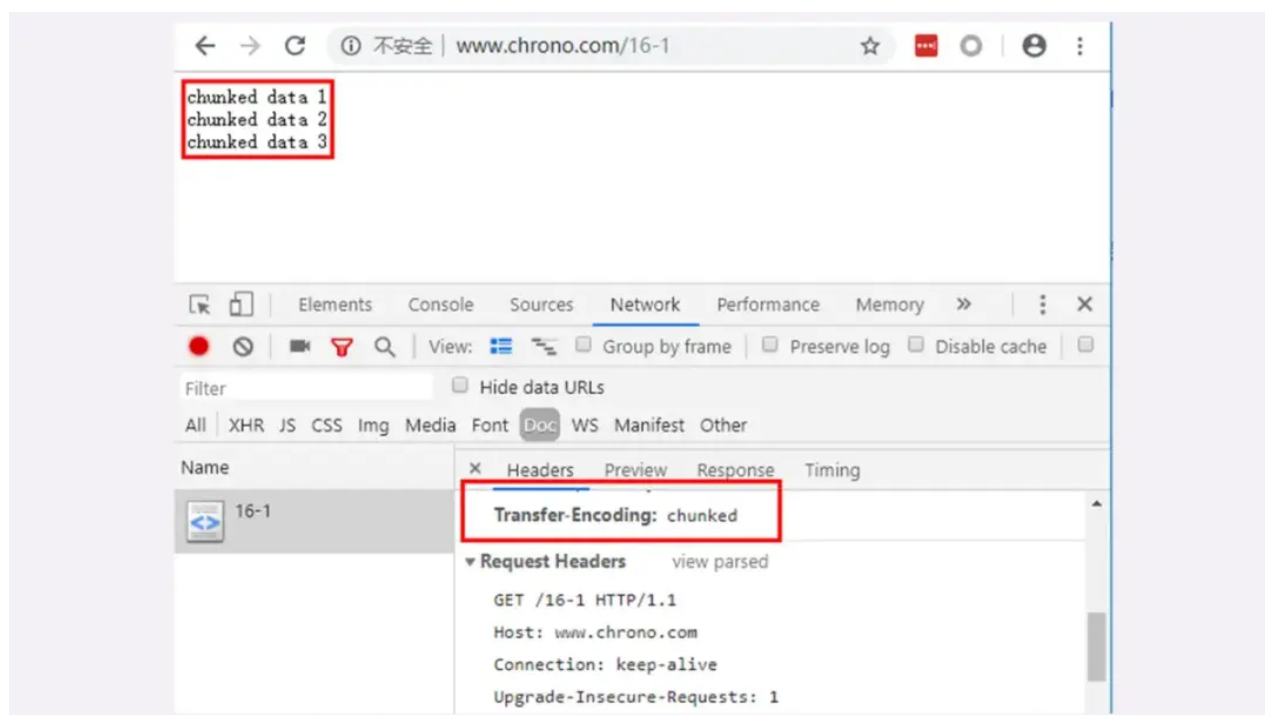
读者答：

分块传输这一块有个nagle算法。他的目的是尽量发送大数据块，以减少发送报文的数量，提高传输效率。nagle算法规定在上一个未被确认的分组的确认到达之前，不能发送下一个分组。

分块传输其实更像是http协议里的chunk传输。如果是特指tcp的话，如果应用层的数据超过mss的大小，数据会在tcp层进行分块。

小林补充：

分块传输感觉是说http协议里的chunk传输。它允许将数据分成多个块（Chunk）进行传输，每个块都包含一段数据和该块数据的长度。在传输数据时，先发送一个块的长度，然后发送该块的数据，接着发送下一个块的长度和数据，以此类推，直到所有的数据都传输完毕。



如果是特指tcp的话，如果应用层的数据超过mss的大小，数据会在tcp层进行分块。

刚才你说nagle算法是为了发送大数据块，数据块是越大越好吗？（我没太听懂这个问题，他又换了个说法，其实我还是没懂，但是他提到了MTU，我就借坡下驴了）

读者答：

您刚才提到了MTU，有可能因为数据块比较大，可能会出现拆包的问题，将一个大数据块分为几个MTU单元进行传输，因为TCP是按照序列号顺序读取的，所以可能会出现阻塞问题。

小林补充：

看业务场景，negle算法不适合像ssh这种传输小报文的场景，会增加延迟。

项目问题

缩减很多，只放出了共性的问题。

- 性能调优是怎么做的？
- 你觉得你的这个项目性能瓶颈在哪里？
- 项目你自己做的吗？开源了吗？

无算法题

面试总结

感觉：

- 腾讯面试比之前的要难，增加自信了，开始会胡说八道了。即使知道自己说的不是对的，例如分块传输那个问题。

不足之处：

- 再积累积累吧

快手面试，一直追着问我。。。。

大家好，我是小林。

这段时间分享了很多校招的面经，有很多读者说想看社招的，其实社招面试是基于你的工作项目来展开问的，比如你项目用了 xxx 技术，那么面试就会追问你项目是怎么用 xxx 技术的，遇到什么难点和挑战，然后再考察一下这个 xxx 技术的原理。

今天就分享一位快手社招面经，岗位是后端开发，问题都是基于项目涉及的技术栈去展开聊的，同时最后也会有算法题。

项目

- 自我介绍+项目介绍
- 就你负责比较多的项目详细说说，项目背景，data模型，流程，难点和挑战
- 讲讲项目后端用到的技术栈，比如mq，rpc，缓存啥的
- 消息队列用过吗，业务场景？
- 怎么保证消息的有序性？

Redis

Redis有哪些数据类型

回答：String，list，map，set，Zset，stream，hyperloglog。。。。

（打断）追问：map怎么扩容，扩容时会影响缓存吗

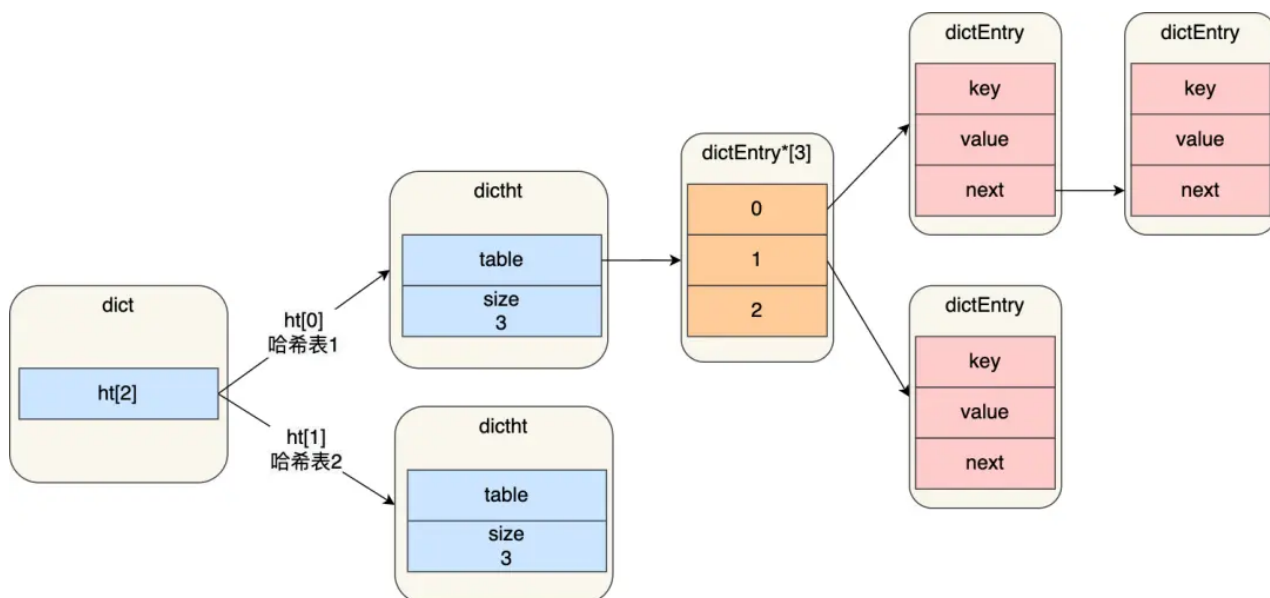
回答：底层有两个dict，一个dict负责请求，到达负载比例进行扩容，渐进式扩容，一部分一部分转移到新的dict

追问：扩容时访问key怎么处理？

回答：先从旧dict获取key，查不到的话，然后从新的dict获取key

小林补充：

进行 rehash 的时候，需要用上 2 个哈希表了。



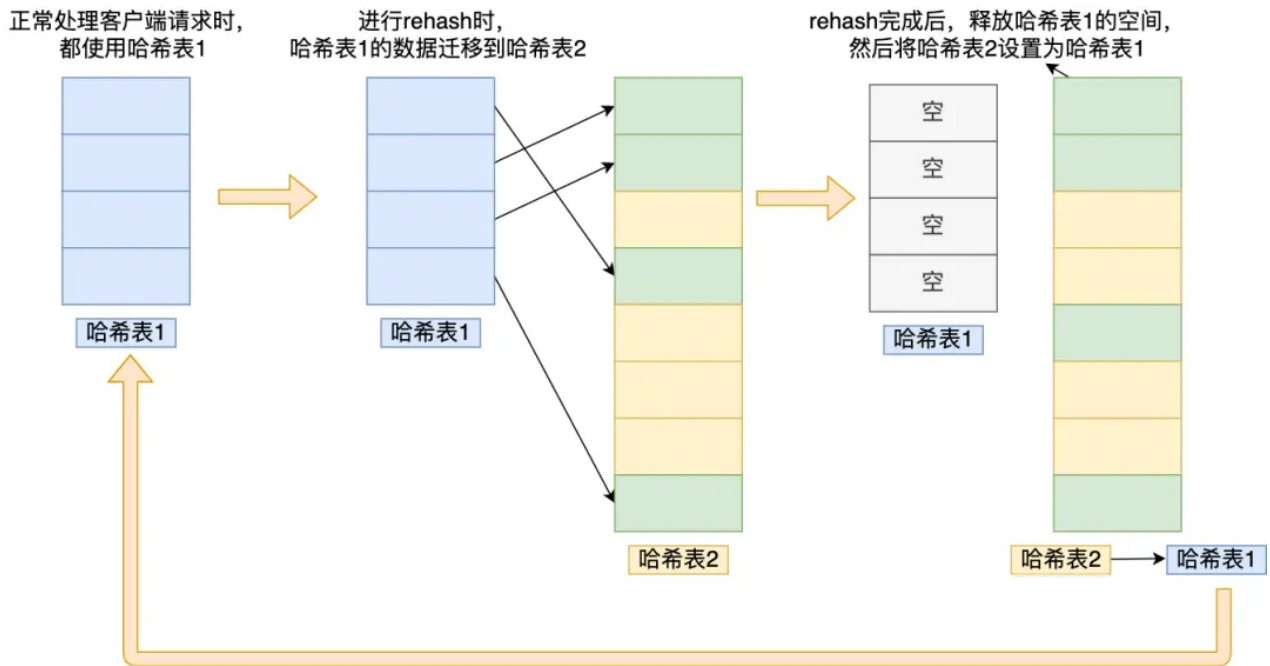
img

在正常服务请求阶段，插入的数据，都会写入到「哈希表 1」，此时的「哈希表 2」并没有被分配空间。

随着数据逐步增多，触发了 rehash 操作，这个过程分为三步：

- 给「哈希表 2」分配空间，一般会比「哈希表 1」大 2 倍；
- 将「哈希表 1」的数据迁移到「哈希表 2」中；
- 迁移完成后，「哈希表 1」的空间会被释放，并把「哈希表 2」设置为「哈希表 1」，然后在「哈希表 2」新创建一个空白的哈希表，为下次 rehash 做准备。

为了方便你理解，我把 rehash 这三个过程画在了下面这张图：



img

这个过程看起来简单，但是其实第二步很有问题，如果「哈希表 1」的数据量非常大，那么在迁移至「哈希表 2」的时候，因为会涉及大量的数据拷贝，此时可能会对 Redis 造成阻塞，无法服务其他请求。

为了避免 rehash 在数据迁移过程中，因拷贝数据的耗时，影响 Redis 性能的情况，所以 Redis 采用了**渐进式 rehash**，也就是将数据的迁移的工作不再是一次性迁移完成，而是分多次迁移。

渐进式 rehash 步骤如下：

- 给「哈希表 2」分配空间；
- 在 rehash 进行期间，每次哈希表元素进行新增、删除、查找或者更新操作时，Redis 除了会执行对应的操作之外，还会顺序将「哈希表 1」中索引位置上的所有 key-value 迁移到「哈希表 2」上；
- 随着处理客户端发起的哈希表操作请求数量越多，最终在某个时间点会把「哈希表 1」的所有 key-value 迁移到「哈希表 2」，从而完成 rehash 操作。

这样就巧妙地把一次性大量数据迁移工作的开销，分摊到了多次处理请求的过程中，避免了一次性 rehash 的耗时操作。

在进行渐进式 rehash 的过程中，会有两个哈希表，所以在渐进式 rehash 进行期间，哈希表元素的删除、查找、更新等操作都会在这两个哈希表进行。比如，查找一个 key 的值的话，先会在「哈希表 1」里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

另外，在渐进式 rehash 进行期间，新增一个 key-value 时，会被保存到「哈希表 2」里面，而「哈希表 1」则不再进行任何添加操作，这样保证了「哈希表 1」的 key-value 数量只会减少，随着 rehash 操作的完成，最终「哈希表 1」就会变成空表。

跳表结构了解吗

回答：第一层是双向链表，会多层来作为链表的索引。

追问：和二叉树有什么区别，从时间复杂度和空间复杂度分析

回答：二叉查找树的时间复杂度是 $O(\log n)$ ，空间复杂度是 $O(n)$ ；跳表的时间复杂度是 $O(\log_{\{k\}} n)$ ， k 为跳表索引步长，空间复杂度是 $O(n)$

MySQL

MySQL事务用过吗，应用场景是什么

自己学习的demo里用过，场景：银行转账

追问：假如是跨行转账怎么解决事务

回答：我想一想。。。

小林补充：用分布式事务

（打断）追问：跨行先不说，先说不跨行，先说说单库事务，事务的4个基本特性

回答：原子性，一致性，隔离性，持久性

追问：一致性指的是什么

回答：一致性指的是事务提交前后数据库状态一致，是原子性，隔离性和持久性的整合

追问：隔离级别有哪几种

回答：读未提交，读已提交，可重复读，序列化

追问：可重复读是是什么意思，怎么实现的

同一个事务中多次读取结果一致。通过Read View + MVCC实现，在事务开始时创建一个Read View，之后都用这个Read View。

追问：可重复读具体实现细节

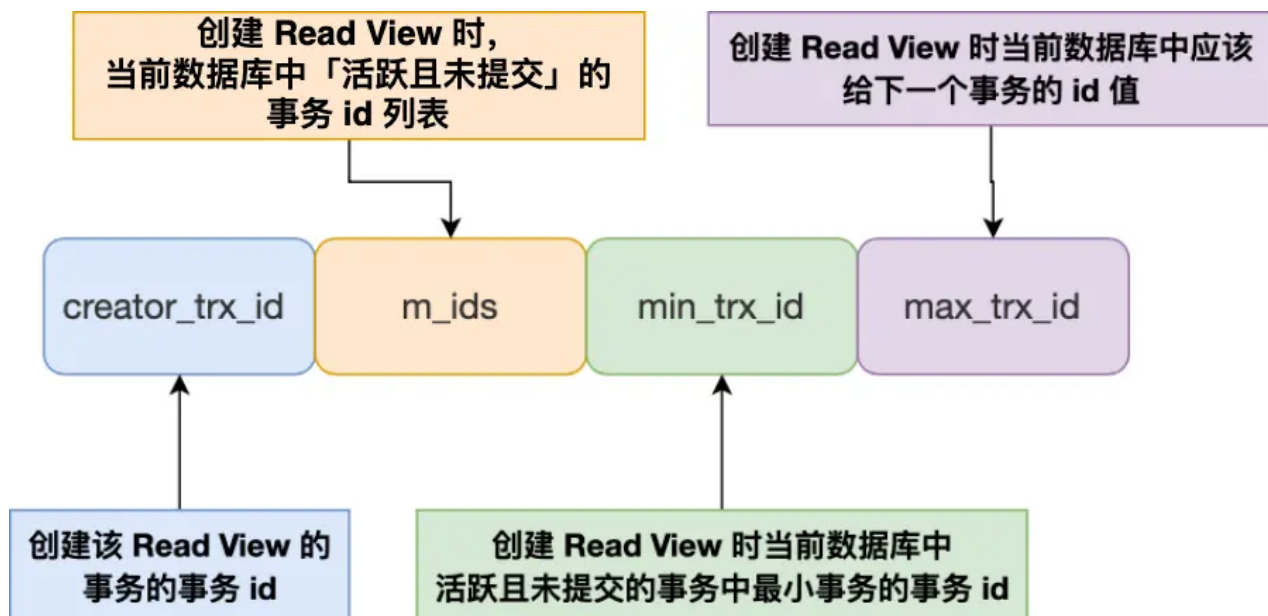
回答：Read View的结构包含（1）当前事务id，（2）active事务id列表，（3）最小active事务id记为min_txn_id，（4）next事务id记为max_txn_id。如果record对应的事务id，小于最小active事务id，直接读；否则读取最小active事务id之前版本的数据（这块的逻辑答得有问题，用语言表达时有点混乱，可以私下多练习练习表达）

小林补充：

我们需要了解两个知识：

- Read View 中四个字段作用；
- 聚簇索引记录中两个跟事务有关的隐藏列；

那 Read View 到底是个什么东西？



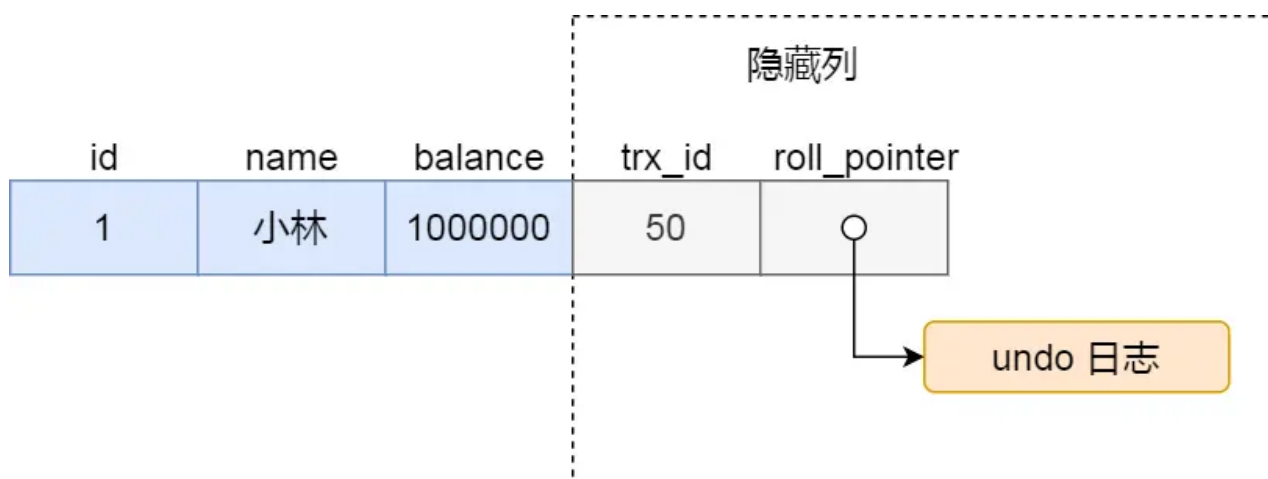
img

Read View 有四个重要的字段：

- m_ids：指的是在创建 Read View 时，当前数据库中「活跃事务」的事务 id 列表，注意是一个列表，“活跃事务”指的就是，启动了但还没提交的事务。
- min_trx_id：指的是在创建 Read View 时，当前数据库中「活跃事务」中事务 id 最小的事务，也就是 m_ids 的最小值。
- max_trx_id：这个并不是 m_ids 的最大值，而是创建 Read View 时当前数据库中应该给下一个事务的 id 值，也就是全局事务中最大的事务 id 值 + 1；
- creator_trx_id：指的是创建该 Read View 的事务的事务 id。

知道了 Read View 的字段，我们还需要了解聚簇索引记录中的两个隐藏列。

假设在账户余额表插入一条小林余额为 100 万的记录，然后我把这两个隐藏列也画出来，该记录的整个示意图如下：



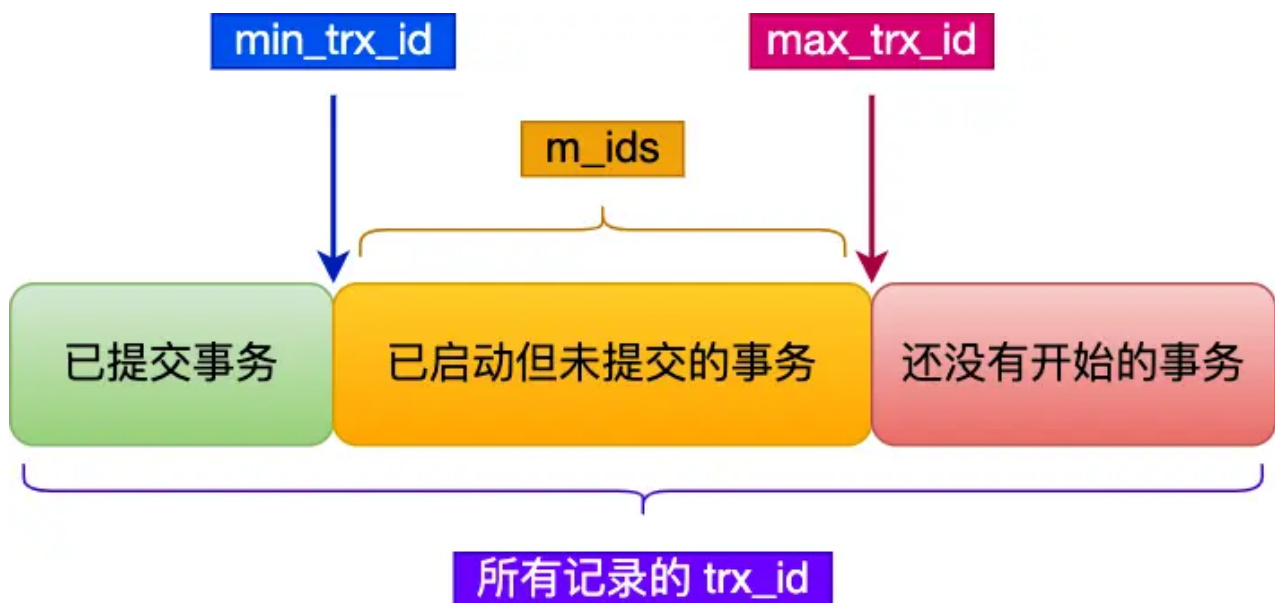
图片

对于使用 InnoDB 存储引擎的数据库表，它的聚簇索引记录中都包含下面两个隐藏列：

- trx_id，当一个事务对某条聚簇索引记录进行改动时，就会把该事务的事务 id 记录在 trx_id 隐藏列里；
- roll_pointer，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 undo 日志中，然

后这个隐藏列是个指针，指向每一个旧版本记录，于是就可以通过它找到修改前的记录。

在创建 Read View 后，我们可以将记录中的 `trx_id` 划分这三种情况：



img

一个事务去访问记录的时候，除了自己的更新记录总是可见之外，还有这几种情况：

- 如果记录的 `trx_id` 值小于 Read View 中的 `min_trx_id` 值，表示这个版本的记录是在创建 Read View 前已经提交的事务生成的，所以该版本的记录对当前事务可见。
- 如果记录的 `trx_id` 值大于等于 Read View 中的 `max_trx_id` 值，表示这个版本的记录是在创建 Read View 后才启动的事务生成的，所以该版本的记录对当前事务不可见。
- 如果记录的 `trx_id` 值在 Read View 的

```
min_trx_id
```

和

```
max_trx_id
```

之间，需要判断 `trx_id` 是否在 `m_ids` 列表中：

- 如果记录的 `trx_id` 在 `m_ids` 列表中，表示生成该版本记录的活跃事务依然活跃着（还没提交事务），所以该版本的记录对当前事务不可见。
- 如果记录的 `trx_id` 不在 `m_ids` 列表中，表示生成该版本记录的活跃事务已经被提交，所以该版本的记录对当前事务可见。

MySQL binlog, redolog和undolog的区别

回答：从这三种log的功能进行分析，undolog用来做事务回滚；redolog用来处理mysql宕机（或进程挂掉）时的数据恢复；binlog记录所有的数据修改操作，可以用来做全量的数据恢复，主从复制（后一点当时没答出来）。

小林补充：

1. binlog（二进制日志）：记录所有对MySQL数据库的修改操作，包括插入、更新和删除等。binlog主要用于数据恢复到指定时间点或者指定事务。可以使用mysqlbinlog命令将binlog文件解析成SQL语句，从而恢复MySQL数据库的状态。
2. redolog（重做日志）：记录所有对MySQL数据库的修改操作，但是只记录了物理操作，比如页的修改。redolog主要用于MySQL的崩溃恢复，即在MySQL崩溃后，通过重做日志，将数据库恢复到最近一次提交的状态。可以使用 Forcing InnoDB Recovery 来进行崩溃恢复。
3. undolog（回滚日志）：用于记录事务的回滚操作，即在事务执行过程中，如果发生了回滚，会将回滚操作记录到undolog中。undolog主要用于 MySQL 的回滚操作，比如使用ROLLBACK语句回滚事务。undolog是InnoDB存储引擎的特有日志，不同于其他存储引擎。

追问：binlog和redolog做数据恢复的区别

回答：redolog有大小限制，数据可能被覆盖，用来处理紧急数据库故障；binlog是全量操作日志，可以进行做全量的数据恢复。

小林补充：

binlog和redolog都是用于MySQL数据库的日志。它们都可以用于数据恢复，但是它们的使用场景和恢复方法有所不同。

binlog是MySQL的二进制日志，它记录了所有对MySQL数据库的修改操作，包括插入、更新和删除等。binlog可以用于恢复MySQL数据库到指定的时间点或者指定的事务。具体来说，可以使用mysqlbinlog命令将binlog文件解析成SQL语句，然后再执行这些SQL语句，从而恢复MySQL数据库的状态。

redolog是MySQL的重做日志，它记录了所有对MySQL数据库的修改操作，但是只记录了物理操作，比如页的修改。redolog可以用于恢复MySQL数据库的崩溃恢复，即在MySQL崩溃后，通过重做日志，将数据库恢复到最近一次提交的状态。具体来说，可以使用innodb_recovery命令来进行崩溃恢复，该命令会根据重做日志来恢复数据库。

因此，binlog和redolog都可以用于数据恢复，但是它们的使用场景和恢复方法有所不同。binlog主要用于数据恢复到指定时间点或者指定事务，而redolog主要用于MySQL的崩溃恢复。

算法

- 合并两个有序数组

面试总结

感觉

基础知识答得还行，编程拉了

不足之处

有些地方的表达逻辑不够清晰，代码得多写。

我扛住字节面试了，太干了！

大家好，我是小林。

春招进展快 2 个月，大家都找到暑期实习了吗？今年相比往年我感觉比较卷，很少见到offer收割机的选手。

不管环境如何，持续学习这个是不能放弃的，心态也要稳一稳，坦然面对失败，失败才是常态，成功可能才是偶然的。

好了，鸡汤喝完了，那我来一点干货吧，又干又涩，毫无感情，但是又很有用。

今天分享一位读者字节后端面经，读者技术栈是C++和GO。

语言

C ++, Python 哪一个更快?

读者答：这个我不知道从哪方面说，就是 C ++ 的话，它其实能够提供开发者非常多的权限，就是说它能涉及到一些操作系统级别的一些操作，速度应该挺快。然后 Python 实现功能还是蛮快的。

小林补充：

一般而言，C++更快一些，因为它是一种编译型语言，可以直接编译成机器码，在执行时不需要解释器的介入，因此执行效率较高。

Python是一种解释型语言，需要在运行时通过解释器将代码转换为机器码来执行，因此相对于C++而言，执行效率较低。但是Python具有很多优秀的库和框架，这些库和框架可以帮助开发人员快速开发出高效的应用程序，从而提高开发效率。

编译型语言和解释型语言有没有了解过

读者答：可能 C ++ 它提供的更偏机器语言一样，就是只需要进行相关的编译查找就可以。但是 Python 的话它可能是更像脚本语言，所以说在进行执行的时候还需要再进行嗯，一方面的句子语法处理，所以执行速度上会慢一些。

堆内存和栈内存有什么区别

读者答：go里面的堆分配的主要是一些比较大的对象，栈里面的话可能就是分配一些临时的对象，或者是比较小的数据变量。然后如果分配的就是比如说如果申请了一个channel，或者是一个非常大的对象的话，那么它就会从默认的从栈上的空间分配到堆上，因为堆上的这空堆上的空间进行分配之后，它能保留的时间会更长一些，这大概就是栈和堆之间的一个区别。

小林补充：

1. 管理方式不同：栈内存由系统自动分配和释放，而堆内存则需要程序员手动分配和释放。
2. 分配方式不同：栈内存是一种连续的内存空间，系统会自动为每个线程分配一定大小的栈空间，函数的参数和局部变量都会在栈上分配内存。而堆内存是不连续的内存空间，程序员需要通过动态分配内存来获得堆内存空间。
3. 大小限制不同：栈内存的大小是固定的，并且比堆内存小得多。程序运行时，每个线程的栈空间通常只有几MB到几十MB，而堆内存的大小则取决于系统的剩余内存大小和程序员的动态分配。
4. 访问速度不同：栈内存的访问速度比堆内存快得多，因为栈内存是连续的，访问局部变量和函数参数时可以直接读取栈指针的偏移量。而堆内存是不连续的，访问速度较慢。

局部变量是放在堆还是放在栈里面？

读者答：局部变量的话，因为它只在一个函数里面进行相关的生命周期的存在，所以应该是在栈上面的。

链表跟数组有哪一些区别？

读者答：链表的长度是不固定的，对它进行相关的插入或者是删除操作是非常快的。查找需要从头到尾遍历值，效率低。

数组来说的话，数组的话是预先分配了一段固定长度的连续的内存空间，通过数组的下标索引来查找和赋值。但是如果要进行插入删除操作的话，那可能会需要就是将数据中你要插入那个位置之后的所有的数组来进行挪位，才能进行相关的插入操作，所以说它这个插入和删除的操作就会相比链表会麻烦。

小林补充：

可以提一下数组因为内存地址是连续，可以增加cpu缓存命中率，而链表的内存地址并不是连续的，cpu缓存命中率会很低。

数组怎么动态扩容？

读者答：go的话它其实提供了一个 slice 这样子的结构，也就是说它底层维护了一个指向数组的指针，然后还维护了一个数组的长度和一个它的空间预存的一个 CAP 的容量值。然后如果将这个当前的数组 space 的大小它是小于 1024 的话，那基本上都是 2 倍的扩容，然后如果超过 1.2 次的大小的话，基本上是 1.25 倍的扩容。

协程与我们普通的线程有什么区别？

读者答：协程可以理解为用户级别的线程，所以说在大小方面和调度方面都是比进程要更加的方便和简便的。

小林补充：

1. 调度方式不同：线程是由操作系统调度的，而协程则是由程序员控制的。当一个线程被调度时，它会被操作系统挂起，等待下一次调度。而协程则是由程序员在代码中主动调用的，可以在不同的任务之间切换，而不需要等待操作系统的调度。
2. 系统资源占用不同：线程是操作系统管理的实体，它占用系统资源比较大，包括内存、线程栈、CPU 时间片等。而协程则是在用户空间中实现的，不需要操作系统的支持，因此占用的资源比较少。
3. 切换成本不同：线程的切换需要保存和恢复线程上下文，需要耗费一定的时间和资源。而协程的切换只需要保存和恢复栈帧等少量数据，因此切换成本比线程低。
4. 编程模型不同：线程是面向操作系统的，而协程是面向任务的。线程需要使用操作系统提供的 API 进行线程间通信和同步，而协程则可以使用语言级别的协程库实现协作式多任务。

协程的的通讯有哪些方式？

读者答：不清楚，go 使用时，可能会配合 channel 来进行使用，起一个死循环，监听不同的信号量进行处理。还是说是指那个 GMP 的模型？

小林补充：

1. 共享内存：协程通过共享内存来交换数据，这种方式简单直接，但需要考虑同步和互斥问题，否则会出现数据竞争等问题。
2. 消息传递：协程通过消息队列等方式来传递数据，这种方式可以避免数据竞争等问题，但需要考虑消息的发送和接收顺序等问题。
3. 信号量：协程通过信号量等方式来实现同步和互斥，这种方式需要考虑好信号量的数量和使用顺序，否则会出现死锁等问题。

常规的多线程开发需要注意哪一些问题？

读者答：场景考虑，你这个线程是不可以就是无限制的创建的吗？而且创建线程是需要开销的，所以一般是会使用线程池这样的方式，先预先创建好一些已经分配好的线程资源，然后有需要用的话就先进行相关的使用，然后这个线程池也负责了一些GC 的处理。

线程池里面大概要开多少的线程数量，这个线程数量的话就会跟你的任务相关。CPU 密集型根据CPU 的核数或者和任务执行的有关的时间来进行这个线程数量的考虑。

读写共享变量会遇到什么问题？

读者答：并发访问

什么是死锁？

读者答：两个或多个进程之间他们都在等待一些资源，然后没有办法完全释放现在已经占有的资源，但是他们需要的资源的话又被其他的进程所占有着，同时这一整个过程中你又没有办法去抢占，造成了一个循环等待的情况（请求保持，互斥，不可剥夺，循环等待）

如何避免出现死锁，怎么排查？

读者答：加锁或者channel，打断点，加日志信息。

小林补充：

避免死锁：

- 按照固定的顺序获取锁：按照固定的顺序获取锁可以避免死锁的发生。例如，如果A线程先获取了锁1，再获取锁2，那么B线程就应该先获取锁2，再获取锁1。
- 设置超时时间：在获取锁的过程中，可以设置超时时间，如果超过一定时间还没有获取到锁，就放弃获取锁，避免因等待锁而导致的死锁。

排查方式：

- 使用工具：可以使用一些工具来帮助检测和定位死锁问题，例如jstack和jconsole等。
- 分析日志：可以分析系统日志和线程日志，查看是否有线程在等待某个锁，从而找出可能导致死锁的原因。
- 代码检查：可以检查代码中是否存在多个线程竞争同一个锁的情况，是否存在锁的嵌套等问题，从而找出可能导致死锁的原因。

密码学和计网

用到过哪些加密算法？

读者答：

- Sha256 或者是 MD5 这样子的加密算法。不能还原成原来的数据，用来比较数据是否一致。
- 对称和非对称的加密算法RSA

小林补充：

1. 对称加密算法：如DES、3DES、AES等，使用相同的密钥加密和解密数据，加解密速度快，但密钥管理困难。
2. 非对称加密算法：如RSA、ECC等，使用公钥加密数据，私钥解密数据，安全性高，但加解密速度较慢。
3. 哈希算法：如MD5、SHA-1、SHA-256等，将任意长度的数据映射成固定长度的哈希值，不可逆、不可篡改，主要用于数据完整性校验。
4. 消息认证码（MAC）：如HMAC、CMAC等，将消息和密钥混合处理生成固定长度的认证码，用于防止数据被篡改。
5. 数字签名算法：如RSA、DSA等，将消息和私钥混合处理生成数字签名，用于验证消息的来源和完整性。

对称和非对称的加密算法的区别，具体讲讲非对称

读者答：对称的话它是有一个公钥和一个私钥的，然后私钥的话是只有自己持有的，这样的话那他别人其实不会获取到你的CL，然后你通过一个嗯可信的第三方来进行相关的加密。这样的话别人只需要通过你的公钥来进行一下获取你的公钥来进行你传输内容的加解密就可以，然后来保证这样子的一个数据的安全性。

HTTP和HTTPS有什么区别？

读者答：因为HTTP是明文传输的，然后 HTTPS 的话就是在这样的基础上增加SSL的一些加密的保护，来保证它传输消息的安全性。

小林补充：

- HTTP 是超文本传输协议，信息是明文传输，存在安全风险的问题。HTTPS 则解决 HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输。
- HTTP 连接建立相对简单，TCP 三次握手之后便可进行 HTTP 的报文传输。而 HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输。
- 两者的默认端口不一样，HTTP 默认端口号是 80，HTTPS 默认端口号是 443。
- HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的。

怎么去使用这些加密算法？

读者答：混合使用对称和非对称加密算法，会话密钥是对称的，获取的过程是非对称的。

小林补充：

通过混合加密的方式可以保证信息的机密性，解决了窃听的风险。



混合加密

HTTPS 采用的是对称加密和非对称加密结合的「混合加密」方式：

- 在通信建立前采用**非对称加密**的方式交换「会话密钥」，后续就不再使用非对称加密。
- 在通信过程中全部使用**对称加密**的「会话密钥」的方式加密明文数据。

采用「混合加密」的方式的原因：

- **对称加密**只使用一个密钥，运算速度快，密钥必须保密，无法做到安全的密钥交换。
- **非对称加密**使用两个密钥：公钥和私钥，公钥可以任意分发而私钥保密，解决了密钥交换问题但速度慢。

TCP 跟 UDP有什么区别？

读者答：TCP 的话它提供了一系列的可靠传输机制来保证它这个传输是可靠的，相较而言的话，那它的传输速度就是慢的，而UDP 的话，它没有做这个可靠的控制，它只是尽力而为，所以说它的传输速度是快的，而且占用的资源也会更小一些。具体使用的话要看不同的那个业务场景来进行相关的使用。

UDP 怎么改造变为可靠传输？

读者答：应用层加序列号和ACK。然后本地会缓存一些自己已经发过的消息，然后同时也要求对方在收到消息之后返回 ACK 来确认这个消息已经收到，如果没有收到 ACK 的话，可能会设置一些定时重传的这样的一些方式，来确保这个消息能够成功的发送到对方。

小林补充：

还要在应用层实现滑动窗口，实现流量控制，根据接收方的接收能力才发送数据，还有拥塞控制，当网络中有大量数据包同时传输时，会导致网络拥塞，从而影响数据传输的质量和效率。TCP拥塞控制通过动态调整数据传输速率，避免了网络拥塞的发生，从而保证了数据传输的可靠性和高效性。

数据库

MySQL常见的性能优化方式

读者答：索引和分库分表

加了索引之后，插入速度跟那个读的速度有什么区别？

读者答：b+树索引结构它其实跟实际的磁盘结构是比较一致的，使得随机写变成相关的顺序写，插入速度变快的。将所有的数据结数据信息的话都存在叶子节点，能够提高范围查找的速度。

Redis在架构中会起到什么作用？

读者答：当缓存使用的

可以直接存到进程内存，现在为什么要专门做Redis？

读者答：因为Redis快（面试官说不是，因为还存在进程通讯，不如直接内存快；自己去实现一份缓存不一定有Redis好，在分布式架构中Redis能做同步）

算法题

- 打印圣诞树
- 每隔1s调用一个会panic然后之后会recover的函数

再战美团！

大家好，我是小林。

分享一位读者面试美团 java 岗位的面经。主要在考察 java+mysql

算法题

最长回文串

根据前序中序恢复二叉树

说说MySQL的索引

回答：

- （1）根据数据结构分类：B+树索引，Hash索引，全文索引；Hash索引O(1)查找，B+树是O(log n)查找，B+树支持范围查找。
- （2）根据存储分类：聚集索引和非聚集索引

说说事务，什么是ACID，什么是一致性

回答：

- 原子性，一致性，隔离性和持久性，一致性指的是事务前和事务后数据满足完整性约束

补充：

MySQL 事务有 ACID 四大特性，分别是原子性、隔离性、持久性、一致性。

- 首先原子性，意思是事务要么全部完成，要么全部不完成，原子性是由 undo log 日志保证的；
- 接着隔离性，意思是防止多个事务并发读写同一个数据的时候，导致数据不一致问题的发生，隔离性是由 MVCC 和锁保证的；
- 然后持久性，意思是保证事务完成后对数据的修改就是永久的，不会因为系统故障而丢失，持久性是由 redo log 日志保证的；
- 最后一致性，意思是事务执行前后，数据库的状态必须保持一致性，一致性是由通过持久性+原子性+隔离性这三个共同保证的；

说说分布式事务的一致性

回答：分布式事务中各个参与者要么都执行，要么都不执行（这里其实讲的是原子性）

追问：你讲的偏向原子性，还有跟时间顺序和先后顺序相关的一致性

追问：项目中用过分布式事务吗

回答：没有

说说Redis的底层数据结构

回答：String, List, Set, ZSet

追问：ZSet底层数据结构

回答：ListPack或跳表加哈希表

追问：还了解其他底层数据结构吗

回答：链表相关，ZipList, QuickList；Hash表相关

Spring bean和application的生命周期，项目中用过那些钩子吗

回答：首先注册bean到IOC容器

追问：bean有几种注入方式

回答：根据构造器进行注入，根据set方法进行注入

追问：假如有多个对象互相依赖怎么处理

回答：。。。

追问：他底层用的是多个map，将不同状态的对象缓存起来，比如半装载，完全装载对象

补充：

Spring 中设计了三级缓存来解决循环依赖问题，当我们去调用 `getBean()` 方法的时候，Spring 会先从一级缓存中去找到目标 Bean，如果发现一级缓存中没有便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标 Bean 还没有实例化。于是，Spring 容器会实例化目标 Bean（PS：刚初始化的 Bean 称为早期 Bean）。

然后，将目标 Bean 放入到二级缓存中，同时，加上标记是 否存在循环依赖。如果不存在循环依赖便会将目标 Bean 存入到二级缓存，否则，便会标记该 Bean 存在循环依赖，然后将等待下一次轮询赋值，也就是解析 `@Autowired` 注解。等 `@Autowired` 注解赋值完成后，会将目标 Bean 存入到一级缓存。Spring 一级缓存中存放所有的成熟 Bean，二级缓存中存放所有的早期 Bean，先取一级缓存，再去二级缓存。

java中线程状态有哪些

回答：创建，就绪，运行，等待，结束

补充：

1. NEW：表示线程已经被创建，但是还没有开始执行。
2. RUNNABLE：表示线程正在Java虚拟机中执行，但是它可能正在等待操作系统分配CPU时间片。
3. BLOCKED：表示线程正在等待锁，以便进入同步块或方法。
4. WAITING：表示线程正在等待另一个线程执行特定的操作，例如调用 `Thread.sleep()` 方法或等待 I/O 操作完成。
5. TIMED_WAITING：类似于WAITING状态，但是线程在等待一段时间后会自动返回RUNNABLE状态，例如调用 `Thread.sleep(long millis)` 方法。
6. TERMINATED：表示线程已经执行完毕，已经退出了。

讲一下JVM内存结构

回答：元数据区，堆区，栈

追问：jdk1.7和1.8的区别了解吗

回答：1.7使用永久代，1.8替换为元数据区，并把常量池放入了堆区

补充：

在 1.7 版本里面，永久代内存是有上限的，虽然我们可以通过参数来设置，但是 JVM 加载的 class 总数、大小是很难确定的。所以很容易出现 OOM 问题。但是元空间是存储在本地内存里面，内存上限比较大，可以很好的避免这个问题。永久代的对象是通过 FullGC 进行垃圾收集，也就是和老年代同时实现垃圾收集。替换成元空间以后，简化了 Full GC。可以在不进行暂停的情况下并发地释放类数据，同时也提升了 GC 的性能 Oracle 要合并 Hotspot 和 JRockit 的代码，而 JRockit 没有永久代。

synchronized是怎么实现的

回答：使用 `monitorenter` 和 `monitorexit`，在对象头进行加锁

补充：最好把锁升级的过程都讲清楚

volatile实现什么能力，怎么实现的

回答：可见性，CPU core分别有自己的cache，彼此之间不可见，使用volatile会强制将cache写入主存，和从主存加载数据

补充：还有禁止指令重排，当一个变量被volatile修饰时，编译器和处理器会禁止对其进行指令重排序，从而保证程序的正确性

java匿名类和lambda表达式为什么要将变量final

回答：final是不可变的，线程安全，避免共享变量被其他线程修改

缓存设计有哪些方案

回答：常见的有cache aside, read/write through

追问：区别是什么？

回答：

- (1) cache aside, client需要和cache和db交互，针对读，client先访问cache，miss后访问db，并写回cache；针对写，client直接写db，并将cache设为失效；
- (2) read/write through, client只需要跟存储服务交互，存储服务去操作cache和db，针对读，存储服务访问cache，miss后，存储服务访问db加载数据并存入cache；针对写，如果数据在cache中，存储服务写cache和db，cache写回db，如果数据不在cache中，存储服务写db；
- (3) write behind caching; client只需要跟cache交互，针对读，client访问cache，miss后，cache访问db加载数据并存入cache；针对写，将数据写到cache，cache异步写回db



图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

① 关注公众号回复「**图解**」
获取图解系列 PDF

② 关注公众号回复「**加群**」
拉你进百人技术交流群

扫一扫，关注「小林coding」公众号

腾讯二面顶住了！评价反馈不错

大家好，我是小林。

分享一篇腾讯春招二面面经，岗位是C++后端，考察的内容是C++、Redis、网络。

shared_ptr的原理

答：内部的共享数据和引用计数实现

补充：

shared_ptr多个指针指向相同的对象。shared_ptr使用引用计数，每一个shared_ptr的拷贝都指向相同的内存。每使用他一次，内部的引用计数加1，每析构一次，内部的引用计数减1，减为0时，自动删除所指向的堆内存。shared_ptr内部的引用计数是线程安全的，但是对象的读取需要加锁。

shared_ptr的一个最大的陷阱是循环引用，循环引用会导致堆内存无法正确释放，导致内存泄漏，可以通过 weak_ptr 来解决这个问题。

多线程怎么保证引用计数的安全的

答：引用计数这个变量是std::atomic，操作时自带锁

常见的锁有哪些

答：读写锁、互斥锁这些，再就是一些锁思想，比如乐观锁、悲观锁、自旋锁

volatile关键字的用处

答：避免编译器额外优化

Redis

微博上的热度排行榜用什么数据结构

答：Zset，讲了讲zrangebyscore

补充：

Zset 类型（Sorted Set，有序集合）可以根据元素的权重来排序，我们可以自己来决定每个元素的权重值。比如说，我们可以根据元素插入 Sorted Set 的时间确定权重值，先插入的元素权重小，后插入的元素权重大。

在面对需要展示最新列表、排行榜等场景时，如果数据更新频繁或者需要分页显示，可以优先考虑使用 Sorted Set。

有序集合比较典型的使用场景就是排行榜。例如学生成绩的排名榜、游戏积分排行榜、视频播放排名、电商系统中商品的销量排名等。

我们以博文点赞排名为例，小林发表了五篇博文，分别获得赞为 200、40、100、50、150。

```
# article:1 文章获得了200个赞
> ZADD user:xiaolin:ranking 200 article:1
(integer) 1
# article:2 文章获得了40个赞
```



```
> ZADD user:xiaolin:ranking 40 article:2
(integer) 1
# article:3 文章获得了100个赞
> ZADD user:xiaolin:ranking 100 article:3
(integer) 1
# article:4 文章获得了50个赞
> ZADD user:xiaolin:ranking 50 article:4
(integer) 1
# article:5 文章获得了150个赞
> ZADD user:xiaolin:ranking 150 article:5
(integer) 1
```

文章 article:4 新增一个赞，可以使用 ZINCRBY 命令（为有序集合key中元素member的分值加上 increment）：

```
> ZINCRBY user:xiaolin:ranking 1 article:4
"51"
```

查看某篇文章的赞数，可以使用 ZSCORE 命令（返回有序集合key中元素个数）：

```
> ZSCORE user:xiaolin:ranking article:4
"50"
```

获取小林文章赞数最多的 3 篇文章，可以使用 ZREVRANGE 命令（倒序获取有序集合 key 从start下标到 stop下标的元素）：

```
# WITHSCORES 表示把 score 也显示出来
> ZREVRANGE user:xiaolin:ranking 0 2 WITHSCORES
1) "article:1"
2) "200"
3) "article:5"
4) "150"
5) "article:3"
6) "100"
```

获取小林 100 赞到 200 赞的文章，可以使用 ZRANGEBYSCORE 命令（返回有序集合中指定分数区间内的成员，分数由低到高排序）：

```
> ZRANGEBYSCORE user:xiaolin:ranking 100 200 WITHSCORES
1) "article:3"
2) "100"
3) "article:5"
4) "150"
5) "article:1"
6) "200"
```

rehash的过程讲一下

答：新旧表双写，逐渐迁移

补充：

为了避免 rehash 在数据迁移过程中，因拷贝数据的耗时，影响 Redis 性能的情况，所以 Redis 采用了**渐进式 rehash**，也就是将数据的迁移的工作不再是一次性迁移完成，而是分多次迁移。

渐进式 rehash 步骤如下：

- 给「哈希表 2」 分配空间，一般会比「哈希表 1」 大 2 倍；
- 在 rehash 进行期间，每次哈希表元素进行新增、删除、查找或者更新操作时，Redis 除了会执行对应的操作之外，还会顺序将「哈希表 1」 中索引位置上的所有 key-value 迁移到「哈希表 2」 上；
- 随着处理客户端发起的哈希表操作请求数量越多，最终在某个时间点会把「哈希表 1」 的所有 key-value 迁移到「哈希表 2」，从而完成 rehash 操作。

这样就巧妙地把一次性大量数据迁移工作的开销，分摊到了多次处理请求的过程中，避免了一次性 rehash 的耗时操作。

在进行渐进式 rehash 的过程中，会有两个哈希表，所以在渐进式 rehash 进行期间，哈希表元素的删除、查找、更新等操作都会在这两个哈希表进行。

比如，查找一个 key 的值的话，先会在「哈希表 1」 里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

另外，在渐进式 rehash 进行期间，新增一个 key-value 时，会被保存到「哈希表 2」 里面，而「哈希表 1」 则不再进行任何添加操作，这样保证了「哈希表 1」 的 key-value 数量只会减少，随着 rehash 操作的完成，最终「哈希表 1」 就会变成空表。

迁移过程中老表是什么时候释放，怎么知道老表可以释放了

答：通过数据长度

补充：

每个 hash table 都有存着一个 used 字段，每次单步 rehash 完成的时候，最后都会检查老表即 ht[0].used 是否变成了 0，变成 0 后，就说明老的哈希表里已经没有数据了，此时就会去 free 掉老表，交换老表新表的指针，rehashidx 置为 -1，然后就完成了整个 rehash。

```
/* Check if we already rehashed the whole table... */
if (d->ht[0].used == 0) {
    zfree(d->ht[0].table);
    d->ht[0] = d->ht[1];
    _dictReset(&d->ht[1]);
    d->rehashidx = -1;
    return 0;
}

/* More to rehash... */
return 1;
```

```
/* Check if we already rehashed the whole table... */
if (d->ht_used[0] == 0) {
    zfree(d->ht_table[0]);
    /* Copy the new ht onto the old one */
    d->ht_table[0] = d->ht_table[1];
    d->ht_used[0] = d->ht_used[1];
    d->ht_size_exp[0] = d->ht_size_exp[1];
    _dictReset(d, 1);
    d->rehashidx = -1;
    return 0;
}
```

不同地区的用户的请求怎么打到附近的地区呢？

答：讲了CDN

补充：

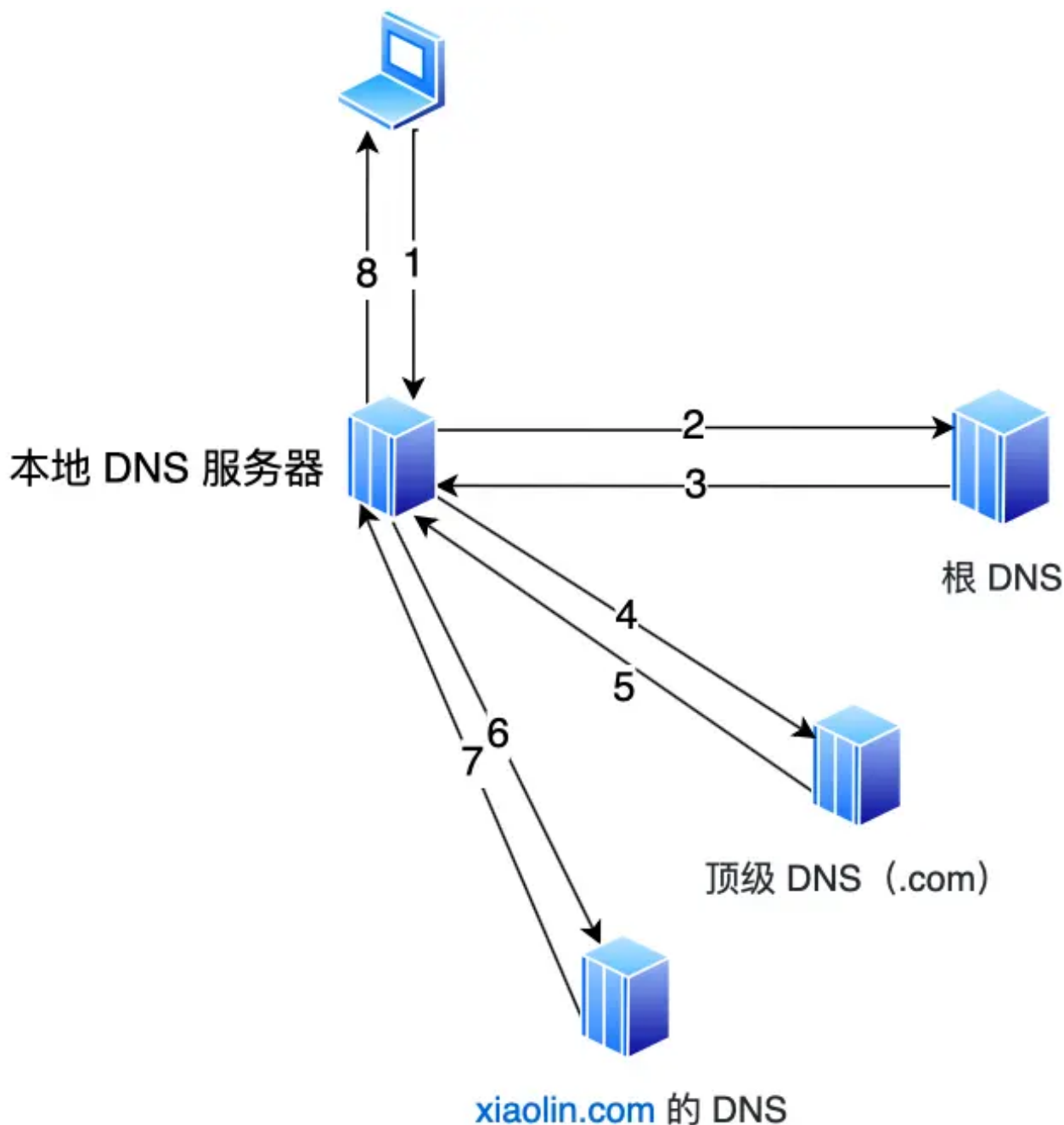
CDN 将内容资源分发到位于多个地理位置机房中的服务器上，这样我们在访问内容资源的时候，不用访问源服务器。而是直接访问离我们最近的 CDN 节点，这样一来就省去了长途跋涉的时间成本，从而实现了网络加速。

找到离用户最近的 CDN 节点是由 CDN 的**全局负载均衡器 (*Global Sever Load Balance, GSLB*)**负责的。

那 GSLB 是在什么时候起作用的呢？在回答这个问题前，我们先来看看在没有 CDN 的情况下，访问域名时发生的事情。

在没有 CDN 的情况下，当我们访问域名时，DNS 服务器最终会返回源服务器的地址。

比如，当我们在浏览器输入 www.xiaolin.com 域名后，在本地 host 文件找不到域名时，客户端就会访问本地 DNS 服务器。

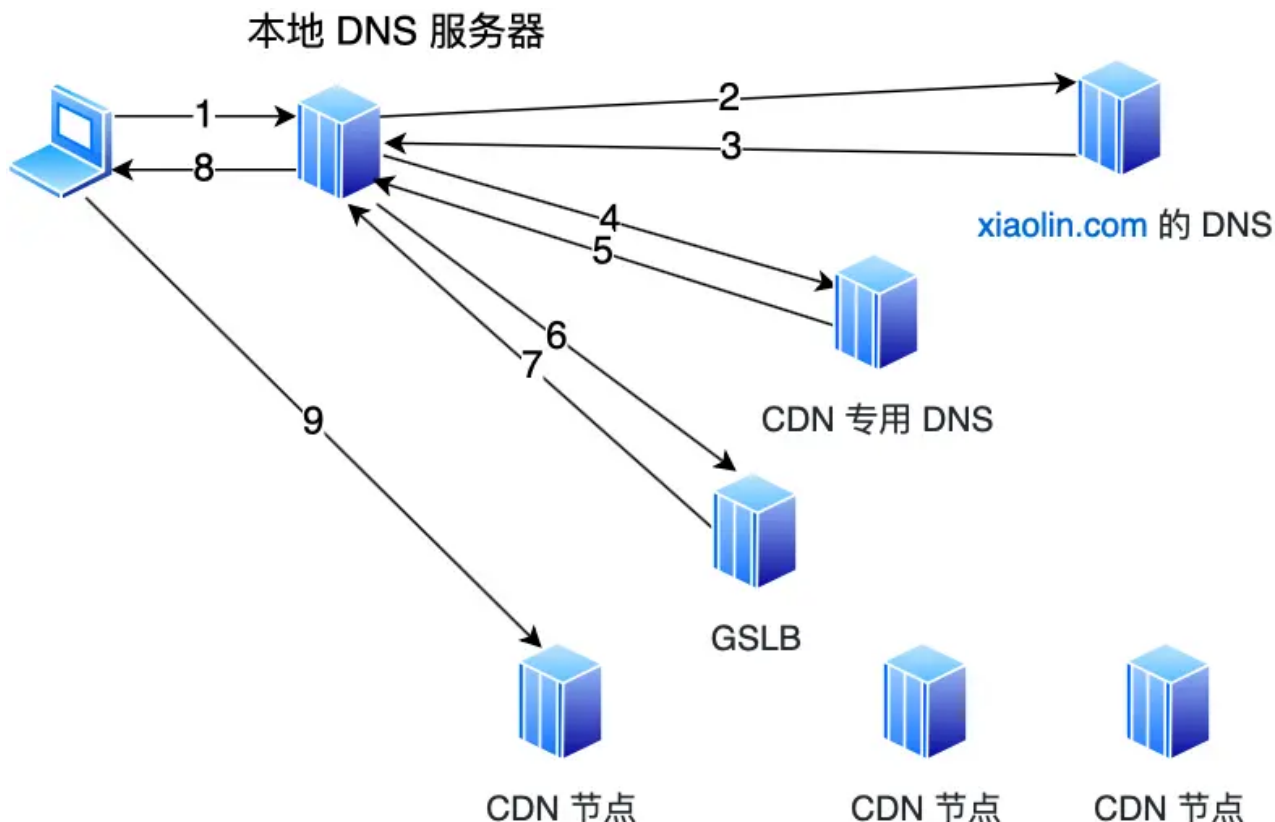


图片

这时候:

- 如果本地 DNS 服务器有缓存该网站的地址，则直接返回网站的地址；
- 如果没有就通过递归查询的方式，先请求根 DNS，根 DNS 返回顶级 DNS (.com) 的地址；再请求 .com 顶级 DNS 得到 xiaolin.com 的域名服务器地址，再从 xiaolin.com 的域名服务器中查询到 www.xiaolin.com 对应的 IP 地址，然后返回这个 IP 地址，同时本地 DNS 缓存该 IP 地址，这样下一次的解析同一个域名就不需要做 DNS 的迭代查询了。

但加入 CDN 后就不一样了。



图片

会在 xiaolin.com 这个 DNS 服务器上，设置一个 CNAME 别名，指向另外一个域名 www.xiaolin.cdn.com，返回给本地 DNS 服务器。

接着继续解析该域名，这个时候访问的就是 xiaolin.cdn.com 这台 CDN 专用的 DNS 服务器，在这个服务器上，又会设置一个 CNAME，指向另外一个域名，这次指向的就是 CDN 的 GSLB。

接着，本地 DNS 服务器去请求 CDN 的 GSLB 的域名，GSLB 就会为用户选择一台合适的 CDN 节点提供服务，选择的依据主要有以下几点：

- 看用户的 IP 地址，查表得知地理位置，找相对最近的 CDN 节点；
- 看用户所在的运营商网络，找相同网络的 CDN 节点；
- 看用户请求 URL，判断哪一台服务器上有用户所请求的资源；
- 查询 CDN 节点的负载情况，找负载较轻的节点；

GSLB 会基于以上的条件进行综合分析后，找出一台最合适的 CDN 节点，并返回该 CDN 节点的 IP 地址给本地 DNS 服务器，然后本地 DNS 服务器缓存该 IP 地址，并将 IP 返回给客户端，客户端去访问这个 CDN 节点，下载资源。

TCP的close_wait在哪端，如果我们场景中出现了大量的close_wait，你觉得要怎么排查

答：被动方，代码逻辑有问题，没close

补充：

CLOSE_WAIT 状态是「被动关闭方」才会有的状态，而且如果「被动关闭方」没有调用 close 函数关闭连接，那么就无法发出 FIN 报文，从而无法使得 CLOSE_WAIT 状态的连接转变为 LAST_ACK 状态。

所以，当服务端出现大量 **CLOSE_WAIT** 状态的连接的时候，说明服务端的程序没有调用 **close** 函数关闭连接。

那什么情况会导致服务端的程序没有调用 **close** 函数关闭连接？这时候通常需要排查代码。

我们先来分析一个普通的 TCP 服务端的流程：

1. 创建服务端 socket，bind 绑定端口、listen 监听端口
2. 将服务端 socket 注册到 epoll
3. epoll_wait 等待连接到来，连接到来时，调用 **accept** 获取已连接的 socket
4. 将已连接的 socket 注册到 epoll
5. epoll_wait 等待事件发生
6. 对方连接关闭时，我方调用 **close**

可能导致服务端没有调用 **close** 函数的原因，如下。

第一个原因：第 2 步没有做，没有将服务端 socket 注册到 epoll，这样有新连接到来时，服务端没办法感知这个事件，也就无法获取到已连接的 socket，那服务端自然就没机会对 socket 调用 **close** 函数了。

不过这种原因发生的概率比较小，这种属于明显的代码逻辑 bug，在前期 read view 阶段就能发现的了。

第二个原因：第 3 步没有做，有新连接到来时没有调用 **accept** 获取该连接的 socket，导致当有大量的客户端主动断开了连接，而服务端没机会对这些 socket 调用 **close** 函数，从而导致服务端出现大量 **CLOSE_WAIT** 状态的连接。

发生这种情况可能是因为服务端在执行 **accept** 函数之前，代码卡在某一个逻辑或者提前抛出了异常。

第三个原因：第 4 步没有做，通过 **accept** 获取已连接的 socket 后，没有将其注册到 epoll，导致后续收到 FIN 报文的时候，服务端没办法感知这个事件，那服务端就没机会调用 **close** 函数了。

发生这种情况可能是因为服务端在将已连接的 socket 注册到 epoll 之前，代码卡在某一个逻辑或者提前抛出了异常。之前看到过别人解决 **close_wait** 问题的实践文章，感兴趣的可以看看：[一次 Netty 代码不健壮导致的大量 CLOSE_WAIT 连接原因分析](#)

第四个原因：第 6 步没有做，当发现客户端关闭连接后，服务端没有执行 **close** 函数，可能是因为代码漏处理，或者是在执行 **close** 函数之前，代码卡在某一个逻辑，比如发生死锁等等。

可以发现，当服务端出现大量 **CLOSE_WAIT** 状态的连接的时候，通常都是代码的问题，这时候我们需要针对具体的代码一步一步的进行排查和定位，主要分析的方向就是服务端为什么没有调用 **close**。

TCP粘包问题怎么解决

- 答：特殊标记
- 追问：打断，如果使用特殊标记解决会遇到什么问题
- 答：正文转义字符

补充：

1、固定长度的消息

这种是最简单方法，即每个用户消息都是固定长度的，比如规定一个消息的长度是 64 个字节，当接收方接满 64 个字节，就认为这个内容是一个完整且有效的消息。

但是这种方式灵活性不高，实际中很少用。

2、特殊字符作为边界

我们可以在两个用户消息之间插入一个特殊的字符串，这样接收方在接收数据时，读到了这个特殊字符，就把认为已经读完一个完整的消息。

HTTP 是一个非常好的例子。

请求方法		空格	URL		空格	协议版本		回车	换行
头部字段名		:	值		回车	换行			
头部字段名		:	值		回车	换行			
回车	换行								
请求正文									

图片

HTTP 通过设置回车符、换行符作为 HTTP 报文协议的边界。

有一点要注意，这个作为边界点的特殊字符，如果刚好消息内容里有这个特殊字符，我们要对这个字符转义，避免被接收方当作消息的边界点而解析到无效的数据。

3、自定义消息结构

我们可以自定义一个消息结构，由包头和数据组成，其中包头包是固定大小的，而且包头里有一个字段来说明紧随其后的数据有多大。

比如这个消息结构体，首先 4 个字节大小的变量来表示数据长度，真正的数据则在后面。

```
struct {
    u_int32_t message_length;
    char message_data[];
} message;
```

当接收方接收到包头的大小（比如 4 个字节）后，就解析包头的内容，于是就可以知道数据的长度，然后接下来就继续读取数据，直到读满数据的长度，就可以组装成一个完整到用户消息来处理了。

protobuf了解吗

答：不了解

补充：

Protobuf 是用于数据序列化和反序列化的格式，类似于 json。但它们有以下几点区别：

- 数据大小：Protobuf是一种二进制格式，相对于JSON来说，数据大小更小，序列化和反序列化的效率更高，因此在网络传输和存储方面具有一定的优势。
- 性能：由于Protobuf是二进制格式，相对于JSON来说，解析速度更快，占用的CPU和内存资源更

少，因此在高并发场景下，性能更优。

- 可读性：JSON是一种文本格式，可读性更好，易于调试和排查问题。而Protobuf是一种二进制格式，可读性较差。

Protobuf适用于高性能、大数据量、高并发等场景，而JSON适用于数据交换、易读性要求高的场景。

说一下代码里使用异步的思路

答：举了个客户端异步connect，使用select检测结果的例子

顺势问select/epoll的区别

答：常规八股回答

补充：

select 和 poll 的缺陷在于，当客户端越多，也就是 Socket 集合越大，Socket 集合的遍历和拷贝会带来很大的开销，因此也很难应对 C10K。

epoll 是解决 C10K 问题的利器，通过两个方面解决了 select/poll 的问题。

- epoll 在内核里使用「红黑树」来关注进程所有待检测的 Socket，红黑树是个高效的数据结构，增删改一般时间复杂度是 $O(\log n)$ ，通过对这棵黑红树的管理，不需要像 select/poll 在每次操作时都传入整个 Socket 集合，减少了内核和用户空间大量的数据拷贝和内存分配。
- epoll 使用事件驱动的机制，内核里维护了一个「链表」来记录就绪事件，只将有事件发生的 Socket 集合传递给应用程序，不需要像 select/poll 那样轮询扫描整个集合（包含有和无事件的 Socket），大大提高了检测的效率。

IO特别密集时epoll效率还高吗

答：可以考虑select/poll，这种情况轮询也很高效，且结构简单。

补充：

可以先解释io特别密集时为什么 epoll 效率不高。原因是：

- 连接密集（短连接特别多），使用epoll的话，每一次连接需要发生epoll_wait->accept->epoll_ctl调用，而使用select只需要select->accept，减少了一次系统调用。
- 读写密集的话，如果收到数据，我们需要响应数据的话，使用epoll的情况下，read 完后也需要epoll_ctl 加入写事件，相比select多了一次系统调用

讲一讲ET模式

答：常规八股回答

补充：

epoll 支持两种事件触发模式，分别是边缘触发（edge-triggered，ET）和水平触发（level-triggered，LT）。

这两个术语还挺抽象的，其实它们的区别还是很好理解的。

- 使用边缘触发模式时，当被监控的 Socket 描述符上有可读事件发生时，服务器端只会从

epoll_wait 中苏醒一次，即使进程没有调用 read 函数从内核读取数据，也依然只苏醒一次，因此我们程序要保证一次性将内核缓冲区的数据读取完；

- 使用水平触发模式时，当被监控的 Socket 上有可读事件发生时，**服务器端不断地从 epoll_wait 中苏醒，直到内核缓冲区数据被 read 函数读完才结束**，目的是告诉我们有数据需要读取；

举个例子，你的快递被放到了一个快递箱里，如果快递箱只会通过短信通知你一次，即使你一直没有去取，它也不会再发送第二条短信提醒你，这个方式就是边缘触发；如果快递箱发现你的快递没有被取出，它就会不停地发短信通知你，直到你取出了快递，它才消停，这个就是水平触发的方式。

这就是两者的区别，水平触发的意思是只要满足事件的条件，比如内核中有数据需要读，就一直不断地把这个事件传递给用户；而边缘触发的意思是只有第一次满足条件的时候才触发，之后就不会再传递同样的事件了。

如果使用水平触发模式，当内核通知文件描述符可读写时，接下来还可以继续去检测它的状态，看它是否依然可读或可写。所以在收到通知后，没必要一次执行尽可能多的读写操作。

如果使用边缘触发模式，I/O 事件发生时只会通知一次，而且我们不知道到底能读写多少数据，所以在收到通知后应尽可能地读写数据，以免错失读写的机会。因此，我们会**循环**从文件描述符读写数据，那么如果文件描述符是阻塞的，没有数据可读写时，进程会阻塞在读写函数那里，程序就没办法继续往下执行。所以，**边缘触发模式一般和非阻塞 I/O 搭配使用**，程序会一直执行 I/O 操作，直到系统调用（如 read 和 write）返回错误，错误类型为 EAGAIN 或 EWOULDBLOCK。

一般来说，边缘触发的效率比水平触发的效率要高，因为边缘触发可以减少 epoll_wait 的系统调用次数，系统调用也是有一定的开销的，毕竟也存在上下文的切换。

对于一个后端服务怎么提升性能

- 回答：讲了存储层面的使用中间缓存、网络框架设计优化
- 追问：提示长连接短连接
- 追问：怎么避免拷贝呢，不同的层面说说
- 回答：C++移动语义、零拷贝
- 追问：零拷贝的使用场景
- 追问：kafka了解吗
- 回答：不了解
- 追问：建议关注一下kafka，很快很好用

算法

手撕：合并两个有序链表

感受

基础问题大部分还好，面试官最后评价还不错。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

① 关注公众号回复「**图解**」
获取图解系列 PDF

② 关注公众号回复「**加群**」
拉你进百人技术交流群

感觉被榨干了，被美团拷打一小时！

大家好，我是小林。

上一次分享了 C++ 面经，这次就来分享 Java 同学的面经理。

今天是美团春招实习的 Java 岗的面经，总共被问了接近 50 个八股文，问了非常多 Java 框架和 Java 并发的问题

面试时长超过了 1 个小时，感觉被榨干了。

MySQL

可重复读和已提交读隔离级别表现的现象是什么，区别是什么样的？

已提交读只能读取其他事务已经提交的数据，但是这个隔离级别就会造成一个不可重复读和幻读的现象，可重复读就是消除了不可重复读和幻读的现象。

补充：

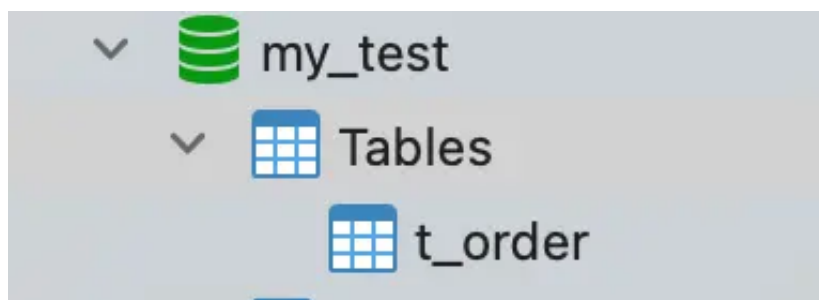
- 读提交，指一个事务提交之后，它做的变更才能被其他事务看到，会有不可重复读、幻读的问题。
- 可重复读，指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，MySQL InnoDB 引擎的默认隔离级别，解决了不可重复读的问题，并且以很大程度上避免幻读现象的发生。

数据文件大体分成哪几种数据文件？

因为是聚簇索引，所以只有一个文件，索引和数据是放在一起的，就是一个bd文件。

补充：我们每创建一个 database（数据库）都会在 /var/lib/mysql/ 目录里面创建一个以 database 为名的目录，然后保存表结构和表数据的文件都会存放在这个目录里。

比如，我这里有一个名为 my_test 的 database，该 database 里有一张名为 t_order 数据库表。



然后，我们进入 /var/lib/mysql/my_test 目录，看看里面有什么文件？

```
[root@xiaolin ~]#ls /var/lib/mysql/my_test
db.opt
t_order.frm
t_order.ibd
```

可以看到，共有三个文件，这三个文件分别代表着：

- db.opt，用来存储当前数据库的默认字符集和字符校验规则。
- t_order.frm，t_order 的表结构会保存在这个文件。在 MySQL 中建立一张表都会生成一个.frm 文件，该文件是用来保存每个表的元数据信息的，主要包含表结构定义。
- t_order.ibd，t_order 的表数据会保存在这个文件。表数据既可以存在共享表空间文件（文件名：ibdata1）里，也可以存放在独占表空间文件（文件名：表名字.ibd）。这个行为是由参数 innodb_file_per_table 控制的，若设置了参数 innodb_file_per_table 为 1，则会将存储的数据、索引等信息单独存储在一个独占表空间，从 MySQL 5.6.6 版本开始，它的默认值就是 1 了，因此从这个版本之后，MySQL 中每一张表的数据都存放在一个独立的 .ibd 文件。

mysql日志文件是分成了哪几种？

mysql有三种日志文件，binlog，redolog和undolog。

补充：

- redo log 重做日志，确保事务的持久性
- undo log 回滚日志，确保事务的原子性，用于回滚事务，同时提供mvcc下的非锁定读
- bin log 二进制日志，用于主从复制场景下，记录master做过的操作
- relay log 中继日志，用于主从复制场景下，slave通过io线程拷贝master的bin log后本地生成的日志
- 慢查询日志，用于记录执行时间过长的sql，需要设置阈值后手动开启

说下MVCC机制的原理？

MVCC就是多版本并发控制，实现了读写的并发控制，在mysql通过readview 隐藏字段和undolog实现了，比如在可重复读里面，比如开启了一个事务，就生成了一个readview，然后记录现在active的事务，判断查询的数据在这个事务可不可读。

索引的类型有哈希索引，B+树索引，而hash索引的时间复杂度是 $O(1)$ ，那为什么我们一般情况下不使用哈希索引，而使用b+树索引呢？

因为hash索引只能做一个等值查询，像范围查询是做不到的。还有哈希可能会出现hash碰撞的问题。

还能想到其他的原因吗？

想不到了

补充：

- 哈希索引的key是经过hash运算得出的，即跟实际数据的值没有关系，因此哈希索引不适用于范围查询和排序操作
- 容易导致全表扫描，因为可能存在不同的key经过hash运算后值相同
- 索引列上的值相同的话，易造成hash冲突，效率低下

对一个慢sql怎么去排查？

如果是在项目中，可以通过SpringAOP去查询这个接口运行的时间，如果是一个sql，可以通过explain的指令去查这个sql的执行计划。

补充：

可通过开启mysql的慢日志查询，设置好时间阈值，进行捕获

索引字段是不是建的越多越好

不是，建的越多会占用越多的空间

补充：

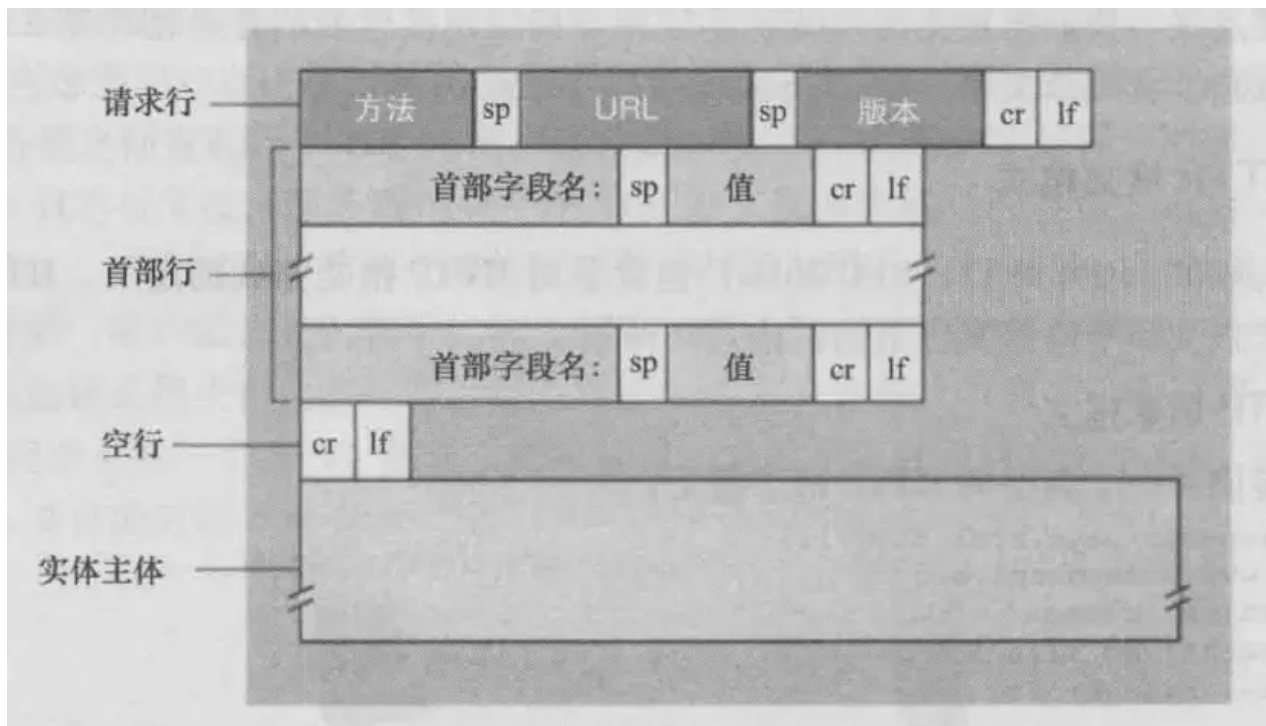
索引越多，在写入频繁的场景下，对于B+树的维护所付出的性能消耗也会越大

网络

http协议的报文的格式有了解吗？

不是太清楚，有报文头，报文体，如果是post请求就会在报文体写上数据。

补充：



HTTP 请求报文结构

HTTP 的请求报文分为三个部分：

请求行、首部行、实体主体。

http常用的状态码？

4XX是请求报文有误，5XX是服务器有错，2XX是成功的。3XX忘了（是重定向

补充：

RFC 规定 HTTP 的状态码为三位数，被分为五类：

- **1xx**：表示目前是协议处理的中间状态，还需要后续操作
- **2xx**：表示成功状态
- **3xx**：重定向状态，资源位置发生变动，需要重新请求
- **4xx**：请求报文有误
- **5xx**：服务端发生错误

Java框架

java这一块对框架都是熟悉的吧？

只用过SSM，SpringBoot还在学中。

MyBatis运用了哪些常见的设计模式？

运用了工厂模式，创建sqlsession的时候用了工厂模式，其他的没想起来。

补充：

- 工厂模式，工厂模式在 MyBatis 中的典型代表是 SqlSessionFactory
- 建造者模式，建造者模式在 MyBatis 中的典型代表是 SqlSessionFactoryBuilder

- 单例模式，单例模式在 MyBatis 中的典型代表是 ErrorContext
- 适配器模式，适配器模式在 MyBatis 中的典型代表是 Log
- 代理模式，代理模式在 MyBatis 中的典型代表是 MapperProxyFactory
- 模板方法模式，模板方法在 MyBatis 中的典型代表是 BaseExecutor
- 装饰器模式，装饰器模式在 MyBatis 中的典型代表是 Cache

MyBatis中创建了一个Mapper接口，在写一个xml文件，java的接口是要实现的，为什么这没有实现呢？

（现在回想起来，是要诱导我说动态代理）一个mapper接口通过namespace的id对应的就是一个xml的文件。没了解背后的原理

补充：

MyBatis中的Mapper接口并不需要实现，它只是定义了一组方法签名。MyBatis会根据Mapper接口中的方法名、参数类型和返回值类型，自动生成实现方法。因此，Mapper接口中的方法不需要实现，也不需要在该接口中编写任何方法体。

相反，你需要编写一个与Mapper接口同名的XML文件，来实现这些方法的具体SQL操作。这样，当你在Java代码中调用Mapper接口中的方法时，MyBatis会自动将该方法映射到对应的XML文件中的SQL语句，并执行该语句。

与传统的JDBC相比，MyBatis的优点？

有些功能封装的更好，像打开一个sqlsession的连接，而且写的代码要少

补充：

- mybatis的全局配置文件中可以设置数据库连接池，和spring整合可以配置数据库连接
- mybatis把sql和代码分离，提供了Mapper.xml映射文件，在映射文件中通过标签来写sql
- mybatis中自动完成java对象和sql中参数的映射
- mybatis中通过ResultSetHandler自动将结果集映射到对应的java对象中

还记得JDBC连接数据库的步骤吗？

不记得了，太久没用JDBC

补充：

使用JDBC连接数据库的步骤如下：

- 加载数据库驱动程序：使用Class.forName()方法加载对应的数据库驱动程序，例如：
Class.forName("com.mysql.jdbc.Driver");
- 建立数据库连接：使用DriverManager.getConnection()方法建立与数据库的连接，需要指定数据库的URL、用户名和密码，例如：Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost/mydatabase", "username",
"password");
- 创建Statement对象：使用Connection对象的createStatement()方法创建一个Statement对象，用于执行SQL语句，例如：Statement stmt = conn.createStatement();
- 执行SQL语句：使用Statement对象的executeQuery()或executeUpdate()方法执行SQL语句，例如：ResultSet rs = stmt.executeQuery("SELECT * FROM mytable");

- 处理查询结果：如果执行的是查询语句，需要使用ResultSet对象来处理查询结果，例如：

```
while (rs.next()) { String name = rs.getString("name"); int age = rs.getInt("age"); }
```
- 关闭数据库连接：在程序结束时，需要使用Connection对象的close()方法关闭数据库连接，例如：

```
conn.close();
```

怎么理解SpringIoc?

SpringIoc就是控制反转，把创建对象的权力交给了框架，然后Spring通过反射就会创建一个对象。当我创建的一个类要用到这个对象，Spring就会把这个对象交给我。

如果让你设计一个SpringIoc，你觉得会从哪些方面考虑这个设计？

- 多线程，这个对象是单例的还是每个线程都特有的。
- 可能会有循环依赖的问题，像对象A有B的引用，然后对象B有A的引用
- bean的生命周期也需要考虑。

补充：

1. Bean的生命周期管理：需要设计Bean的创建、初始化、销毁等生命周期管理机制，可以考虑使用工厂模式和单例模式来实现。
2. 依赖注入：需要实现依赖注入的功能，包括属性注入、构造函数注入、方法注入等，可以考虑使用反射机制和XML配置文件来实现。
3. Bean的作用域：需要支持多种Bean作用域，比如单例、原型、会话、请求等，可以考虑使用Map来存储不同作用域的Bean实例。
4. AOP功能的支持：需要支持AOP功能，可以考虑使用动态代理机制和切面编程来实现。
5. 异常处理：需要考虑异常处理机制，包括Bean创建异常、依赖注入异常等，可以考虑使用try-catch机制来处理异常。
6. 配置文件加载：需要支持从不同的配置文件中加载Bean的相关信息，可以考虑使用XML、注解或者Java配置类来实现。

Spring给我们提供了很多扩展点，这些有了解吗？

不太清楚扩展点指的什么

补充：

Spring框架提供了许多扩展点，使得开发者可以根据需求定制和扩展Spring的功能。以下是一些常用的扩展点：

1. BeanFactoryPostProcessor：允许在Spring容器实例化bean之前修改bean的定义。常用于修改bean属性或改变bean的作用域。
2. BeanPostProcessor：可以在bean实例化、配置以及初始化之后对其进行额外处理。常用于代理bean、修改bean属性等。
3. PropertySource：用于定义不同的属性源，如文件、数据库等，以便在Spring应用中使用。
4. ImportSelector和ImportBeanDefinitionRegistrar：用于根据条件动态注册bean定义，实现配置类的模块化。
5. Spring MVC中的HandlerInterceptor：用于拦截处理请求，可以在请求处理前、处理中和处理后执行特定逻辑。
6. Spring MVC中的ControllerAdvice：用于全局处理控制器的异常、数据绑定和数据校验。
7. Spring Boot的自动配置：通过创建自定义的自动配置类，可以实现对框架和第三方库的自动配

置。

8. 自定义注解：创建自定义注解，用于实现特定功能或约定，如权限控制、日志记录等。

Servlet有写过简单的代码吗？

没用框架之前，就用的Servlet，用了框架就用的框架

大致了解SpringMVC的处理流程吗？

首先通过一个DispatchServlet去转接请求，到HandlerMapping去返回一个执行链，就比如拦截器到哪个Controller，返回以后就到Handler适配器获取这个请求要求的Controller，然后去Controller这里返回一个数据或者页面ModelAndView，然后给前端。

SpringAOP主要想解决什么问题

提供了一个扩展功能，可以一个类的某个方法进行加强，比如在之前加强，在之后加强，环绕加强。

补充：

Spring AOP主要解决的是横切关注点的问题，即在一个系统中，可能存在多个模块或组件都需要实现类似的功能，比如日志记录、权限校验、事务管理等等。如果每个模块都去实现这些功能，就会导致代码冗余，可维护性和可扩展性降低。而AOP则是基于动态代理的机制，在不修改原有代码的情况下，通过在代码执行前后插入增强代码的方式，实现对横切关注点的统一处理，从而提高代码的复用性和可维护性。

SpringAOP的原理了解吗

基于一个动态代理的设计模式，如果动态加强的类实现了某个接口，就会用JDK动态代理，如果是对于没有实现接口的类，就会用CGLIB动态代理模板，去生成一个被代理对象的一个子类来作为代理对象。

补充：

Spring AOP的主要目的是将横切关注点（如日志、安全和事务管理等）从业务逻辑中分离出来，从而提高代码的模块性和可维护性。

原理主要包括以下几个方面：

1. 代理模式：Spring AOP基于代理模式实现，主要有两种代理方式，JDK动态代理和CGLIB代理。JDK动态代理要求目标类必须实现接口，而CGLIB代理则可以针对没有实现接口的类进行代理。
2. 切面（Aspect）：切面是将横切关注点模块化的实现。切面通常包含通知（Advice）和切点（Pointcut）。通知是在特定的切点执行的动作，切点则用于定义通知应该在何处执行。
3. 连接点（Joinpoint）：连接点代表在应用程序中可以插入切面的点，如方法调用、异常处理等。
4. 织入（Weaving）：织入是将切面应用到目标对象的过程，从而创建代理对象。在Spring AOP中，织入过程发生在运行时。

通过以上原理，Spring AOP能够在不修改原有业务代码的情况下，将横切关注点进行模块化管理，提高代码的可读性和易维护性。

动态代理和静态代理的区别

静态代理是自己手写一个代理类，但是动态代理不需要直接实现这个代理类的，相当于静态代理在编译的时候就已经变成了一个二进制文件了，动态代理在运行的时候动态生成类的字节码文件。

动态代理中如果有实现了接口，就会用JDK动态代理呢？

不太了解（后来查了应该是因为JDK动态代理会继承Proxy类，但是java是单继承）

代理模式和适配器模式有什么区别？

代理模式主要是去加强一个类的方法。适配器模式是接口转换成一个想要的接口（这个问题被面试说回答的不好）

补充：

代理模式和适配器模式是两种常用的设计模式，它们的区别主要体现在以下几个方面：

1. 作用不同：代理模式是为了控制对对象的访问，而适配器模式是为了解决接口不匹配的问题。
2. 解决问题的角度不同：代理模式是从外部控制访问，保护目标对象，而适配器模式是从内部改变对象接口，让其能够适配客户端的要求。
3. 实现方式不同：代理模式通常使用面向对象的继承或者组合方式实现，而适配器模式则通常使用对象组合方式实现。
4. 适用场景不同：代理模式适用于需要对对象进行控制和保护的情况，例如远程代理、虚拟代理等。适配器模式适用于需要将一个类的接口转换成客户端期望的另一个接口的情况，例如旧系统的升级改造、不兼容接口的统一等。

java 并发

java线程的生命周期有了解吗？

new就是创建一个线程，变成ready的状态，如果分配到时间片，就会是一个运行的状态，等待其他线程做出一个动作就是waiting的状态，如果是等待其他资源的释放，就是block的状态，最后是一个终止的状态。

使用多线程要注意哪些问题？

避免死锁，保证数据的可见性或者多个线程对这个数据的一致性。

保证数据的一致性有哪些方案呢？

比如有volatile修饰一个变量，或者synchronized或者加锁。

线程池有了解吗？线程池大概的原理？

分为核心线程池，线程池的最大容量，还有等待任务的队列，提交一个任务，如果核心线程没有满，就创建一个线程，如果满了，就会加入等待队列，如果等待队列满了，就会增加线程，如果达到最大线程数量，如果都达到最大线程数量，就会按照一些丢弃的策略进行处理。

ArrayList和LinkedList有什么区别

ArrayList通过数组存储数据，如果查询的话会快速定位到这个数据，但是新增和删除数据会涉及数据的移动，像LinkedList就是双向链表，定位数据会一个个查，新增和删除是一个 $O(1)$ 。

补充：

ArrayList和LinkedList是Java中的两种常用的List实现，它们的区别主要体现在底层数据结构、性能和使用场景上：

1. 底层数据结构：ArrayList基于动态数组实现，LinkedList基于双向链表实现。
2. 插入和删除操作性能：
 - ArrayList：在插入和删除元素时，需要移动元素以保持数组的连续性，所以在非尾部的插入和删除操作性能较差，时间复杂度为 $O(n)$ 。
 - LinkedList：由于基于链表实现，插入和删除元素只需修改指针，所以在任何位置的插入和删除操作性能较好，时间复杂度为 $O(1)$ 。
3. 访问和查找操作性能：
 - ArrayList：由于基于数组实现，支持随机访问，访问和查找元素的时间复杂度为 $O(1)$ 。
 - LinkedList：由于基于链表实现，需要顺序遍历链表，访问和查找元素的时间复杂度为 $O(n)$ 。
4. 内存占用：
 - ArrayList：内存占用相对较小，因为只需存储元素本身。
 - LinkedList：由于需要存储额外的指针信息（前后节点指针），内存占用相对较大。
5. 使用场景：
 - ArrayList：更适合频繁访问和查找元素的场景，如查询操作较多的情况。
 - LinkedList：更适合频繁插入和删除元素的场景，如在列表中间进行大量的增删操作。

ArrayList线程安全吗？把ArrayList变成线程安全有哪些方法？

线程不安全，使用它的线程安全类Vector（被问还有呢？）

补充：

将ArrayList变成线程安全有几种方法：

1. 使用Collections.synchronizedList()方法将ArrayList转换为线程安全的List。该方法会返回一个线程安全的List，使用该List时需要在访问它的方法上添加synchronized关键字，以保证多线程访问的安全性。
2. 使用CopyOnWriteArrayList类来代替ArrayList。CopyOnWriteArrayList是一种线程安全的List实现，它通过在写操作时复制整个数组来保证线程安全性，在读操作时不需要加锁，因此可以提高读取效率。
3. 使用Lock接口来实现同步。可以使用ReentrantLock类来实现对ArrayList的同步操作，该类提供了与synchronized类似的功能，但是具有更高的灵活性。比如可以使用tryLock()方法来尝试获取锁，避免了线程的长时间等待。
4. 使用读写锁来实现同步。可以使用ReentrantReadWriteLock类来实现对ArrayList的读写操作的同步。该类提供了读锁和写锁两种锁，多个线程可以同时获取读锁，但是只有一个线程可以获取写锁，在写操作时需要先获取写锁，以保证线程安全。

其他

对面向对象的理解？

像面向过程就是把问题分解成一个一个函数，然后调用函数去解决问题。而面向对象就是把这个世界抽象成一个一个对象，然后赋予这些对象一个属性，成员变量和方法，然后去调用对象的方法去解决问题，耦合性比较低。

面向过程的方法存在哪些问题？

耦合度比较高，分工不好分工。

补充：

1. 可维护性较差：面向过程编程主要依赖于函数和过程，随着代码规模的增大，可能会导致代码结构复杂，不易维护。
2. 可复用性较低：面向过程编程难以实现模块化，导致代码难以复用，进一步增加开发时间和成本。
3. 扩展性不足：面向过程编程在代码逻辑发生变化时，往往需要对程序进行大量的修改，这样的代码扩展性不足。
4. 抽象能力有限：面向过程编程主要关注过程和算法，而不是数据结构和对象，这使得它在表达现实世界的复杂问题时抽象能力有限。
5. 封装性差：面向过程编程没有提供良好的封装机制，程序中的数据和处理过程容易暴露，可能导致数据安全性和程序稳定性问题。
6. 强耦合：面向过程编程的方法往往导致程序组件之间存在强耦合，当一个组件发生变化时，可能会影响其他组件的正常工作。

面向过程好处是什么？

解决问题思路比较简单。

补充：

- 面向过程编程采用自顶向下的编程方式，将问题分解为一个个小的模块，便于理解和编写。
- 每个模块相对独立，出现问题时可以单独调试，降低了调试难度。
- 面向过程编程适合解决简单、逻辑性强的问题，对于初学者来说，学习成本较低。

面试总结

感觉：

- 面试官有引导，大多问的是八股，会的就回答的比较流畅，不熟悉的就磕磕巴巴
- 面试官给的反馈基础还行，但是深度不够，对刚刚设计模式的对比回答不满意，和mybatis的原理回答也不太满意

不足之处：

- 对框架还是不够熟练，回答不够全面，经常被面试官问还有呢，但就回答不上来了
- 网络这方面看了就忘，面试官建议讲项目从以下四点：项目的背景、核心解决思路、技术上的挑战点、最后取得什么结果。

腾讯面试体验倍儿好

大家好，我是小林。

今天分享一位读者的腾讯春招实习面经，岗位Java后端，主要问了MySQL、Java、网络这三大块。

他觉得这场面试非常有收获，虽然都是基础问题，但是往下深挖根茎叶脉全部相连。反问环节面试官给了我很多建议，包括面试、策略、基础、算法等等，是一次宝贵的学习经历。

MySQL

介绍一下MySQL的索引机制

索引可以帮助我们快速搜索数据，innodb 存储引擎用的是 b+树索引，叶子节点存放的是索引+数据，非叶子节点只存放索引。

可以按照四个角度来分类索引。

- 按「数据结构」分类：**B+tree索引**、**Hash索引**、**Full-text索引**。
- 按「物理存储」分类：**聚簇索引（主键索引）**、**二级索引（辅助索引）**。
- 按「字段特性」分类：**主键索引**、**唯一索引**、**普通索引**、**前缀索引**。
- 按「字段个数」分类：**单列索引**、**联合索引**。

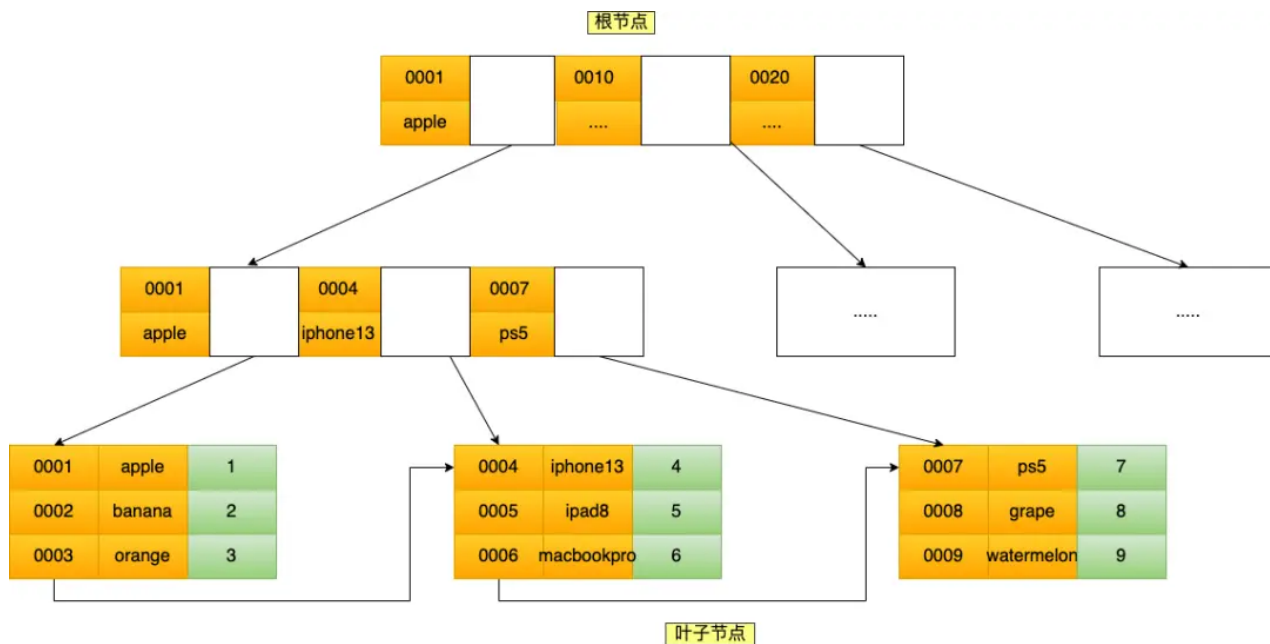
联合索引是什么？

通过将多个字段组合成一个索引，该索引就被称为联合索引。

比如，将商品表中的 product_no 和 name 字段组合成联合索引 (product_no, name)，创建联合索引的方式如下：

```
CREATE INDEX index_product_no_name ON product(product_no, name);
```

联合索引 (product_no, name) 的 B+Tree 示意图如下（图中叶子节点之间我画了单向链表，但是实际上是双向链表，原图我找不到了，修改不了，偷个懒我不重画了，大家脑补成双向链表就行）。



联合索引

可以看到，联合索引的非叶子节点用两个字段的值作为 B+Tree 的 key 值。当在联合索引查询数据时，先按 product_no 字段比较，在 product_no 相同的情况下再按 name 字段比较。

也就是说，联合索引查询的 B+Tree 是先按 product_no 进行排序，然后再 product_no 相同的情况再按 name 字段排序。

因此，使用联合索引时，存在**最左匹配原则**，也就是按照最左优先的方式进行索引的匹配。在使用联合索引进行查询的时候，如果不遵循「最左匹配原则」，联合索引会失效，这样就无法利用到索引快速查询的特性了。

什么是聚簇索引？

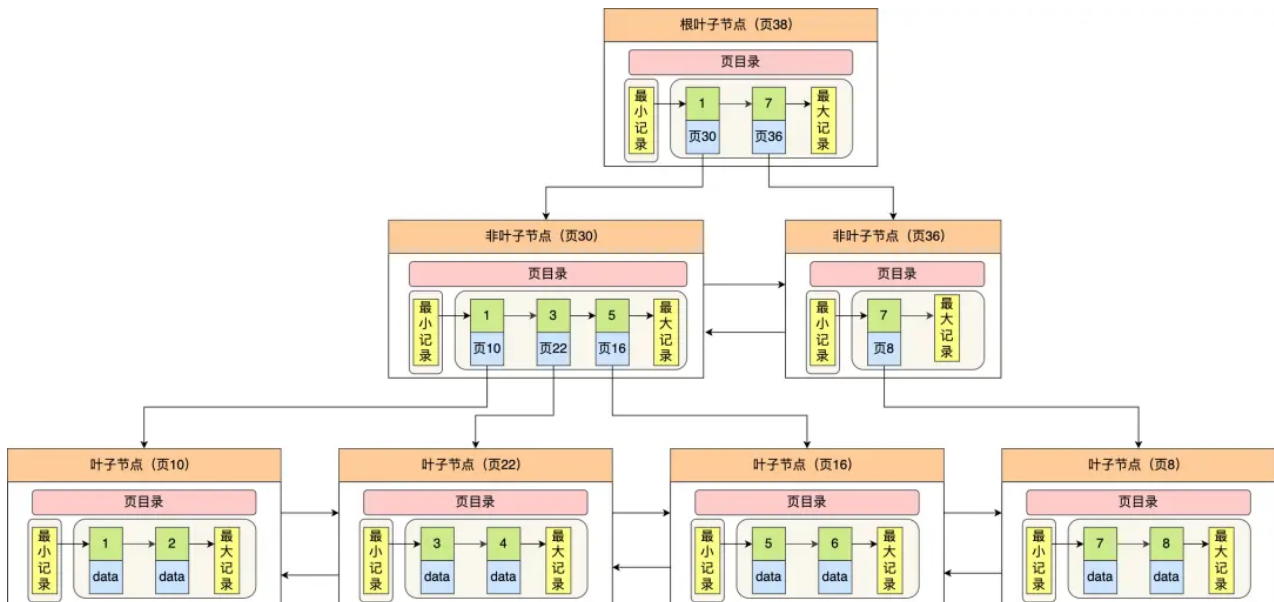
聚簇索引的 B+Tree 的叶子节点存放的是实际数据，所有完整的用户记录都存放在主键索引的 B+Tree 的叶子节点里。

什么是覆盖索引？

在查询时使用了二级索引，如果查询的数据能在二级索引里查询的到，那么就不需要回表，这个过程就是覆盖索引。如果查询的数据不在二级索引里，就会先检索二级索引，找到对应的叶子节点，获取到主键值后，然后再检索主键索引，就能查询到数据了，这个过程就是回表。

整个索引查询的过程是怎样的？

InnoDB 里的 B+ 树中的每个节点都是一个数据页，结构示意图如下：



图片

B+ 树如何实现快速查找主键为 6 的记录，以上图为例：

- 从根节点开始，通过二分法快速定位到符合页内范围包含查询值的页，因为查询的主键值为 6，在 [1, 7) 范围之间，所以到页 30 中查找更详细的目录项；
- 在非叶子节点（页30）中，继续通过二分法快速定位到符合页内范围包含查询值的页，主键值大于 5，所以就到叶子节点（页16）查找记录；
- 接着，在叶子节点（页16）中，通过槽查找记录时，使用二分法快速定位要查询的记录在哪个槽（哪个记录分组），定位到槽后，再遍历槽内的所有记录，找到主键为 6 的记录。

可以看到，在定位记录所在哪一个页时，也是通过二分法快速定位到包含该记录的页。定位到该页后，又会在该页内进行二分法快速定位记录所在的分组（槽号），最后在分组内进行遍历查找。

事务的隔离级别有哪些？

- **读未提交**，指一个事务还没提交时，它做的变更就能被其他事务看到；
- **读提交**，指一个事务提交之后，它做的变更才能被其他事务看到；
- **可重复读**，指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，**MySQL InnoDB 引擎的默认隔离级别**；
- **串行化**：会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生了读写冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行；

脏读、幻读、不可重读分别是什么意思？

- **脏读**：如果一个事务「读到」了另一个「未提交事务修改过的数据」，就意味着发生了「脏读」现象。
- **幻读**：在一个事务内多次查询某个符合查询条件的「记录数量」，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了「幻读」现象。
- **不可重复读**：在一个事务内多次读取同一个数据，如果出现前后两次读到的数据不一样的情况，就意味着发生了「不可重复读」现象。

InnoDB 多版本并发控制的具体原理，底层细节？

对于「读提交」和「可重复读」隔离级别的事务来说，它们是通过 Read View 来实现的，它们的区别在于创建 Read View 的时机不同，大家可以把 Read View 理解成一个数据快照，就像相机拍照那样，定格某一时刻的风景。

- 「读提交」隔离级别是在「每个select语句执行前」都会重新生成一个 Read View；
- 「可重复读」隔离级别是执行第一条select时，生成一个 Read View，然后整个事务期间都在用这个 Read View。

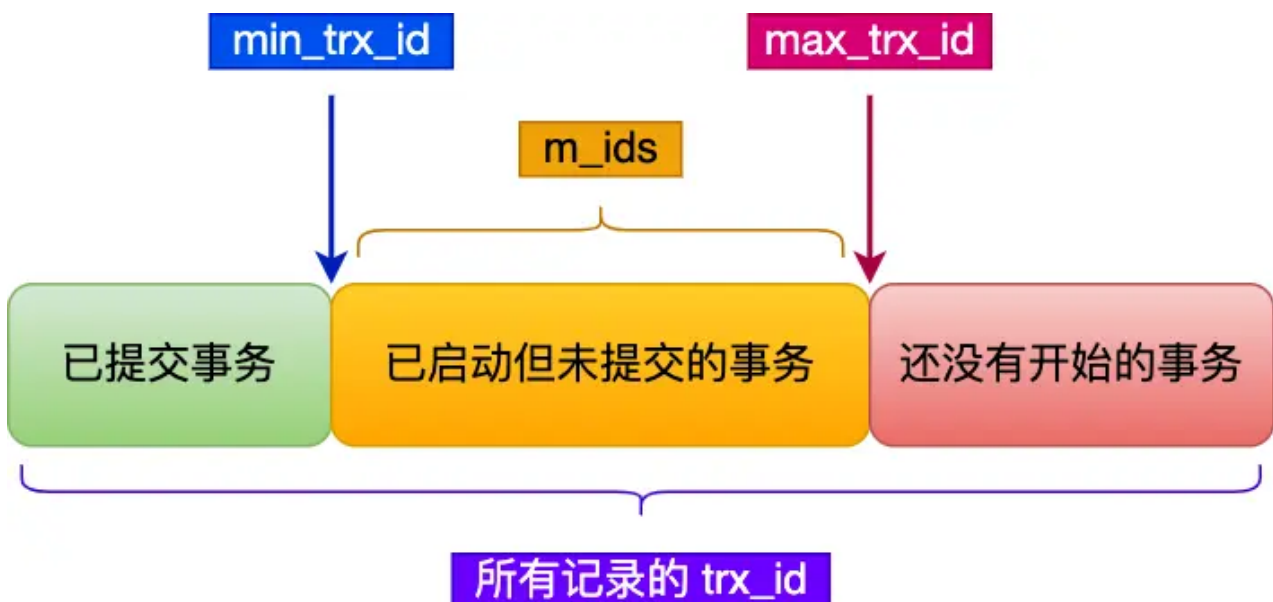
Read View 有四个重要的字段：

- `m_ids`：指的是在创建 Read View 时，当前数据库中「活跃事务」的**事务 id 列表**，注意是一个列表，“活跃事务”指的就是，启动了但还没提交的事务。
- `min_trx_id`：指的是在创建 Read View 时，当前数据库中「活跃事务」中事务 id 最小的事务，也就是 `m_ids` 的最小值。
- `max_trx_id`：这个并不是 `m_ids` 的最大值，而是创建 **Read View** 时当前数据库中应该给下一个事务的 **id 值**，也就是全局事务中最大的事务 id 值 + 1；
- `creator_trx_id`：指的是创建该 **Read View** 的事务的事务 id。

对于使用 InnoDB 存储引擎的数据库表，它的聚簇索引记录中都包含下面两个隐藏列：

- `trx_id`，当一个事务对某条聚簇索引记录进行改动时，就会把该事务的事务 id 记录在 `trx_id` 隐藏列里；
- `roll_pointer`，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 undo 日志中，然后这个隐藏列是个指针，指向每一个旧版本记录，于是就可以通过它找到修改前的记录。

在创建 Read View 后，我们可以将记录中的 `trx_id` 划分这三种情况：



img

一个事务去访问记录的时候，除了自己的更新记录总是可见之外，还有这几种情况：

- 如果记录的 `trx_id` 值小于 Read View 中的 `min_trx_id` 值，表示这个版本的记录是在创建 Read View 前已经提交的事务生成的，所以该版本的记录对当前事务可见。

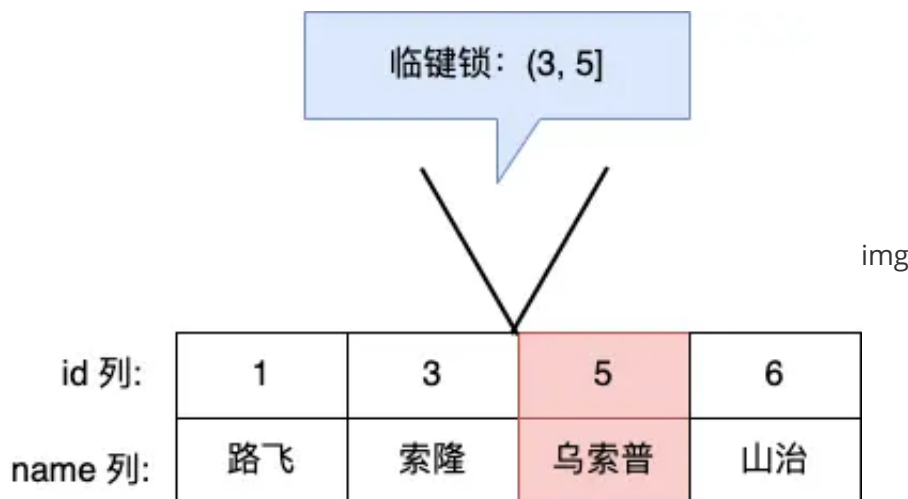
- 如果记录的 `trx_id` 值大于等于 Read View 中的 `max_trx_id` 值，表示这个版本的记录是在创建 Read View 后才启动的事务生成的，所以该版本的记录对当前事务不可见。
- 如果记录的 `trx_id` 值在 Read View 的 `min_trx_id` 和 `max_trx_id` 之间，需要判断 `trx_id` 是否在 `m_ids` 列表中：
 - 如果记录的 `trx_id` 在 `m_ids` 列表中，表示生成该版本记录的活跃事务依然活跃着（还没提交事务），所以该版本的记录对当前事务不可见。
 - 如果记录的 `trx_id` 不在 `m_ids` 列表中，表示生成该版本记录的活跃事务已经被提交，所以该版本的记录对当前事务可见。

这种通过「版本链」来控制并发事务访问同一个记录时的行为就叫 MVCC（多版本并发控制）。

next Key是什么，怎么实现？

Next-Key Lock 称为临键锁，是 Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。

假设，表中有一个范围 `id` 为 (3, 5] 的 next-key lock，那么其他事务即不能插入 `id = 4` 记录，也不能修改 `id = 5` 这条记录。



所以，next-key lock 即能保护该记录，又能阻止其他事务将新纪录插入到被保护记录前面的间隙中。

索引失效的场景有哪些，你知道什么改进方法吗？

- 当我们使用左或者左右模糊匹配的时候，也就是 `like %xx` 或者 `like %xx%` 这两种方式都会造成索引失效；
- 当我们在查询条件中对索引列使用函数，就会导致索引失效。
- 当我们在查询条件中对索引列进行表达式计算，也是无法走索引的。
- MySQL 在遇到字符串和数字比较的时候，会自动把字符串转为数字，然后再进行比较。如果字符串是索引列，而条件语句中的输入参数是数字的话，那么索引列会发生隐式类型转换，由于隐式类型转换是通过 CAST 函数实现的，等同于对索引列使用了函数，所以就会导致索引失效。
- 联合索引要能正确使用需要遵循最左匹配原则，也就是按照最左优先的方式进行索引的匹配，否则就会导致索引失效。
- 在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。

提交事务的一整个过程，每个日志都是怎么工作的？

具体更新一条记录 `UPDATE t_user SET name = 'xiaolin' WHERE id = 1;` 的流程如下：

1. 执行器负责具体执行，会调用存储引擎的接口，通过主键索引树搜索获取 `id = 1` 这一行记录：
 - 如果 `id=1` 这一行所在的数据页本来就在 `buffer pool` 中，就直接返回给执行器更新；
 - 如果记录不在 `buffer pool`，将数据页从磁盘读入到 `buffer pool`，返回记录给执行器。
2. 执行器得到聚簇索引记录后，会看一下更新前的记录和更新后的记录是否一样：
 - 如果一样的话就不进行后续更新流程；
 - 如果不一样的话就把更新前的记录和更新后的记录都当作参数传给 InnoDB 层，让 InnoDB 真正的执行更新记录的操作；
3. 开启事务，InnoDB 层更新记录前，首先要记录相应的 `undo log`，因为这是更新操作，需要把被更新的列的旧值记下来，也就是要生成一条 `undo log`，`undo log` 会写入 `Buffer Pool` 中的 `Undo` 页面，不过在内存修改该 `Undo` 页面后，需要记录对应的 `redo log`。
4. InnoDB 层开始更新记录，会先更新内存（同时标记为脏页），然后将记录写到 `redo log` 里面，这个时候更新就算完成了。为了减少磁盘 I/O，不会立即将脏页写入磁盘，后续由后台线程选择一个合适的时机将脏页写入到磁盘。这就是 **WAL 技术**，MySQL 的写操作并不是立刻写到磁盘上，而是先写 `redo` 日志，然后在合适的时间再将修改的行数据写到磁盘上。
5. 至此，一条记录更新完了。
6. 在一条更新语句执行完成后，然后开始记录该语句对应的 `binlog`，此时记录的 `binlog` 会被保存到 `binlog cache`，并没有刷新到硬盘上的 `binlog` 文件，在事务提交时才会统一将该事务运行过程中的所有 `binlog` 刷新到硬盘。
7. 事务提交（为了方便说明，这里不说组提交的过程，只说两阶段提交）：
 - **prepare 阶段**：将 `redo log` 对应的事务状态设置为 `prepare`，然后将 `redo log` 刷新到硬盘；
 - **commit 阶段**：将 `binlog` 刷新到磁盘，接着调用引擎的提交事务接口，将 `redo log` 状态设置为 `commit`（将事务设置为 `commit` 状态后，刷入到磁盘 `redo log` 文件）；
8. 至此，一条更新语句执行完成。

Java

JVM内存区域 每一个区域的内容？

JVM的内存结构主要分为以下几个部分：

- 程序计数器（Program Counter Register）：每个线程都有一个程序计数器。当线程执行 Java 方法时，程序计数器保存当前执行指令的地址，以便在 JVM 调用其他方法或恢复线程执行时重新回到正确的位置。
- Java 虚拟机栈（Java Virtual Machine Stacks）：每个线程都有一个虚拟机栈。虚拟机栈保存着方法执行期间的局部变量、操作数栈、方法出口等信息。线程每调用一个 Java 方法时，会创建一个栈帧（Stack Frame），栈帧包含着该方法的局部变量、操作数栈、方法返回地址等信息。栈帧在方法执行结束后会被弹出。
- 本地方法栈（Native Method Stack）：与 Java 虚拟机栈类似，但是为本地方法服务。
- Java 堆（Java Heap）：Java 堆是 Java 虚拟机中最大的一块内存区域，用于存储各种类型的对象实

例，也是垃圾收集器的主要工作区域。Java 堆是所有线程共享的部分。

- 方法区（Method Area）：方法区也是所有线程共享的部分，它用于存储类的加载信息、静态变量、常量池、方法字节码等数据。在 Java 8 及以前的版本中，方法区被实现为永久代（Permanent Generation），在 Java 8 中被改为元空间（Metaspace）。

JVM异常问题？

- StackOverflowError（线程请求栈深度超过虚拟机所允许的最大深度）
- OutOfMemoryError（堆内存不够用）
- PermGen space（方法区内存不够用）。

String保存在哪里呢

String 保存在字符串常量池中，不同于其他对象，它的值是不可变的，且可以被多个引用共享。

java版本改动问题 1.7 1.8有哪些主要区别？

- Java 7 新特性：钻石操作符、try-with-resource语句、支持动态类型语言、Fork/Join框架等。
- Java 8 新特性：Lambda 表达式、Stream API、新的 Date/Time API、Nashorn JavaScript 引擎等。

怎么找到需要回收的垃圾？

Java中的垃圾回收机制通过判断对象是否可达来确定哪些对象可以被回收。当一个对象没有任何引用指向它时，它就可以被回收，这个过程由JVM的垃圾回收器自动完成。

JVM使用可达性分析算法来判断对象是否可达。从GC Roots对象开始，通过一系列的引用链来遍历所有的对象，如果一个对象不可达，则说明它已经死亡，可以被回收了。

在Java中，有4种引用类型：强引用、软引用、弱引用和虚引用。其中，强引用是最常见的引用类型，只要强引用存在，垃圾回收器就不会回收该对象。软引用、弱引用和虚引用则分别表示对对象的软引用、弱引用和虚引用，当垃圾回收器进行垃圾回收时，会根据不同的引用类型来决定是否回收对象。

有哪些常见的垃圾回收器，举几个说说？

1. Serial收集器：单线程的垃圾回收器，使用标记-复制算法，适合小型应用程序或客户端应用程序。
2. Parallel收集器：多线程的垃圾回收器，使用标记-复制算法，适合在后台运行的中型应用程序。
3. CMS收集器：并发垃圾回收器，使用标记-清除算法，适合对响应时间有要求的中型应用程序。
4. G1收集器：并发垃圾回收器，使用标记-整理算法，适合对响应时间有要求且堆内存较大的应用程序。

其中，Serial收集器和Parallel收集器是新生代收集器，CMS和G1是老年代收集器。

HashMap底层怎么实现的？线程安全吗？

HashMap底层是基于数组和链表实现的。简单来说，HashMap将key通过hash算法映射到数组中，然后在对应的链表中查找value。当多个key的hash值相同时，会在同一个数组位置上使用链表来存储这些key-value。但是，当链表长度太长时，会影响HashMap的性能，因此在JDK1.8中，当链表长度超过阈值时，会将链表转换为红黑树，以提高查找效率。

HashMap不是线程安全的，因为多个线程同时访问HashMap时可能会导致数据不一致的问题。可以使用ConcurrentHashMap来实现线程安全的Map。

红黑树是什么？

红黑树是一种自平衡的二叉查找树，可以保证在最坏情况下基本动态操作的时间复杂度为 $O(\log n)$ 。红黑树中的每个节点都有一个颜色属性，可以是红色或黑色。红黑树满足以下5个性质：

1. 每个节点要么是红色，要么是黑色。
2. 根节点是黑色的。
3. 每个叶子节点（NIL节点，空节点）是黑色的。
4. 如果一个节点是红色的，则它的两个子节点都是黑色的。
5. 对于每个节点，从该节点到其所有后代叶子节点的简单路径上，均包含相同数目的黑色节点。

通过这些性质，红黑树可以保证在插入和删除节点时，自动调整树的结构，以保持树的平衡和性质的满足。相比于普通的二叉查找树，红黑树的平衡性更好，查找、插入和删除都具有更稳定的时间复杂度，因此在很多场景下被广泛应用。

什么是公平锁和非公平锁？

公平锁和非公平锁是针对锁的获取方式而言的。

公平锁是指多个线程按照申请锁的顺序来获取锁，即先到先得的原则。当线程A释放锁后，线程B、C、D依次获取锁，如果此时线程E申请锁，则它需要等待B、C、D依次获取到锁并释放锁后才能获取锁。

非公平锁是指多个线程获取锁的顺序是随机的，不保证公平性。当线程A释放锁后，线程B、C、D等线程都可以竞争获取到锁，而此时线程E也可以通过竞争获取到锁。

在实际应用中，公平锁可以避免饥饿现象，但是由于需要维护线程队列，因此效率相对较低。而非公平锁由于不需要维护线程队列，因此效率相对较高，但是可能会导致某些线程长时间无法获取锁。

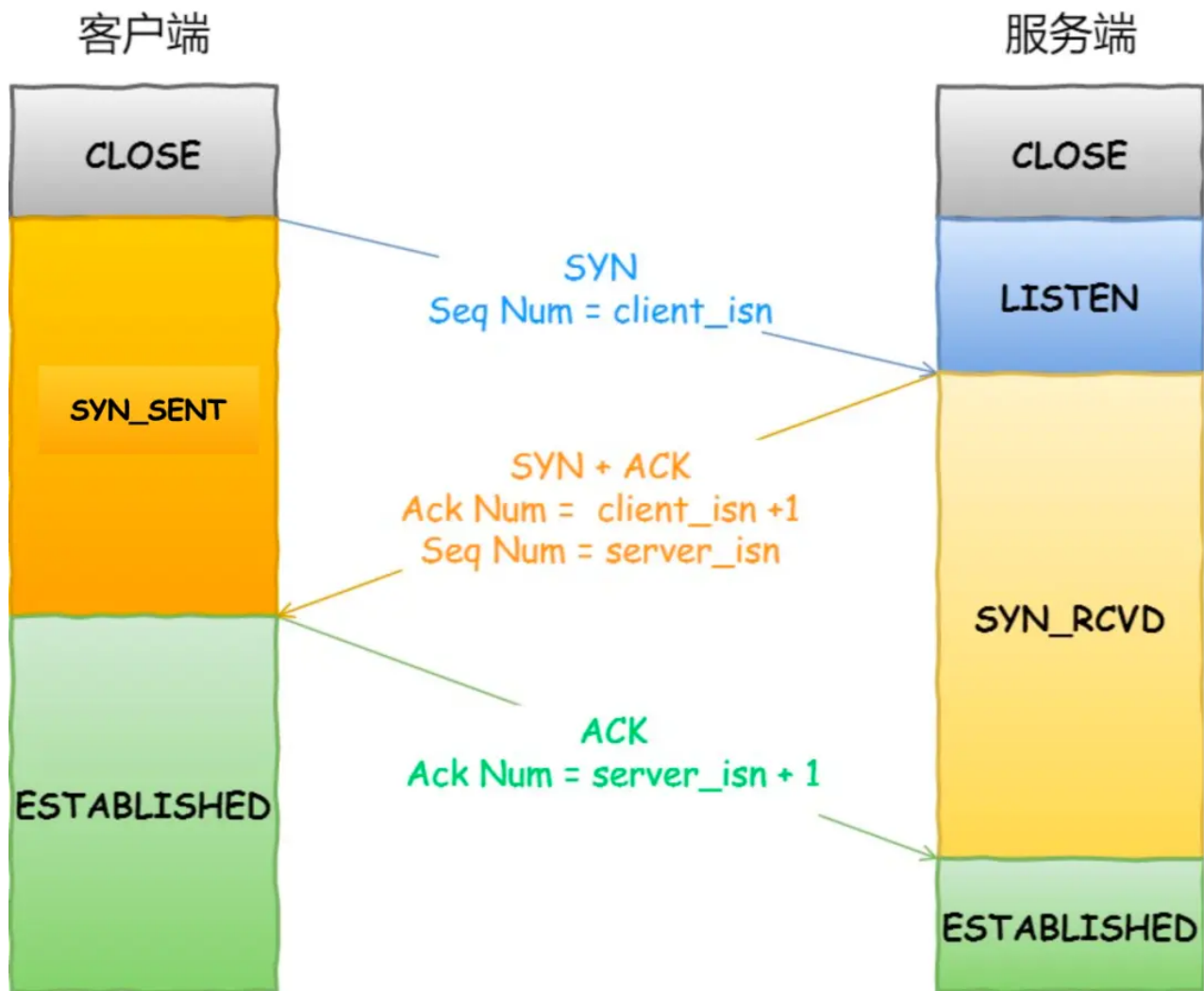
ThreadLocal 是什么？

ThreadLocal是Java中的一个线程封闭技术，它可以让每个线程都拥有自己单独的变量副本，从而保证线程安全。ThreadLocal提供了一种线程本地存储的机制，为每个线程提供一个独立的变量副本，使得每个线程中的变量互不干扰。

计算机网络

TCP 三次握手、四次挥手的過程？

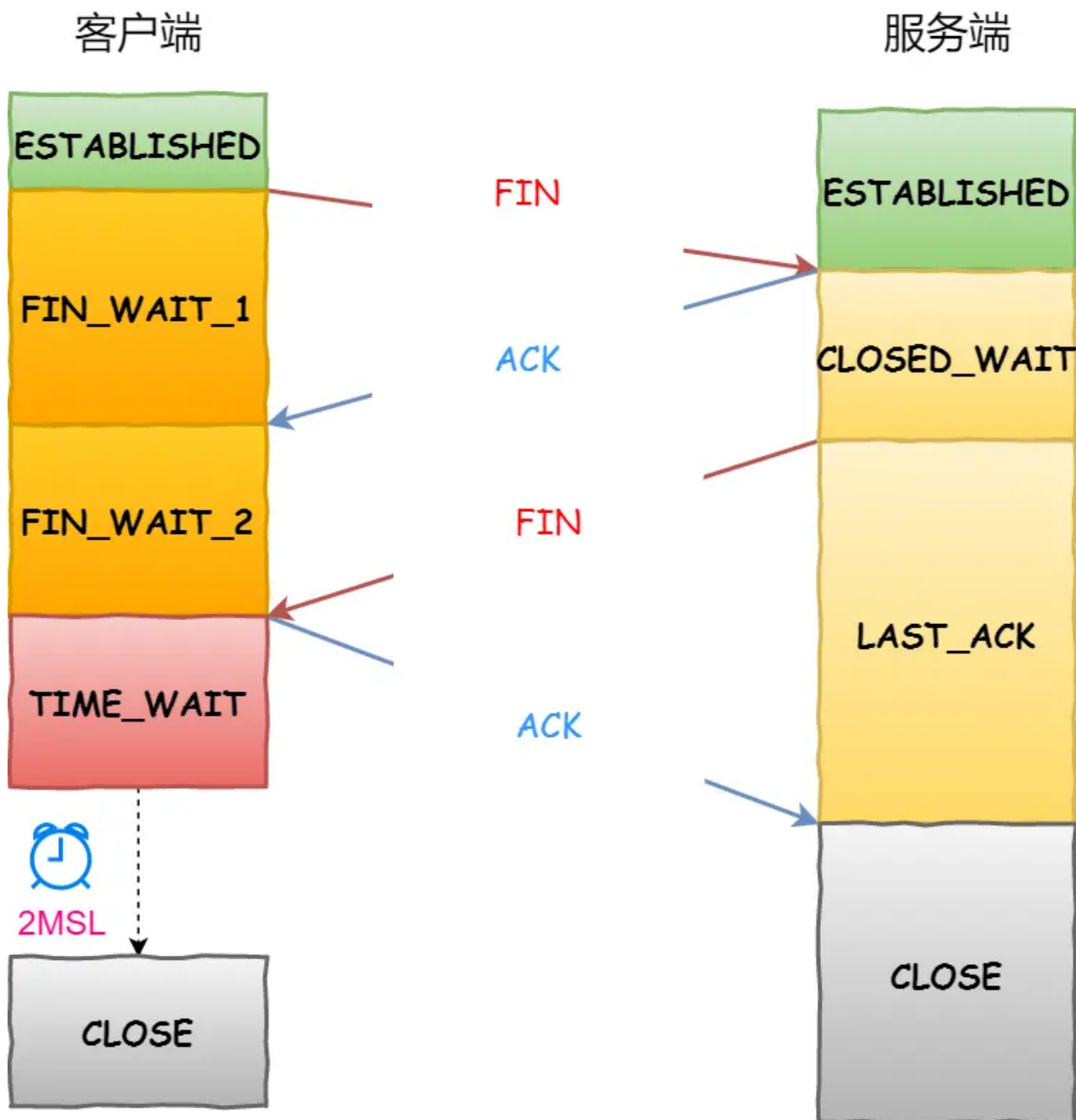
三次握手的过程：



TCP 三次握手

- 一开始，客户端和服务端都处于 **CLOSE** 状态。先是服务端主动监听某个端口，处于 **LISTEN** 状态
- 客户端会随机初始化序号 (**client_isn**)，将此序号置于 TCP 首部的「序号」字段中，同时把 **SYN** 标志位置为 **1**，表示 **SYN** 报文。接着把第一个 SYN 报文发送给服务端，表示向服务端发起连接，该报文不包含应用层数据，之后客户端处于 **SYN-SENT** 状态。
- 服务端收到客户端的 **SYN** 报文后，首先服务端也随机初始化自己的序号 (**server_isn**)，将此序号填入 TCP 首部的「序号」字段中，其次把 TCP 首部的「确认应答号」字段填入 **client_isn + 1**，接着把 **SYN** 和 **ACK** 标志位置为 **1**。最后把该报文发给客户端，该报文也不包含应用层数据，之后服务端处于 **SYN-RCVD** 状态。
- 客户端收到服务端报文后，还要向服务端回应最后一个应答报文，首先该应答报文 TCP 首部 **ACK** 标志位置为 **1**，其次「确认应答号」字段填入 **server_isn + 1**，最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 **ESTABLISHED** 状态。
- 服务端收到客户端的应答报文后，也进入 **ESTABLISHED** 状态。

四次挥手的過程：



- 客户端打算关闭连接，此时会发送一个 TCP 首部 **FIN** 标志位被置为 1 的报文，也即 **FIN** 报文，之后客户端进入 **FIN_WAIT_1** 状态。
- 服务端收到该报文后，就向客户端发送 **ACK** 应答报文，接着服务端进入 **CLOSE_WAIT** 状态。
- 客户端收到服务端的 **ACK** 应答报文后，之后进入 **FIN_WAIT_2** 状态。
- 等待服务端处理完数据后，也向客户端发送 **FIN** 报文，之后服务端进入 **LAST_ACK** 状态。
- 客户端收到服务端的 **FIN** 报文后，回一个 **ACK** 应答报文，之后进入 **TIME_WAIT** 状态。
- 服务端收到了 **ACK** 应答报文后，就进入了 **CLOSE** 状态，至此服务端已经完成连接的关闭。
- 客户端在经过 **2MSL** 一段时间后，自动进入 **CLOSE** 状态，至此客户端也完成连接的关闭。

为什么要三次握手、四次挥手？

三次握手的原因：

- 三次握手可以阻止重复历史连接的初始化
- 三次握手可以同步双方的初始序列号
- 三次握手可以避免资源浪费

四次挥手的原因：

- 服务端通常需要等待完成数据的发送和处理，所以服务端的 ACK 和 FIN 一般都会分开发送，因此是需要四次挥手。

LINUX有哪些IO机制？

- 阻塞IO（Blocking IO）：应用程序在进行IO操作时，会一直阻塞等待IO完成，期间无法进行其他操作。
- 非阻塞IO（Non-blocking IO）：应用程序在进行IO操作时，会立即返回，无论IO操作是否完成，应用程序都可以进行其他操作。需要通过轮询的方式来判断IO是否完成，因此效率较低。
- IO多路复用（IO Multiplexing）：通过select、poll、epoll等系统调用，在一个进程中可以同时监控多个文件描述符，当有任何一个文件描述符就绪时，就可以进行IO操作。
- 信号驱动IO（Signal Driven IO）：应用程序在进行IO操作时，向内核注册一个信号处理函数，内核在IO完成时会向应用程序发送一个信号，应用程序收到信号后再进行数据处理。
- 异步IO（Asynchronous IO）：应用程序进行IO操作时，可以立即返回，内核负责将数据读取到指定的缓冲区中，并在完成后通知应用程序，应用程序可以继续其他操作。异步IO需要操作系统和硬件的支持，目前主要应用于高性能IO场景。

select poll epoll，底层实现有什么区别？

select 和 poll 并没有本质区别，它们内部都是使用「线性结构」来存储进程关注的 Socket 集合。

在使用的时候，首先需要把关注的 Socket 集合通过 select/poll 系统调用从用户态拷贝到内核态，然后由内核检测事件，当有网络事件产生时，内核需要遍历进程关注 Socket 集合，找到对应的 Socket，并设置其状态为可读/可写，然后把整个 Socket 集合从内核态拷贝到用户态，用户态还要继续遍历整个 Socket 集合找到可读/可写的 Socket，然后对其处理。

很明显发现，select 和 poll 的缺陷在于，当客户端越多，也就是 Socket 集合越大，Socket 集合的遍历和拷贝会带来很大的开销，因此也很难应对 C10K。

epoll 是解决 C10K 问题的利器，通过两个方面解决了 select/poll 的问题。

- epoll 在内核里使用「红黑树」来关注进程所有待检测的 Socket，红黑树是个高效的数据结构，增删改一般时间复杂度是 $O(\log n)$ ，通过对这棵黑红树的管理，不需要像 select/poll 在每次操作时都传入整个 Socket 集合，减少了内核和用户空间大量的数据拷贝和内存分配。
- epoll 使用事件驱动的机制，内核里维护了一个「链表」来记录就绪事件，只将有事件发生的 Socket 集合传递给应用程序，不需要像 select/poll 那样轮询扫描整个集合（包含有和无事件的 Socket），大大提高了检测的效率。

BIO 和 NIO 的区别?

Java中，BIO和NIO都是IO模型，它们的主要区别在于：

- 阻塞和非阻塞：BIO采用阻塞模式，即在进行IO操作时，线程会一直阻塞等待IO完成。而NIO采用非阻塞模式，即在进行IO操作时，线程会立即返回，无论IO操作是否完成，线程都可以进行其他操作。
- IO模型：BIO采用同步阻塞IO模型，即一个线程只能处理一个连接，当有大量连接时，需要大量的线程来处理，会导致系统资源浪费。NIO采用同步非阻塞IO模型，将客户端发送的连接请求都会注册到多路复用器上，这样一个线程可以处理多个连接，当有大量连接时，只需要少量的线程来处理，可以有效地提高系统资源利用率。

算法

链表判断相交

```
public ListNode getIntersectionNode(ListNode headA, ListNode headB) {  
    if (headA == null || headB == null) return null;  
    ListNode pA = headA, pB = headB;  
    while (pA != pB) {  
        pA = pA == null ? headB : pA.next;  
        pB = pB == null ? headA : pB.next;  
    }  
    return pA;  
}
```

面试总结

感觉

非常有收获的一次面试，值得之后自己单独写一写文章去记录一下。基础不牢地动山摇，虽然都是基础问题，但是往下深挖根茎叶脉全部相连，这些问题面试题里面都有解，但是自己真的是只知道表面，浅浅看个大概就上战场了，还有就是非常感谢面试官，反问环节给了我很多建议，包括面试、策略、基础、算法等等，是一次宝贵的学习经历，说不遗憾是假的，但我很开心，有所收获就好。

不足之处

基础不够扎实，很多只知其表不知其里，而且面试官经验丰富，非常敏锐，当时压力也很大，被问倒了心态也不稳，有的可以答出来的也没想起来怎么说。

没有几个答的特别好的，稍微说一点就会被问深直到不会，这里就不贴当时回答了，说的都很浅，下来每一个问题都要仔细研究，要将知识连接成网。

自己给自己构造一个场景，顺着把用到的技术全部梳理一遍，能讲出这个整体过程，有头有尾，才能真的理解，比如hashmap插入数据的过程、threadlocal创建释放的过程、String怎么实现的字符串拼接、SpringBoot框架搭建的每一步等等。

阿里问的相当基础！

大家好，我是小林。

今天分享一篇一位同学暑期实习面试阿里Java后端岗位的一面的面经。

主要拷打了项目+Java 集合+Java并发+网络+mysql，一场面试大概问了 20 个题目，问的还是比较基础，不算太难。

问题记录

自我介绍

balabala（略）

简历上有两个项目，选一个你比较熟悉的介绍

balabala（略）

项目用到了哪个线程实现类？

用了ScheduledThreadPool这个线程实现类

为什么要使用这个线程类？

这个实现类可以设置定期的执行任务，它支持定时或周期性执行任务，比如每隔 10 秒钟执行一次任务，我通过这个实现类设置定期执行任务的策略。

你还了解别的线程实现类吗？

除了这个之外，还有就是newSingleThreadExecutor，别的就不太熟悉了。

小林补充：

除了 ScheduledThreadPool 线程池之外，还有 4 种常见的线程池如下：

- FixedThreadPool：它的核心线程数和最大线程数是一样的，所以可以把它看作是固定线程数的线程池，它的特点是线程池中的线程数除了初始阶段需要从 0 开始增加外，之后的线程数量就是固定的，就算任务数超过线程数，线程池也不会再创建更多的线程来处理任务，而是会把超出线程处理能力的任务放到任务队列中进行等待。而且就算任务队列满了，到了本该继续增加线程数的时候，由于它的最大线程数和核心线程数是一样的，所以也无法再增加新的线程了。
- CachedThreadPool：可以称作可缓存线程池，它的特点在于线程数是几乎可以无限增加的（实际最大可以达到 Integer.MAX_VALUE，为 $2^{31}-1$ ，这个数非常大，所以基本不可能达到），而当线程闲置时还可以对线程进行回收。也就是说该线程池的线程数量不是固定不变的，当然它也有一个用于存储提交任务的队列，但这个队列是 SynchronousQueue，队列的容量为0，实际不存储任何任务，它只负责对任务进行中转和传递，所以效率比较高。
- SingleThreadExecutor：它会使用唯一的线程去执行任务，原理和 FixedThreadPool 是一样的，只不过这里线程只有一个，如果线程在执行任务的过程中发生异常，线程池也会重新创建一个线程来执行后续的任务。这种线程池由于只有一个线程，所以非常适合用于所有任务都需要按被提交的顺序依次执行的场景，而前几种线程池不一定能够保障任务的执行顺序等于被提交的顺序，因为它们是多线程并行执行的。
- SingleThreadScheduledExecutor：它实际和 ScheduledThreadPool 线程池非常相似，它只是

ScheduledThreadPool 的一个特例，内部只有一个线程。

看你项目上有用到雪花算法，你为什么要使用雪花算法？

除了考虑使用主键自增保持主键的唯一性外，我就使用到了雪花算法，算出一个不会重复的数做为id，保证主键唯一。

那你还了解别的生成主键的策略吗，你觉得他们能代替雪花算法吗

除了主键自增和雪花算法，别的我暂时没了解的，但是只要是能保证主键唯一的主键生成策略都可以使用

小林补充：

- 除了雪花算法之外，还有很多优秀的互联网公司也提供了唯一 ID 生成的方案或框架，比如美团开源的 Leaf，百度开源的 UidGenerator 等等。
- UUID 虽然也可以保证唯一性，但是 UUID 的值是随机的，无序的，不太适合作为主键，因为随机插入，可能会引起页分裂的问题，从而影响查询性能。

List的实现类

ArrayList、Vector、LinkedList

小林补充：

Java中的List接口有多个实现类，常用的包括：

1. ArrayList：基于动态数组实现，优势在于支持随机访问和快速插入/删除元素，适用于频繁读取和遍历的场景。
2. LinkedList：基于双向链表实现，优势在于支持快速插入/删除元素，适用于频繁插入/删除元素的场景。
3. Vector：和ArrayList类似，但由于其线程安全性，适用于多线程环境。
4. Stack：基于Vector实现，是一个后进先出（LIFO）的数据结构，适用于需要按照后进先出顺序处理元素的场景。

List和Set的区别

- List是有序的，Set是无序的
- List可以存放相同的元素，Set不可以存放重复的元素

小林补充：

1. 顺序：List是有序的集合，它可以按照元素插入的顺序进行存储和访问。而Set是无序的集合，元素在集合中的位置是不固定的。
2. 重复元素：List允许存储重复的元素，即可以有多个相同的对象。Set不允许存储重复的元素，即每个对象在集合中只能出现一次。
3. 实现类：List的常用实现类有ArrayList和LinkedList，分别使用数组和链表作为底层数据结构。Set的常用实现类有HashSet、LinkedHashSet和TreeSet，分别基于哈希表、链表+哈希表和红黑树实现。
4. 性能：由于底层数据结构的差异，List和Set在增加、删除、查找等操作上的性能表现有所不同。例如，ArrayList在随机访问元素时性能较好，而LinkedList在插入和删除元素时性能较好。HashSet

在查找、添加和删除元素时性能较好，但不保证元素顺序。TreeSet在保持元素排序的同时，也能提供较好的查找性能。

针对你说的List和Set的性质，那你会用这两种结构解决哪些问题

对于取消重复数据的场景，选择set，对于只是保存数据、或者需要按存储顺序进行访问的场景使用List。

小林补充：

List（列表）适用于以下场景：

1. 有序数据：列表中的元素按照插入顺序存储，因此适用于需要保持元素顺序的场景。
2. 允许重复元素：列表允许存储重复的元素，因此适用于需要统计元素出现次数的场景。
3. 需要根据索引进行查找、插入和删除操作：列表允许通过索引值直接访问、插入或删除元素，适用于需要频繁进行这些操作的场景。

Set（集合）适用于以下场景：

1. 去重：集合中的元素不能重复，因此适用于去除数据中重复元素的场景。
2. 无需关心元素顺序：集合中的元素没有固定顺序，适用于元素顺序无关紧要的场景。
3. 快速判断元素是否存在：集合提供了高效率的查找算法，适用于需要快速判断某个元素是否存在于数据集中的场景。
4. 集合运算：集合支持交集、并集、差集等运算，适用于需要进行这些运算的场景。

常用的网络状态码有哪些

100开头是表示协议执行的中间状态，一般不常用，400开头的表示协议执行失败，例如404是指服务端找不到页面的请求地址，500是协议的完成（这个没记住答错了，面试官提示了，说那200表示什么）。

小林补充：

常用的网络状态码分为五类，分别是：

1xx（信息）：表示接收到请求，需要继续处理。

- 100 Continue：继续，客户端应继续其请求。

2xx（成功）：表示请求已成功被服务器接收、理解和接受。

- 200 OK：请求成功，请求所希望的响应头或数据体将随此响应返回。
- 201 Created：请求已成功，并因此创建了一个新的资源。
- 204 No Content：无内容，服务器成功处理，但未返回内容。

3xx（重定向）：需要后续操作才能完成这一请求。

- 301 Moved Permanently：永久重定向，请求的资源已被永久移动到新位置。
- 302 Found：临时重定向，请求的资源临时从不同位置响应。
- 304 Not Modified：资源未修改，使用缓存的资源。

4xx（客户端错误）：请求包含错误语法或无法完成。

- 400 Bad Request：客户端请求的语法错误，服务器无法理解。
- 401 Unauthorized：请求需要用户验证。

- 403 Forbidden: 服务器理解请求客户端的请求, 但是拒绝执行它。
- 404 Not Found: 请求的资源无法在服务器上找到。

5xx (服务器错误): 服务器未能实现合法的请求。

- 500 Internal Server Error: 服务器内部错误, 无法完成请求。
- 501 Not Implemented: 服务器不支持请求的功能。
- 503 Service Unavailable: 服务器暂不可用, 可能是服务器过载或停机维护。

ps: 《HTTP 常见面试题》完整详细讲解: https://xiaolincoding.com/network/2_http/http_interview.html

流量控制和拥塞控制的原理

- 流量控制是服务端和客户端协议的窗口实现, 在客户端发送数据的时候, 服务端返回窗口的容量, 客户端通过容量来调整发送信息的大小
- 拥塞控制是通过滑动窗口实现, 服务端只接收拥塞窗口大小内的数据, 客户端发送的报文丢失后, 没有收到服务端的确认信息, 就将没有收到确认信息的报文再发送。

小林补充:

TCP一种面向连接的、可靠的传输层协议。为了确保数据的有效传输, TCP 提供了两种重要的控制机制: 流量控制和拥塞控制。

流量控制 (Flow Control)

流量控制的主要目的是防止发送方向接收方发送过多数据, 导致接收方处理不过来。TCP 使用滑动窗口机制来实现流量控制。在 TCP 连接中, 接收方为每个连接分配一个接收缓冲区。接收方通过通知发送方自己的窗口大小, 告知发送方可以发送的数据量。窗口大小表示接收方当前能接受的数据字节数。

滑动窗口的工作原理如下:

- 发送方根据接收方的窗口大小来确定发送的数据量。
- 当接收方收到数据后, 发送确认报文, 同时更新窗口大小。
- 发送方收到确认报文后, 更新已发送但未确认的数据量, 并根据新的窗口大小调整发送速率。

拥塞控制 (Congestion Control)

拥塞控制的目的是防止过多的数据进入网络, 导致网络拥塞。TCP 使用四种算法来实现拥塞控制: 慢开始、拥塞避免、快速重传和快速恢复。

- 慢开始: 发送方初始拥塞窗口设置为一个较小的值。随后, 每收到一个确认报文, 拥塞窗口大小加倍。这样, 发送速率会以指数形式增长, 直到达到一个阈值 (sssthresh)。
- 拥塞避免: 当拥塞窗口到达阈值后, 发送方转入拥塞避免阶段, 窗口大小每经过一个往返时间 (RTT) 就增加1。这样, 拥塞窗口的大小呈线性增长, 避免了网络拥塞。
- 快速重传: 当发送方连续收到三个重复的确认报文, 表示可能有一个数据包丢失。此时, 发送方立即重传丢失的数据包, 而不是等待超时重传。
- 快速恢复: 快速重传后, 发送方降低拥塞窗口阈值, 然后进入拥塞避免阶段。这样可以在丢包后尽快恢复传输速率。

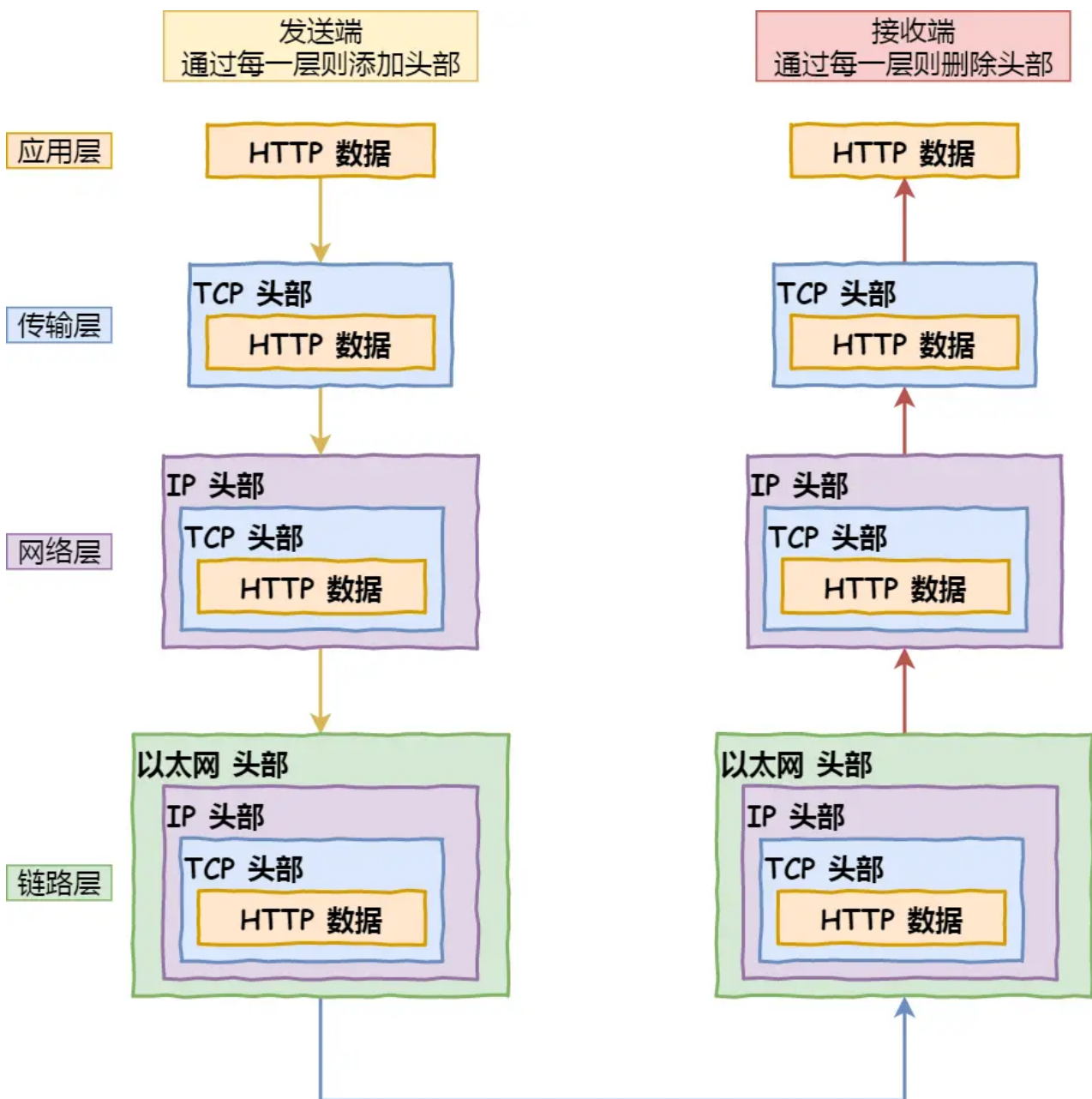
通过这两种控制机制, TCP 能确保在各种网络条件下有效、可靠地传输数据。

ps: 《TCP 重传、滑动窗口、流量控制、拥塞控制》完整详细讲解: https://xiaolincoding.com/network/3_tcp/tcp_feature.html

一条url请求页面的执行过程

(这块我答的不太好, 忘了dns服务器的名字) 浏览器先解析url地址, 然后生成http消息, 生成的消息需要知道ip地址才能发送, 就先去浏览器的缓存中查询, 没有的话查看操作系统的缓存, 如果还是没有就在本地dns中查询, 本地dns查询不到后会先访问根dns, 根dns查询的是存放这个ip的二级dns服务器(忘了名字), 二级dns服务器会向对应的权威dns服务器查询, 权威dns服务器会向本地返回ip地址, 然后本地通过这个ip地址和请求的服务端建立起tcp连接, 服务端向本地发送请求的资源。

小林补充:



ps: 《键入网址到网页显示, 期间发生了什么?》完整详细讲解: https://xiaolincoding.com/network/1_base/what_happen_url.html

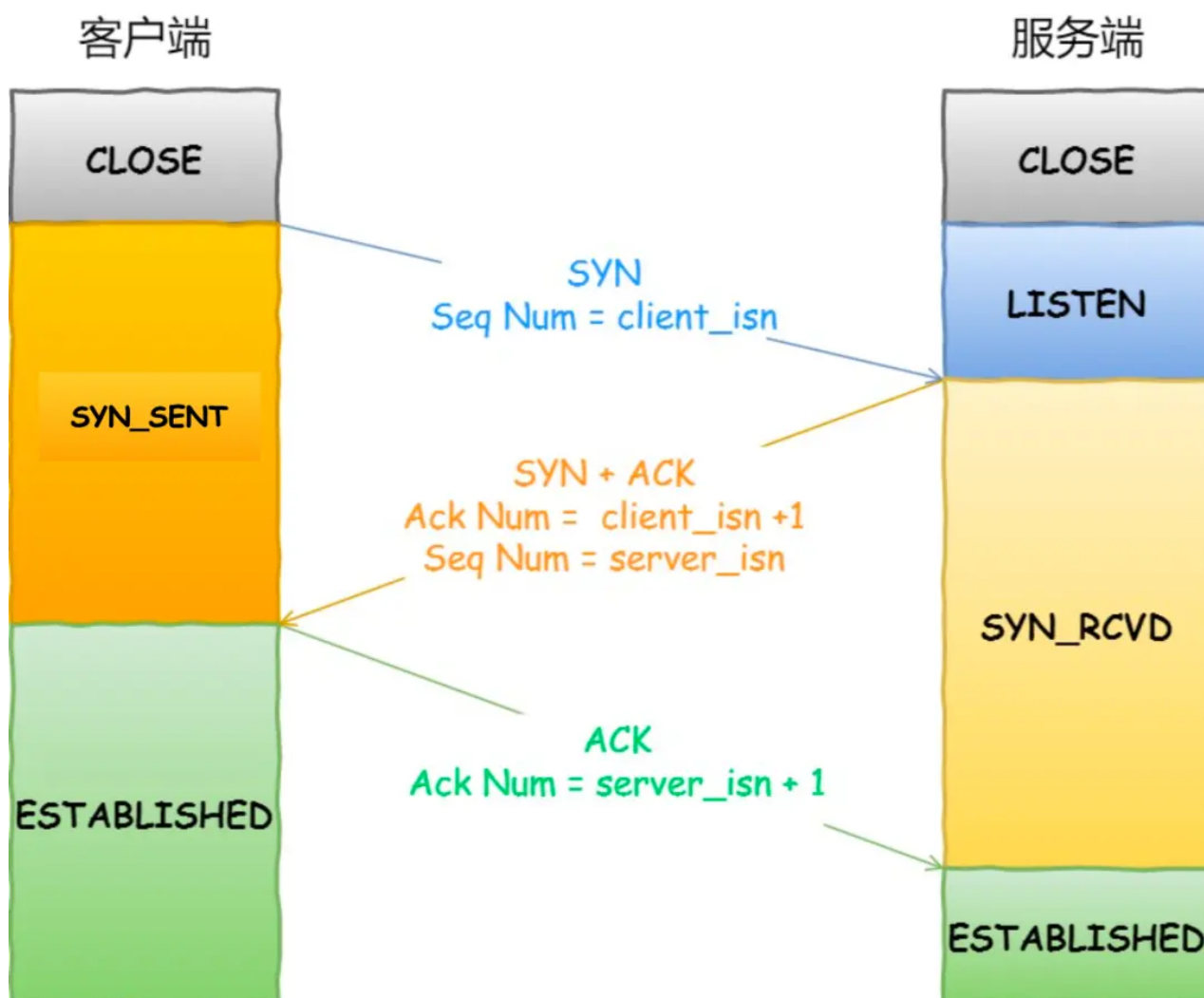
TCP是如何建立连接的

三次握手

- 1、客户端发送请求建立连接报文，报文的SYN=1
- 2、服务端收到后，发送连接报文，报文的SYN=1，并且发送一个序号字段
- 3、客户端收到后报文后，客户端到服务端的连接已经建立，客户端发送报文对上一个报文的序号端进行确认

小林补充：

TCP 是面向连接的协议，所以使用 TCP 前必须先建立连接，而建立连接是通过三次握手来进行的。三次握手的过程如下图：



TCP 三次握手

- 一开始，客户端和服务端都处于 **CLOSE** 状态。先是服务端主动监听某个端口，处于 **LISTEN** 状态
- 客户端会随机初始化序号 (`client_isn`)，将此序号置于 TCP 首部的「序号」字段中，同时把 **SYN** 标志位置为 **1**，表示 **SYN** 报文。接着把第一个 SYN 报文发送给服务端，表示向服务端发起连接，该报文不包含应用层数据，之后客户端处于 **SYN-SENT** 状态。
- 服务端收到客户端的 **SYN** 报文后，首先服务端也随机初始化自己的序号 (`server_isn`)，将此序号填入 TCP 首部的「序号」字段中，其次把 TCP 首部的「确认应答号」字段填入 `client_isn + 1`，接着把 **SYN** 和 **ACK** 标志位置为 **1**。最后把该报文发给客户端，该报文也不包含应用层数

据，之后服务端处于 `SYN-RCVD` 状态。

- 客户端收到服务端报文后，还要向服务端回应最后一个应答报文，首先该应答报文 TCP 首部 `ACK` 标志位置为 `1`，其次「确认应答号」字段填入 `server_isn + 1`，最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 `ESTABLISHED` 状态。
- 服务端收到客户端的应答报文后，也进入 `ESTABLISHED` 状态。

ps：《TCP 三次握手与四次挥手面试题》完整详细讲解：https://xiaolincoding.com/network/3_tcp/tcp_interview.html

http和https的区别

- 1、https是需要通过CA申请才能获得，所以数量是比较少的
- 2、http发送的报文是明文，所以不安全，https在传输层之上加了ssl协议

小林补充：

- HTTP是一种用于传输超文本的协议，数据传输是明文的，不具备加密和安全性。HTTP使用的端口号是 80
- HTTPS是在HTTP的基础上加入了SSL/TLS协议进行加密和身份验证的安全版本。它使用加密的SSL/TLS协议进行数据传输，保证了数据的机密性和完整性。HTTPS使用的端口号是443。

ps：《HTTP 常见面试题》完整详细讲解：https://xiaolincoding.com/network/2_http/http_interview.html

数据库的索引

B+树索引，hash索引、全文索引

B+树索引的话是innodb采用的索引，索引的叶子节点上是数据，非叶子节点是索引信息

hash索引单个的查找效率很高

为什么采用B+树索引，它有什么优点

这里我将B+树和B树、红黑树做了比较。

B+树相对于B树，只有叶子节点存储的是数据信息，非叶子节点都是索引信息，所以在查找时加载到内存中的数据少，B+树的增删相对于B树来说比较稳定，不会发生频繁的父子结点替换，B+树的叶子结点是连接的，所以很容易实现范围查询

B+树相对于红黑树，首先B+树的层高比较小，意味着读取数据时IO磁盘的次数比较少，红黑树增删结点时需要保持子树的稳定性，增删的效率很低，B+树更容易实现范围查询。

小林补充：

树的高度决定于磁盘 I/O 操作的次数，因为树是存储在磁盘中的，访问每个节点，都对应一次磁盘 I/O 操作，也就是说树的高度就等于每次查询数据时磁盘 IO 操作的次数，所以树的高度越高，就会影响查询性能。

B 树和 B+ 都是通过多叉树的方式，会将树的高度变矮，所以这两个数据结构非常适合检索存于磁盘中的数据。

但是 MySQL 默认的存储引擎 InnoDB 采用的是 B+ 作为索引的数据结构，原因有：

- B+ 树的非叶子节点不存放实际的记录数据，仅存放索引，因此数据量相同的情况下，相比存储即存索引又存记录的 B 树，B+ 树的非叶子节点可以存放更多的索引，因此 B+ 树可以比 B 树更「矮胖」，查询底层节点的磁盘 I/O 次数会更少。
- B+ 树有大量的冗余节点（所有非叶子节点都是冗余索引），这些冗余索引让 B+ 树在插入、删除的效率都更高，比如删除根节点的时候，不会像 B 树那样会发生复杂的树的变化；
- B+ 树叶节点之间用链表连接了起来，有利于范围查询，而 B 树要实现范围查询，因此只能通过树的遍历来完成范围查询，这会涉及多个节点的磁盘 I/O 操作，范围查询效率不如 B+ 树。

ps：《为什么 MySQL 采用 B+ 树作为索引？》完整详细讲解：https://xiaolincoding.com/mysql/index/why_index_chose_bpuls_tree.html

数据库中事务可能带来的问题

有脏读、不可重复读、幻读三个问题：

- 脏读：一个事务读取另一个事务没有提交的数据
- 不可重复读：一个事务重复读取一条数据时发现读取到的数据不相同
- 幻读：一个事务后读取的数据相比之前读取的数据中多了一些数据

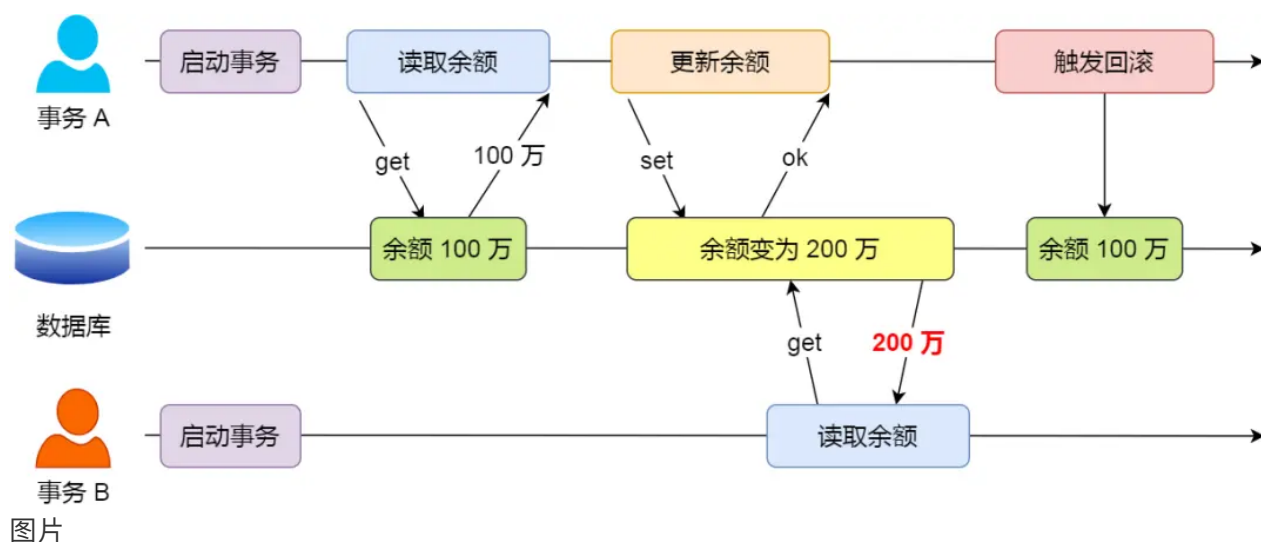
小林补充：

MySQL 服务端是允许多个客户端连接的，这意味着 MySQL 会出现同时处理多个事务的情况。

那么在同时处理多个事务的时候，就可能出现脏读（dirty read）、不可重复读（non-repeatable read）、幻读（phantom read）的问题。

1、脏读：如果一个事务「读到」了另一个「未提交事务修改过的数据」，就意味着发生了「脏读」现象。

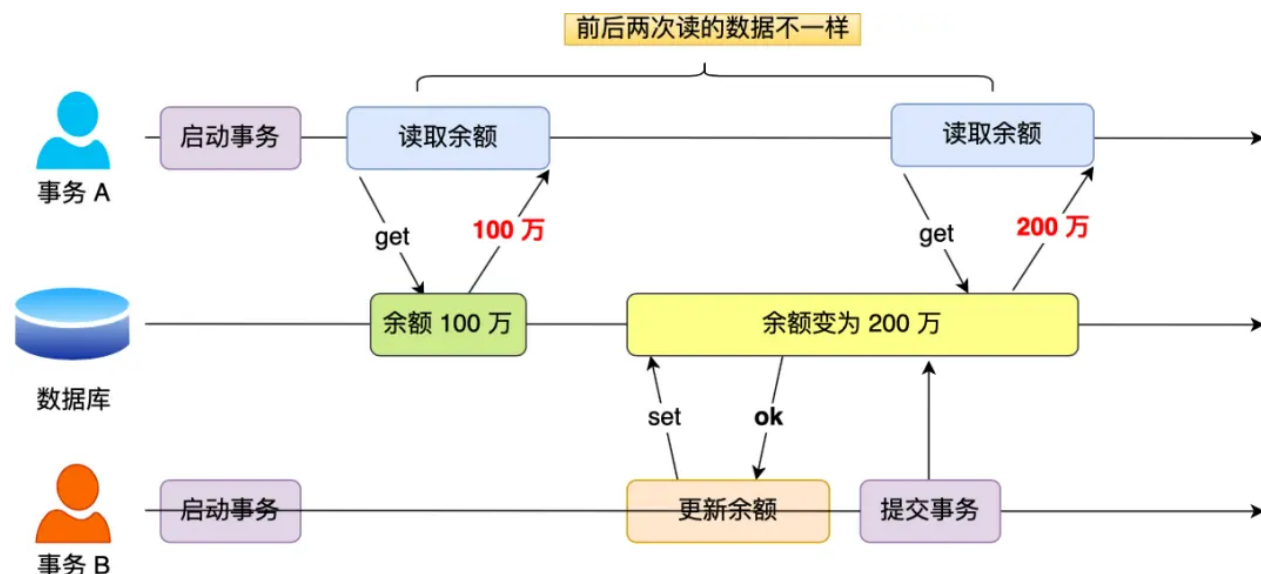
举个栗子，假设有 A 和 B 这两个事务同时在处理，事务 A 先从数据库中读取小林的余额数据，然后再执行更新操作，如果此时事务 A 还没有提交事务，而此时正好事务 B 也从数据库中读取小林的余额数据，那么事务 B 读取到的余额数据是刚才事务 A 更新后的数据，即使没有提交事务。



因为事务 A 是还没提交事务的，也就是它随时可能发生回滚操作，如果在上面这种情况事务 A 发生了回滚，那么事务 B 刚才得到的数据就是过期的数据，这种现象就被称为脏读。

2、不可重复读：在一个事务内多次读取同一个数据，如果出现前后两次读到的数据不一样的情况，就意味着发生了「不可重复读」现象。

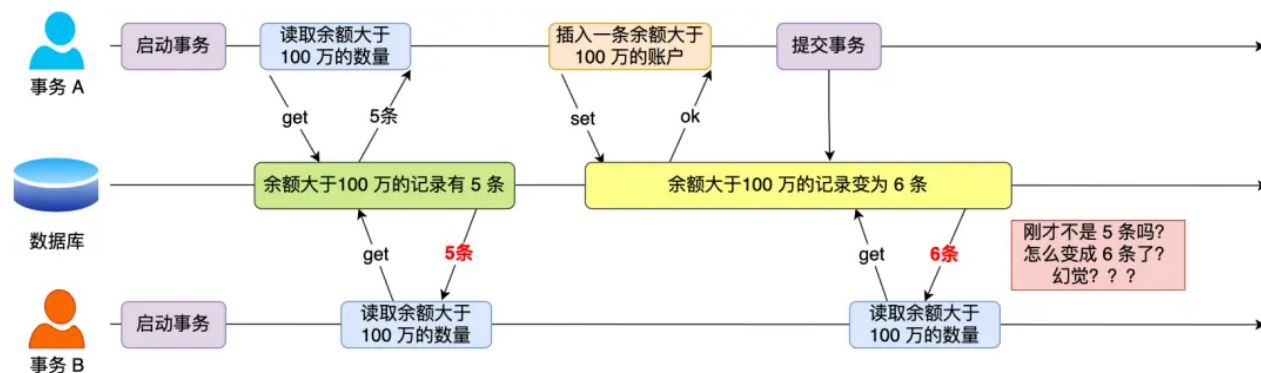
举个例子，假设有 A 和 B 这两个事务同时在处理，事务 A 先从数据库中读取小林的余额数据，然后继续执行代码逻辑处理，在这过程中如果事务 B 更新了这条数据，并提交事务，那么当事务 A 再次读取该数据时，就会发现前后两次读到的数据是不一致的，这种现象就被称为不可重复读。



图片

3、幻读：在一个事务内多次查询某个符合查询条件的「记录数量」，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了「幻读」现象。

举个例子，假设有 A 和 B 这两个事务同时在处理，事务 A 先从数据库查询账户余额大于 100 万的记录，发现共有 5 条，然后事务 B 也按相同的搜索条件也是查询出了 5 条记录。



图片

接下来，事务 A 插入了一条余额超过 100 万的账号，并提交了事务，此时数据库超过 100 万余额的账号个数就变为 6。然后事务 B 再次查询账户余额大于 100 万的记录，此时查询到的记录数量有 6 条，发现和前一次读到的记录数量不一样了，就感觉发生了幻觉一样，这种现象就被称为幻读。

ps：《事务隔离级别是怎么实现的？》完整详细讲解：<https://xiaolincoding.com/mysql/transaction/mvcc.html>

通过什么隔离级别解决这些问题

脏读：读写已提交

不可重复读：可重复读

幻读：串行化

小林补充：

针对不同的隔离级别，并发事务时可能发生的现象也会不同。



图片

也就是说：

- 在「读未提交」隔离级别下，可能发生脏读、不可重复读和幻读现象；
- 在「读提交」隔离级别下，可能发生不可重复读和幻读现象，但是不可能发生脏读现象；
- 在「可重复读」隔离级别下，可能发生幻读现象，但是不可能脏读和不可重复读现象；
- 在「串行化」隔离级别下，脏读、不可重复读和幻读现象都不可能会发生。

所以，要解决脏读现象，就要升级到「读提交」以上的隔离级别；要解决不可重复读现象，就要升级到「可重复读」的隔离级别，要解决幻读现象不建议将隔离级别升级到「串行化」。

不同的数据库厂商对 SQL 标准中规定的 4 种隔离级别的支持不一样，有的数据库只实现了其中几种隔离级别，我们讨论的 **MySQL** 虽然支持 4 种隔离级别，但是与 **SQL** 标准中规定的各级隔离级别允许发生的现象却有些出入。

MySQL 在「可重复读」隔离级别下，可以很大程度上避免幻读现象的发生（注意是很大程度避免，并不是彻底避免），所以 MySQL 并不会使用「串行化」隔离级别来避免幻读现象的发生，因为使用「串行化」隔离级别会影响性能。

ps：《事务隔离级别是怎么实现的？》完整详细讲解：<https://xiaolincoding.com/mysql/transaction/mvcc.html>

mysql的隔离级别是什么?mysql是如何实现的?

不可重复读，但是很大程度上避免幻读

- 快照读（只读）：MVCC
- 当前读（更新操作）：记录锁+间隙锁

小林补充：

MySQL InnoDB 引擎的默认隔离级别虽然是「可重复读」，但是它很大程度上避免幻读现象（并不是完全解决了），解决的方案有两种：

- 针对**快照读**（普通 select 语句），是通过 **MVCC** 方式解决了幻读，因为可重复读隔离级别下，事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，即使中途有其他事务插入了一条数据，是查询不出来这条数据的，所以就很好了避免幻读问题。
- 针对**当前读**（select ... for update 等语句），是通过 **next-key lock**（记录锁+间隙锁）方式解决了幻读，因为当执行 select ... for update 语句的时候，会加上 next-key lock，如果有其他事务在 next-key lock 锁范围内插入了一条记录，那么这个插入语句就会被阻塞，无法成功插入，所以就很好了避免幻读问题。

ps：《事务隔离级别是怎么实现的？》完整详细讲解：<https://xiaolincoding.com/mysql/transaction/mvcc.html>

算法

写一个数据库的多表联查问题

没写出来（平时写的少，只知道命令，不熟练），讲解下思路

反问

1、咱们部门具体做什么

2、进来需要转语言吗

面试总结

1. 一定要补足sql方面的知识，多写一写
2. 计网的知识回答的太模糊
3. 多看java并发相关的知识
4. 平时还是要积累，多问问为什么



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

腾讯测开，也不简单啊...

大家好，我是小林。

很多同学觉得后端岗位太卷，都会去投一下测开岗位，但是不知道如何准备测开。

其实测开的面试和后端差不多的，一样是八股文+项目+算法，所以直接按后端开发的方向准备就行了。

今天分享一位同学腾讯测开的面试，腾讯测开的部门是用 go 语言，面试者是 java 技术栈。

这场面试，最后还问了一个智力题，果然是腾讯的风格，目前只发现腾讯爱问这类题目，主要是考察面试者聪明不聪明。

问题记录

自我介绍环节

略

对go了解吗?部门主要是偏go语言的

答：说了之前学过一段时间,但是太长时间没有接触了,如果给一个礼拜的话就可以将相关知识捡回来

项目问答:说的是项目中写了用 Redis 提高了QPS,那么如果QPS提高了,用户客户端无限制地请求服务器,那么会导致Redis的连接打满吗?比如说能承载的QPS极限是10W,来了12W的请求,该怎么办呢?

说了使用限流的思路,可以使用其他中间件,比如说使用令牌桶,每秒钟以一定的速率放入令牌,然后只有取到令牌的请求线程才能执行秒杀操作,否则的话就可以让这个线程暂停,或者是执行自动重试、退出。

面试官补充:可以设置一个队列,就是将获得令牌失败的线程放入到一个队列中,然后的等待令牌有空余的时候,这时候就唤醒线程去重新竞争令牌,同时要考虑队列是否会产生内存溢出问题

项目问答:Redis中有遇到并发竞争问题吗?

答:说了秒杀优惠券的引发的并发竞争的问题,引出分布式锁,通过分布式锁来解决并发竞争问题,说了使用Redisson来实现这个分布式锁。

项目问答:分布式锁的实现是怎么样的?

回答:

- 1.互斥,只有一个线程能够在一个时间段内操作这个线程,通过setnx实现。
- 2.可重入,可以通过一个设置一个hash结构,这个哈希结构是这样设置的 {线程ID}=>{锁的重入次数},当线程到来的时候就判断有没有这个数据,如果有这个数据的话,那么就检查线程ID是否和自己的ID一样,如果一样的话就锁重试次数++,当重入结束的时候,就让这个锁的重入次数--,当锁的重入次数==0的时候,删除这个键值对。
- 3.实现锁的自动续期,通过看门狗机制,这个点有点记不清了,于是凭着印象简单说了一下,就是开辟一个守护线程,然后这个守护线程定期去更新锁的过期时间,当业务没有结束的时候就一直更新这个锁的过期时间
- 4.实现锁的自动重试,当线程获取锁失败后,可以让线程自旋等待锁
- 5.防止死锁,设置锁的过期时间

项目问答:守护线程是什么意思?

答:就是为了防止锁在业务没有执行完成后就释放掉了,开启一个线程来定期对这把锁进行延期操作

项目问答:假设这样一个场景,就是业务线程挂掉了,然后守护线程一直还在更新这把锁的延期时间,会怎么样?怎么办?(不会)

答:猜的....从业务线程的角度回答,可以通过语言的特性,就是try-catch-finally的特性,来保证这把锁一定会释放,从守护线程的角度来看,可以通过统计这个业务的执行时长,定义一个最长的业务执行时间,如果在这个时间内一直还在更新这把锁的过期时间,那么就释放这个锁,然后上报业务层

补充:

业务线程挂了分情况。一种是整个程序被干掉了,比如掉线了。一个情况是程序出现bug了,导致业务线程在执行解锁逻辑之前被干掉了,或者说业务线程死循环了、忘记解锁了这类情况,都归属于bug。

第一种情况,程序被干掉了,那么守护线程也没了,所以不会续期了,时间到了就释放锁,这个没问题。

第二种情况,根据看门狗机制,它就是会无限续期。相当于变成了一个死锁。这是由它的工作原理决定的,无解。但是可以自己魔改一下看门狗机制,比如设定为续期1000次后还要续期,就有可能出问题了,那就释放锁。但是这个方案,聊胜于无。还不如设置一个较长的过期时间。

我觉得这种情况不应该考虑怎么改进看门狗机制,而是应该考虑怎么监控它是否在正常运行。比如续期了1000次还在续期,就发个预警出来,人工看看啥情况,然后具体情况具体分析,是bug就修bug,是正常运行就先不管。人工一介入,就没有啥不能解决的。

信号量的工作原理是什么？

答:从底层的数据结构来说,信号量包含两个字段,第一个字段是这个信号量代表的资源个数,当 >0 的时候,比如说是 n ,那么就代表说有 n 个资源可用,当 <0 ,代表说有 n 个线程因为这个资源被阻塞了,第二个字段是阻塞队列,当有线程释放资源的时候,就会唤醒这个阻塞队列中的线程。

补充:

信号量是操作系统提供了一种协调共享资源访问的方法。

通常信号量表示资源的数量,对应的变量是一个整型 (`sem`) 变量。

另外,还有两个原子操作的系统调用函数来控制信号量的, 分别是:

- *P* 操作: 将 `sem` 减 1, 相减后, 如果 `sem < 0`, 则进程/线程进入阻塞等待, 否则继续, 表明 *P* 操作可能会阻塞;
- *V* 操作: 将 `sem` 加 1, 相加后, 如果 `sem <= 0`, 唤醒一个等待中的进程/线程, 表明 *V* 操作不会阻塞;

很多人问, *V* 操作 中 `sem <= 0` 的判断是不是写反了?

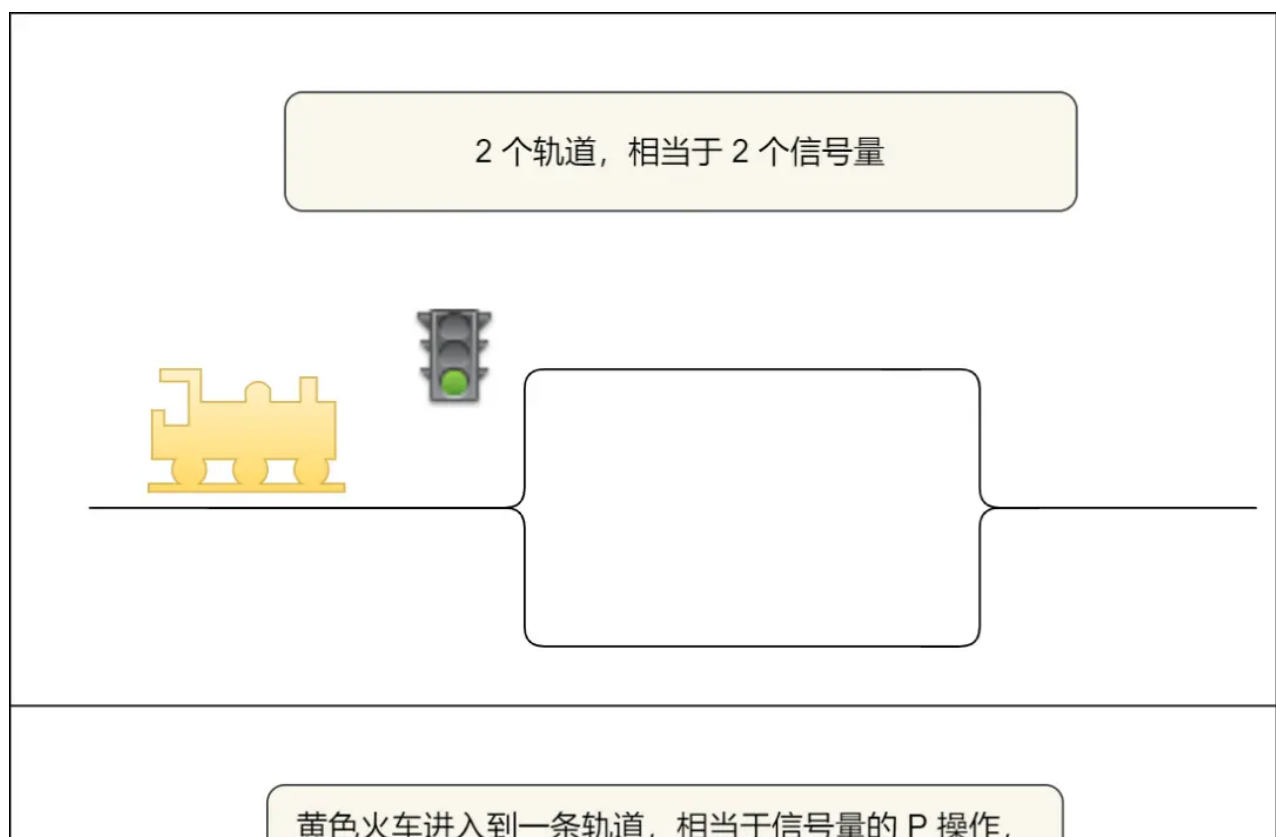
没写反, 我举个例子, 如果 `sem = 1`, 有三个线程进行了 *P* 操作:

- 第一个线程 *P* 操作后, `sem = 0`;
- 第二个线程 *P* 操作后, `sem = -1`;
- 第三个线程 *P* 操作后, `sem = -2`;

这时, 第一个线程执行 *V* 操作后, `sem` 是 -1 , 因为 `sem <= 0`, 所以要唤醒第二或第三个线程。

P 操作是用在进入临界区之前, *V* 操作是用在离开临界区之后, 这两个操作是必须成对出现的。

举个类比, 2 个资源的信号量, 相当于 2 条火车轨道, *PV* 操作如下图过程:



资源 - 1, 于是就只剩一个轨道了



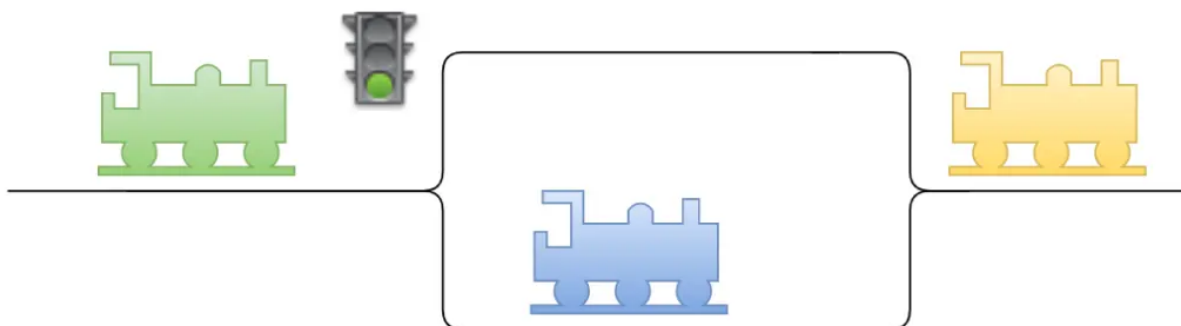
此时又来了蓝色火车, 由于还有一条轨道未被占用, 所以交通灯依然是绿灯



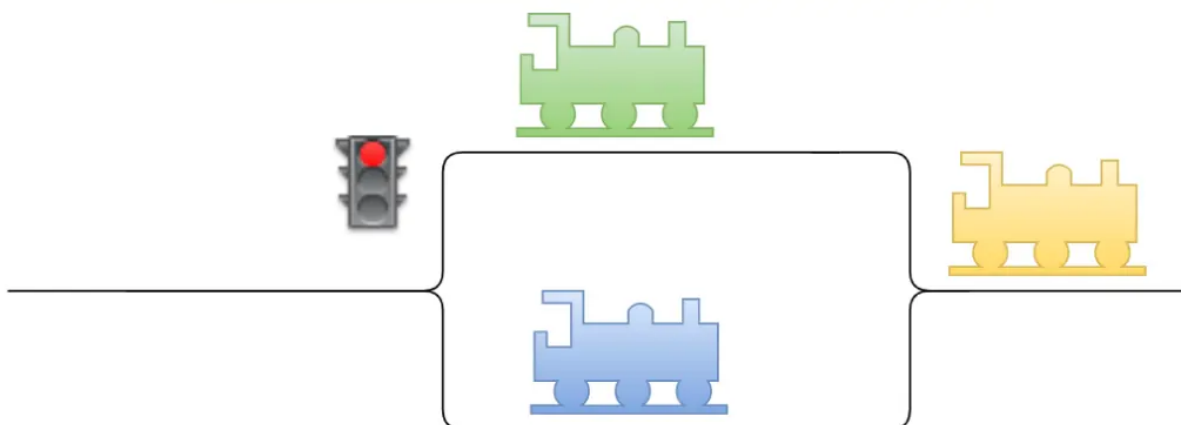
接着, 蓝色火车占用了另外的轨道, 也就是 P 操作, 此时交通灯应变为红灯, 因为没有轨道可以使用了后续来的绿色火车必须要等待。



黄色火车离开轨道后，相当于 V 操作，此时轨道资源就变为 1，交通灯就亮起绿灯



绿色火车发现亮起绿灯，于是就进入火车轨道，由于轨道资源被耗尽为 0，于是交通灯就亮起了红灯



信号量与火车轨道

操作系统是如何实现 PV 操作的呢？

信号量数据结构与 PV 操作的算法描述如下图：

```

// 信号量数据结构
type struct sem_t{
    int sem;        // 资源个数
    queue_t *q;     // 等待队列
} sem_t;

// 初始化信号量
void init(sem_t *s, int sem)
{
    s->sem = sem;
    queue_init(s->q);
}

// P 操作
void P(sem_t *s)
{
    s->sem--;
    if(s->sem < 0)
    {
        1. 保留调用线程 CPU 现场;
        2. 将该线程的 TCB 插入到 s 的等待队列;
        3. 设置该线程为等待状态;
        4. 执行调度程序;
    }
}

// V 操作
void V(sem_t *s)
{
    s->sem++;
    if(s->sem <= 0)
    {
        1. 移出 s 等待队列首元素;
        2. 将该线程的 TCB 插入就绪队列;
        3. 设置该线程为「就绪」状态;
    }
}

```

PV 操作的算法描述

PV 操作的函数是由操作系统管理和实现的，所以操作系统已经使得执行 PV 函数时是具有原子性的。

为什么有HTTP协议了?还要用RPC?

答:1.从项目实践的角度上来讲,使用HTTP协议规定的协议码对于RPC框架提供的异常信息返回的角度来说,HTTP所蕴含的信息不够丰富,不够直观,比如说通信双方的执行状态只能够通过状态码来描述,而RPC框架可以返回完整的异常信息

2.从编码的角度上来看,HTTP的编码比较冗余,也就是说需要通过定义一大堆的状态码来定义当前的状态,对于不同的服务,这些编码可能还有不同的含义,不合适

3.HTTP通常来说前端和后端通信使用的,包含了大量的浏览器跳转状态定义,这些状态对于后端服务器之间的联调是没有意义的,因此在这样的情况下,而RPC自定义的协议,可以免去这些冗余的状态,更加定制化地是设定这个后端服务联调

补充:

- RPC 本质上不算是协议,而是一种调用方式,而像 gRPC 和 Thrift 这样的具体实现,才是协议,它们是实现了 RPC 调用的协议。目的是希望程序员能像调用本地方法那样去调用远端的服务方法。同时 RPC 有很多种实现方式,不一定非得基于 TCP 协议。
- 从发展历史来说,HTTP 主要用于 B/S 架构,而 RPC 更多用于 C/S 架构。但现在其实已经没分那么清了,B/S 和 C/S 在慢慢融合。很多软件同时支持多端,所以对外一般用 HTTP 协议,而内部集群的微服务之间则采用 RPC 协议进行通讯。
- RPC 其实比 HTTP 出现的要早,且比目前主流的 HTTP/1.1 性能要更好,所以大部分公司内部都还在使用 RPC。
- HTTP/2.0在 HTTP/1.1的基础上做了优化,性能可能比很多 RPC 协议都要好,但由于是这几年才出来的,所以也不太可能取代掉 RPC。

go的map底层原理是什么?

答:不会...问面试官能不能说Java的HashMap,说可以。

HashMap的底层是基于数组+链表/红黑树来实现的,其中数组是 HashMap 的主体,而链表和红黑树是为了解决哈希冲突而实现的,哈希冲突指的是多个不同的key,通过hash()计算出来的index是一致的,为了解决这个问题,可以使用拉链法来实现,也就是发现通过key计算出来的位置有元素的时候,这时候就将 {value} 插入到这个下标上的链表或者红黑树,除此之外,还有再哈希法,开放寻址法等方法来解决这个问题

写代码中用到并发吗?

答:说了项目中用到线程池

Git用过吗?Git上有敏感数据吗?比如说docker-compose或者是脚本文件中有敏感信息,你怎么来管理?

答:说了通过本地配置文件,比如说 docker-compose 中配置密钥资源的时候,将敏感数据部署在物理机本地,然后在文件内部通过引用的方式来引用这个密钥,这样的话就可以避免密钥上传到git上,导致泄露,还有的话就是可以通过合理配置权限,具体来说就是在 docker-compose 的目录中配置一个 .ini 文件,然后将键值对,例如 MYSQL_PASSWORD=123456 填入到这个文件中,然后在 docker-compose 中通过 \$MYSQL_PASSWORD 来引用这个数据,因此 docker-compose 这些要上git的数据就不会部署到git上了

知道测试开发是怎么做的吗?

答:

- 1.写测试用例,设计测试用例
- 2.开发测试工具

Linux操作过吗?说一说常见的指令

答:

- 1.文件相关(mv mkdir cd ls)
- 2.docker相关(docker container ls docker ps -a)
- 3.权限相关(chmod chown useradd groupadd)
- 4.网络相关(netstat ip addr)
- 5.测试相关(测试连通性:ping 测试端口连通性:telnet)

智力题

屋里四盏灯，屋外四个开关，一个开关仅控制一盏灯，屋外看不到屋里，怎样只进屋一次，就知道哪个开关控制哪盏

chatgpt 4.0 的回答：

为了找出哪个开关控制哪盏灯，你可以按照以下步骤操作：

1. 先打开第一个开关，等待几分钟。
2. 然后关闭第一个开关，打开第二个开关。
3. 进入屋内。

现在你会看到以下情况：

- 一盏灯亮着：这盏灯是由第二个开关控制的。
- 一盏灯熄灭，但灯泡发热：这盏灯是由第一个开关控制的，因为它之前亮着但现在已经关闭。
- 剩下两盏灯熄灭且不发热：分别由第三个和第四个开关控制。

为了确定第三个和第四个开关分别控制哪盏灯，只需在屋内尝试打开其中一个。如果这盏灯亮起，那么就知道它由当前打开的开关控制，另一盏灯则由未尝试的开关控制。这样就可以在进屋一次的情况下，找出四个开关分别控制哪盏灯。

算法题

10w个字的的文章,都是小写字母,求其中出现次数最多的字母

答：统计词频+排序+输出

反问

部门技术栈是什么?测开具体做啥?

面试总结

感觉：

- 有些问题回答得不是很清晰,对项目还是不太熟

不足之处：

- 智力题没什么思路,应该还要继续看相关的题目

米哈游稳住了！问的很基础！

大家好，我是小林。

五一收假了，又要卷起来了。

今天分享米哈游的春招实习面经，主要考察了java+操作系统+mysql+网络，这四个方面。

面试流程，共1小时，1min自我介绍，20min写题，剩下问题基础知识。

Java

String，StringBuilder，StringBuffer区别？单线程大量操作字符串用哪个？

回答：用StringBuilder

补充：

String、StringBuilder和StringBuffer都是Java中用于操作字符串的类。

String是不可变的字符序列，每次对String进行修改时都会创建一个新的String对象，因此在大量操作字符串时，使用String会频繁地创建对象，导致性能较低。

StringBuilder和StringBuffer都是可变的字符序列，可以对其进行多次修改而不创建新的对象。两者的区别在于线程安全性，StringBuffer是线程安全的，而StringBuilder是非线程安全的。因为StringBuffer的所有共有方法都是同步的，所以在多线程环境下使用StringBuffer可以保证线程安全，但是会降低性能。而StringBuilder没有同步方法，所以在单线程环境下使用StringBuilder性能更高。

因此，在单线程环境下进行大量的字符串操作时，应该使用StringBuilder，可以获得更好的性能。在多线程环境下，使用StringBuffer可以保证线程安全，但是会牺牲一定的性能。

综上所述，单线程大量操作字符串时应该使用StringBuilder，而在多线程环境下应该使用StringBuffer。

synchronized偏向锁直接升级为重量级锁吗？重量级锁怎么实现的？

回答：扯了一下synchronized四种锁

补充：

偏向锁不会直接升级为重量级锁，而是会先升级为轻量级锁，如果轻量级锁竞争失败，则再升级为重量级锁。

重量级锁的实现一般是通过操作系统的互斥量（mutex）来实现的。当一个线程获取重量级锁时，会将该线程挂起，直到锁被释放。这种锁的性能比较低，因为每次加锁和释放锁都需要涉及到操作系统的系统调用，会有较大的开销。因此，在实际应用中，要尽量避免使用重量级锁。

Java中的异常

回答：分类不大记得。讲了异常的捕获，from\to\target指针

补充：

当程序出现异常时，Java会抛出一个异常对象。Java中的异常可以分为三类：

1. Checked Exception（受检异常）：这种异常在编译时就可以被检测出来，必须要在代码中进行处理或者声明抛出，否则编译不通过。这类异常主要是由程序的外部环境引起的，例如文件不存在、网络连接失败等。常见的Checked Exception包括IOException、SQLException等。
2. Unchecked Exception（非受检异常）：这种异常通常是由程序内部错误引起的，例如NullPointerException、ArrayIndexOutOfBoundsException、IllegalArgumentException等。这类异常不需要在代码中声明抛出，也可以不进行处理，但是如果进行处理，程序会崩溃。
3. Error：这种异常通常是由JVM或者硬件引起的，例如OutOfMemoryError、StackOverflowError等。这类异常也不需要再在代码中声明抛出，也可以不进行处理，但是如果进行处理，程序也会崩溃。

在Java中，异常处理通常包括try-catch语句和throw语句。try-catch语句可以捕获异常并进行处理，而throw语句可以手动抛出异常。

操作系统

进程间的通信方式？管道模型的分类？

最简单的方式就是**管道**，管道分为「匿名管道」和「命名管道」。

- **匿名管道**顾名思义，它没有名字标识，匿名管道是特殊文件只存在于内存，没有存在于文件系统中，shell 命令中的「|」竖线就是匿名管道，通信的数据是无格式的流并且大小受限，通信的方式是单向的，数据只能在一个方向上流动，如果要双向通信，需要创建两个管道，再来匿名管道是只能用于存在父子关系的进程间通信，匿名管道的生命周期随着进程创建而建立，随着进程终止而消失。
- **命名管道**突破了匿名管道只能在亲缘关系进程间的通信限制，因为使用命名管道的前提，需要在文件系统创建一个类型为 p 的设备文件，那么毫无关系的进程就可以通过这个设备文件进行通信。另外，不管是匿名管道还是命名管道，进程写入的数据都是缓存在内核中，另一个进程读取数据时候自然也是从内核中获取，同时通信数据都遵循先进先出原则。

消息队列克服了管道通信的数据是无格式的字节流的问题，消息队列实际上是保存在内核的「消息链表」，消息队列的消息体是可以用户自定义的数据类型，发送数据时，会被分成一个一个独立的消息体，当然接收数据时，也要与发送方发送的消息体的数据类型保持一致，这样才能保证读取的数据是正确的。消息队列通信的速度不是最及时的，毕竟每次数据的写入和读取都需要经过用户态与内核态之间的拷贝过程。

共享内存可以解决消息队列通信中用户态与内核态之间数据拷贝过程带来的开销，它直接分配一个共享空间，每个进程都可以直接访问，就像访问进程自己的空间一样快捷方便，不需要陷入内核态或者系统调用，大大提高了通信的速度，享有**最快的进程间通信方式之名**。但是便捷高效的共享内存通信，带来新的问题，多进程竞争同个共享资源会造成数据的错乱。

那么，就需要**信号量**来保护共享资源，以确保任何时候只能有一个进程访问共享资源，这种方式就是互斥访问。信号量不仅可以实现访问的互斥性，还可以实现进程间的同步，信号量其实是一个计数器，表示的是资源个数，其值可以通过两个原子操作来控制，分别是 P 操作和 V 操作。

与信号量名字很相似的叫**信号**，它俩名字虽然相似，但功能一点儿都不一样。信号可以在应用进程和内核之间直接交互，内核也可以利用信号来通知用户空间的进程发生了哪些系统事件，信号事件的来源主要有硬件来源（如键盘 Ctrl+C）和软件来源（如 kill 命令），一旦有信号发生，进程有三种方式响应信号 1. 执行默认操作、2. 捕捉信号、3. 忽略信号。有两个信号是应用进程无法捕捉和忽略的，即 `SIGKILL` 和 `SIGSTOP`，这是为了方便我们能在任何时候结束或停止某个进程。

前面说到的通信机制，都是工作于同一台主机，如果要与不同主机的进程间通信，那么就需要 **Socket 通信**了。Socket 实际上不仅用于不同的主机进程间通信，还可以用于本地主机进程间通信，可根据创建 Socket 的类型不同，分为三种常见的通信方式，一个是基于 TCP 协议的通信方式，一个是基于 UDP 协议的通信方式，一个是本地进程间通信方式。

内核态和用户态区别？内核态的底层操作有什么？为什么要分两个不同的态？

内核态和用户态是操作系统中的两种运行模式。它们的主要区别在于权限和可执行的操作：

1. 内核态（Kernel Mode）：在内核态下，CPU 可以执行所有的指令和访问所有的硬件资源。这种模式下的操作具有更高的权限，主要用于操作系统内核的运行。
2. 用户态（User Mode）：在用户态下，CPU 只能执行部分指令集，无法直接访问硬件资源。这种模式下的操作权限较低，主要用于运行用户程序。

内核态的底层操作主要包括：内存管理、进程管理、设备驱动程序控制、系统调用等。这些操作涉及到操作系统的核心功能，需要较高的权限来执行。

分为内核态和用户态的原因主要有以下几点：

1. 安全性：通过对权限的划分，用户程序无法直接访问硬件资源，从而避免了恶意程序对系统资源的破坏。
2. 稳定性：用户态程序出现问题时，不会影响到整个系统，避免了程序故障导致系统崩溃的风险。
3. 隔离性：内核态和用户态的划分使得操作系统内核与用户程序之间有了明确的边界，有利于系统的模块化和维护。

内核态和用户态的划分有助于保证操作系统的安全性、稳定性和易维护性。

mysql

一条语句，问怎么加索引比较好？

在 MySQL 中，创建索引可以提高查询性能。为了给某个列添加索引，我们可以使用以下语句：

```
CREATE INDEX index_name ON table_name(column_name);
```

在这里，`index_name` 是你为索引指定的名称，`table_name` 是你要添加索引的表名，而 `column_name` 是你要添加索引的列名。

在选择要添加索引的列时，请考虑以下几点：

1. 对于经常用于查询条件的列，添加索引可以提高查询速度。
2. 对于具有许多重复值的列，添加索引的性能提升可能不明显。
3. 尽量避免在非常大的表上创建过多索引，因为这会影响插入和更新操作的性能。

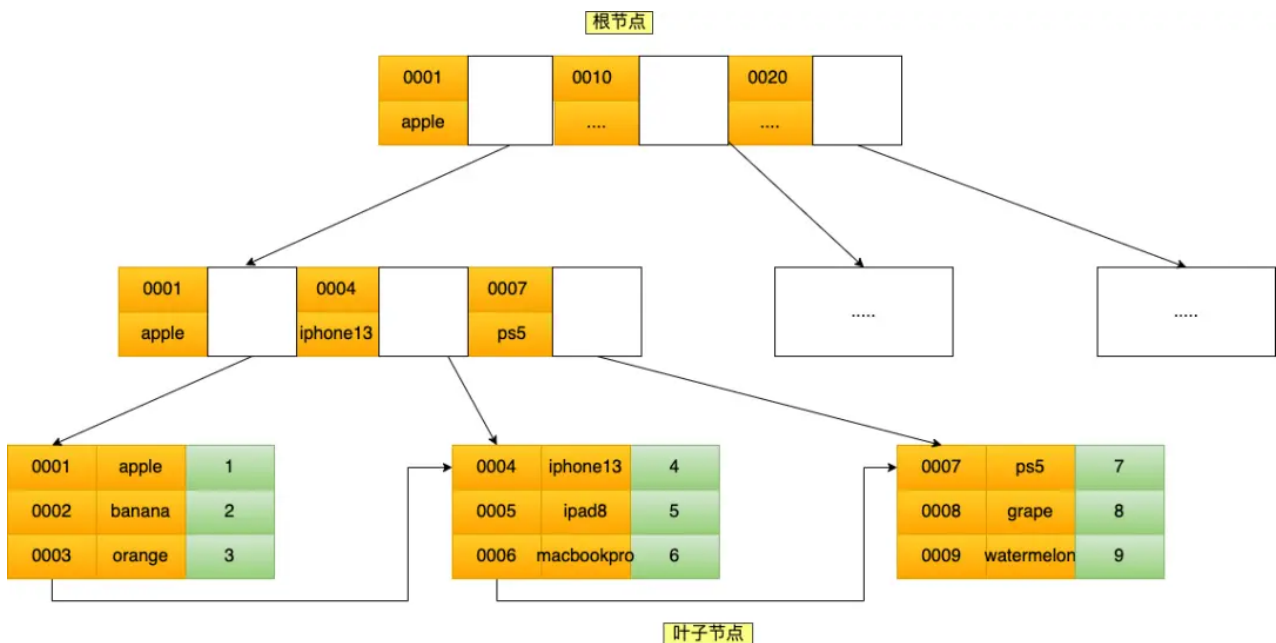
什么是联合索引？

通过将多个字段组合成一个索引，该索引就被称为联合索引。

比如，将商品表中的 product_no 和 name 字段组合成联合索引 (product_no, name)，创建联合索引的方式如下：

```
CREATE INDEX index_product_no_name ON product(product_no, name);
```

联合索引 (product_no, name) 的 B+Tree 示意图如下（图中叶子节点之间我画了单向链表，但是实际上是双向链表，原图我找不到了，修改不了，偷个懒我不重画了，大家脑补成双向链表就行）。



联合索引

可以看到，联合索引的非叶子节点用两个字段的值作为 B+Tree 的 key 值。当在联合索引查询数据时，先按 product_no 字段比较，在 product_no 相同的情况下再按 name 字段比较。

也就是说，联合索引查询的 B+Tree 是先按 product_no 进行排序，然后再 product_no 相同的情况再按 name 字段排序。

因此，使用联合索引时，存在**最左匹配原则**，也就是按照最左优先的方式进行索引的匹配。在使用联合索引进行查询的时候，如果不遵循「最左匹配原则」，联合索引会失效，这样就无法利用到索引快速查询的特性了。

mysql的四种隔离级别？

- **读未提交 (read uncommitted)**，指一个事务还没提交时，它做的变更就能被其他事务看到；
- **读提交 (read committed)**，指一个事务提交之后，它做的变更才能被其他事务看到；
- **可重复读 (repeatable read)**，指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，MySQL InnoDB 引擎的默认隔离级别；

- **串行化 (serializable)**；会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生了读写冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行；

针对不同的隔离级别，并发事务时可能发生的现象也会不同。



图片

也就是说：

- 在「读未提交」隔离级别下，可能发生脏读、不可重复读和幻读现象；
- 在「读提交」隔离级别下，可能发生不可重复读和幻读现象，但是不可能发生脏读现象；
- 在「可重复读」隔离级别下，可能发生幻读现象，但是不可能脏读和不可重复读现象；
- 在「串行化」隔离级别下，脏读、不可重复读和幻读现象都不可能发生。

什么是脏读、幻读、不可重复读？

- 脏读：如果一个事务「读到」了另一个「未提交事务修改过的数据」，就意味着发生了「脏读」现象。
- 不可重复读：在一个事务内多次读取同一个数据，如果出现前后两次读到的数据不一样的情况，就意味着发生了「不可重复读」现象。
- 幻读：在一个事务内多次查询某个符合查询条件的「记录数量」，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了「幻读」现象。

mysql的innodb如何避免不可重复读？

mysql的默认隔离级别是可重复读，可重复读隔离级别在开启事务后，执行第一个select 语句的时候，会生成一个 Read View，后面整个事务期间的select都在用这个 Read View，所以事务期间上读取的数据都是一致的，不会出现前后读取的数据不一致的问题，所以避免了不可重复读。

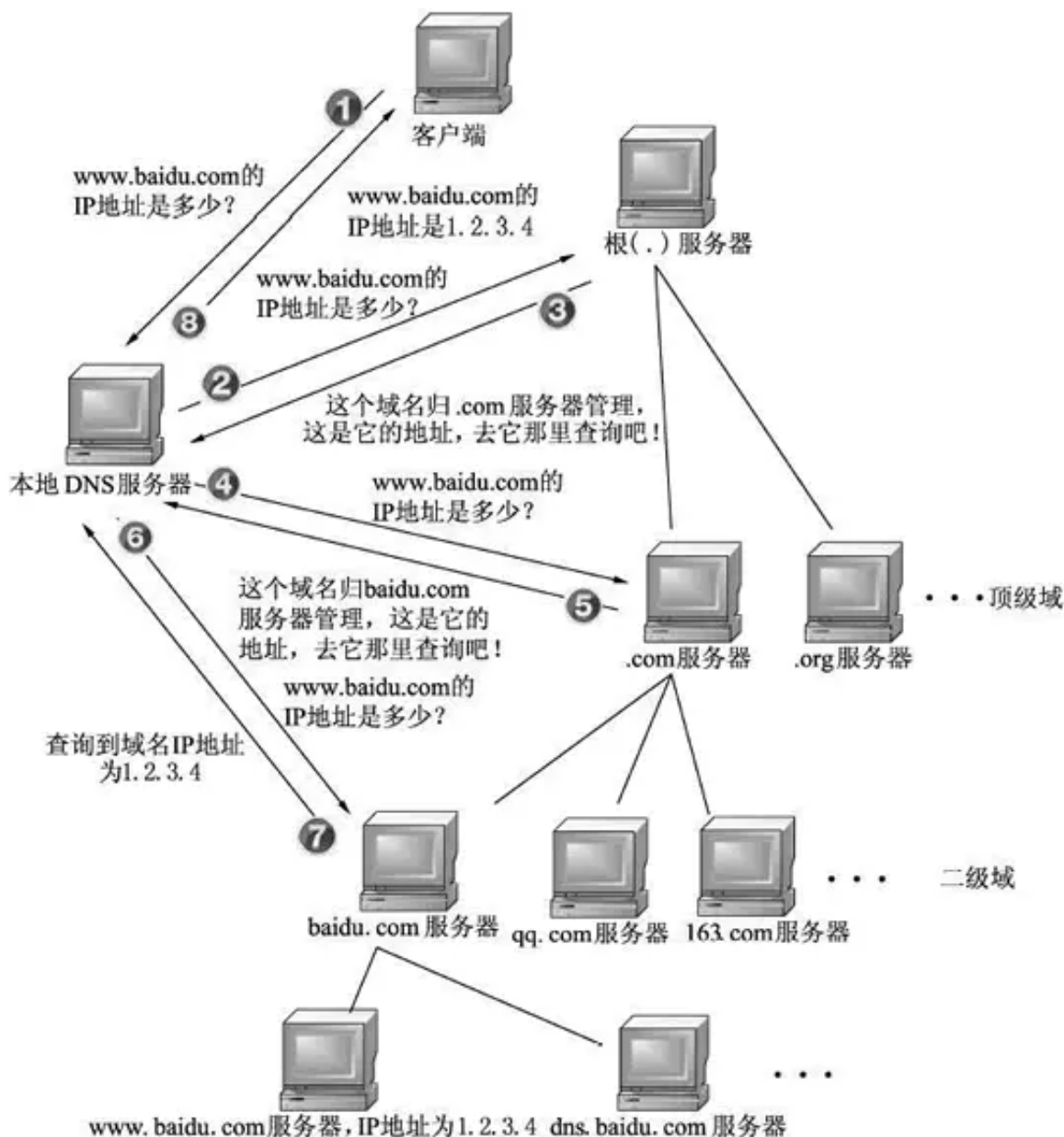
网络

输入网址后发生了什么？

回答：应用层DNS解析，传输层TCP连接，网络层IP，数据链路MAC，真实物理层，接收到之后再一层层扒皮。

DNS解析的具体过程

DNS 域名解析，简单地说就是把域名翻译成 IP 地址。例如：把 www.baidu.com 这个域名翻译成对应 IP 220.181.38.251，这里只是举个例子。



域名解析流程

上图中分 8 个步骤介绍了域名解析的流程，但在此之前会先检查本机的缓存配置+ hosts 解析，然后才真正执行上图的流程：

首先，再进行dns服务器解析之前，会查缓存，总共有两次缓存的查询：

- 浏览器缓存检查：浏览器会首先搜索浏览器自身的 DNS 缓存，缓存时间比较短，大概只有1分钟，且只能容纳 1000 条缓存，看自身的缓存中是否有对应的条目，而且没有过期，如果有且没有过期

则解析到此结束。

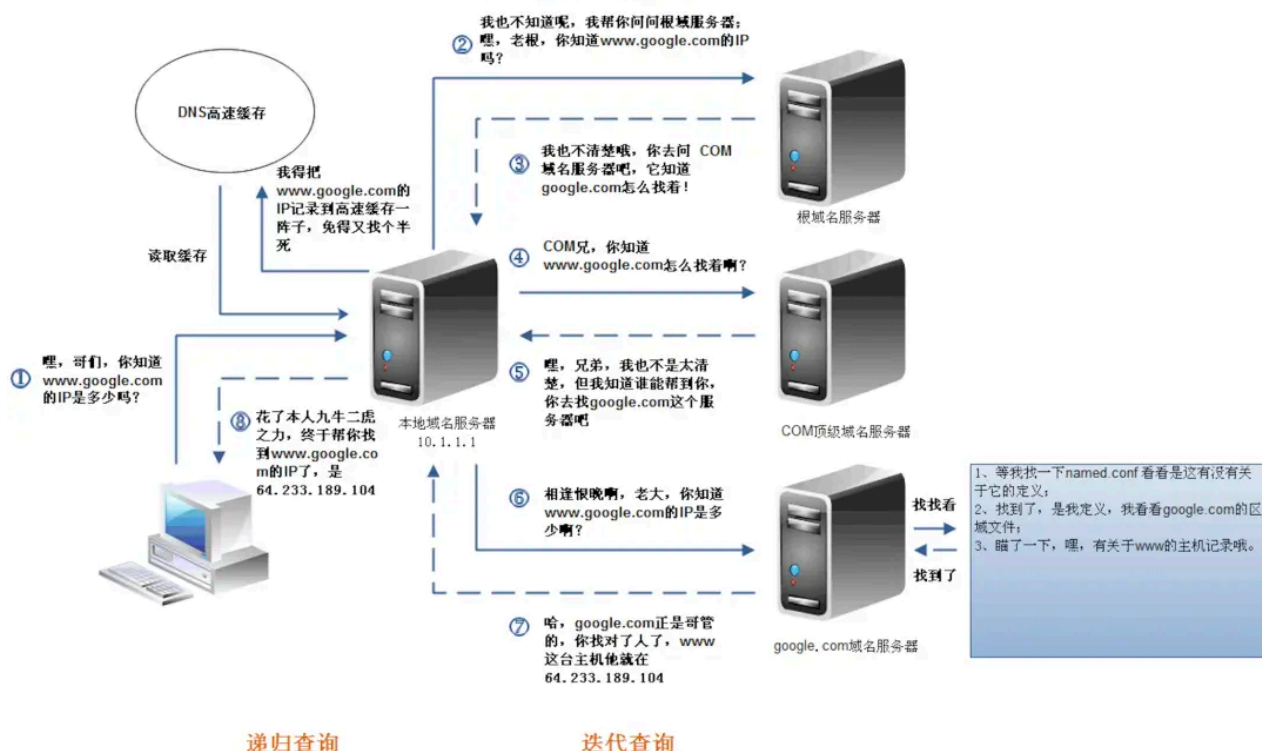
- 操作系统缓存检查 + hosts 解析：如果浏览器的缓存里没有找到对应的条目，操作系统也会有一个域名解析的过程，那么浏览器先搜索操作系统的 DNS 缓存中是否有这个域名对应的解析结果，如果找到且没有过期则停止搜索，解析到此结束。在 Linux 中可以通过 `/etc/hosts` 文件来设置，可以将任何域名解析到任何能够访问的 IP 地址。如果在这里指定了一个域名对应的 IP 地址，那么浏览器会首先使用这个 IP 地址。当解析到这个配置文件中的某个域名时，操作系统会在缓存中缓存这个解析结果，缓存的时间同样是受这个域名的失效时间和缓存的空间大小控制的。

接着就进行dns解析：

- 第一步：客户端通过浏览器访问域名为 www.baidu.com (<http://www.baidu.com>) 的网站，发起查询该域名的 IP 地址的 DNS 请求。该请求发送到了本地 DNS 服务器上。本地 DNS 服务器会首先查询它的缓存记录，如果缓存中有此条记录，就可以直接返回结果。如果没有，本地 DNS 服务器还要向 DNS 根服务器进行查询。
- 第二步：本地 DNS 服务器向根服务器发送 DNS 请求，请求域名为 www.baidu.com (<http://www.baidu.com>) 的 IP 地址。
- 第三步：根服务器经过查询，没有记录该域名及 IP 地址的对应关系。但是会告诉本地 DNS 服务器，可以到域名服务器上继续查询，并给出域名服务器的地址(.com 服务器)。
- 第四步：本地 DNS 服务器向 .com 服务器发送 DNS 请求，请求域名 www.baidu.com (<http://www.baidu.com>) 的 IP 地址。
- 第五步：com 服务器收到请求后，不会直接返回域名和 IP 地址的对应关系，而是告诉本地 DNS 服务器，该域名可以在 [baidu.com](http://www.baidu.com) 域名服务器上解析获取 IP 地址，并告诉 [baidu.com](http://www.baidu.com) 域名服务器的地址。
- 第六步：本地 DNS 服务器向 [baidu.com](http://www.baidu.com) 域名服务器发送 DNS 请求，请求域名 www.baidu.com (<http://www.baidu.com>) 的 IP 地址。
- 第七步：[baidu.com](http://www.baidu.com) 服务器收到请求后，在自己的缓存表中发现了该域名和 IP 地址的对应关系，并将 IP 地址返回给本地 DNS 服务器。
- 第八步：本地 DNS 服务器将获取到与域名对应的 IP 地址返回给客户端，并且将域名和 IP 地址的对应关系保存在缓存中，以备下次别的用户查询时使用。

最后引用大佬的一张图做下总结：

DNS域名解析过程



TCP拆包沾包原因

TCP拆包和沾包现象是由于TCP协议的特性以及网络传输过程中的各种因素所导致的。

1. TCP协议是基于字节流的传输层协议，没有固定的分包边界。发送方将数据分成多个小的数据包进行传输，接收方再将数据包组合成完整的数据。在这个过程中，可能会出现拆包和沾包现象。
2. 网络传输中的延迟和拥塞会影响数据包发送的速度和到达接收方的顺序。这可能导致数据包的拆分和组合不规律，从而出现拆包和沾包现象。
3. 接收方的缓冲区大小限制。当接收方的缓冲区不足以容纳一个完整的数据包时，可能会将数据包拆分成多个部分，导致拆包现象。

为了解决TCP拆包和沾包的问题，可以采用以下方法：

1. 在应用层实现数据包的边界识别，例如通过添加包头，包头中包含数据包长度等信息，使得接收方能够准确地将数据包进行拼接。
2. 使用固定长度的数据包或者特殊的分隔符，以便于接收方识别数据包的边界。
3. 使用更高级的传输层协议，如WebSocket，它在TCP基础上增加了数据帧的概念，可以更好地解决拆包和沾包问题。

算法

- 手写LRU

面试感受

照着简历模块挑八股问，八股也很基础，没问项目，进入二面了

关注作者

新的图解文章都在公众号首发，别忘记关注了哦！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群